

Obrada slike i računalni vid



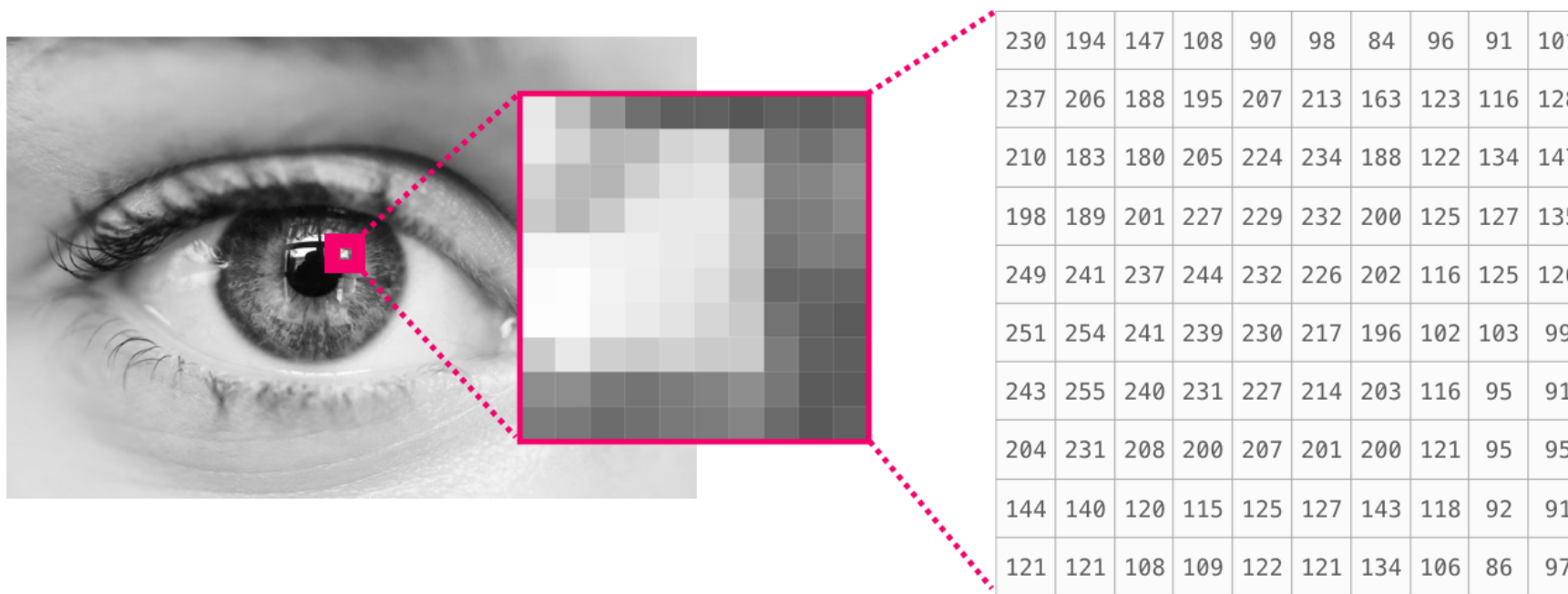
Sadržaj

- Obrada slike
 - Filtriranje slike
 - Geometrijske transformacije
 - Detekcija rubova
- Segmentacija slike
 - Amplitudna segmentacija slike
 - Segmentacija na temelju rubova
- Detekcija objekata
 - Detekcija objekata
 - Izlučivanje značajki
 - Detekcija objekata primjenom značajki

Filtriranje slike

Zapis slike na računalu

- jednobojna (monokromatska slika)
 - prikazuje se samo u nijansama jedne boje, predstavljena je jednom 2D matricom; najčešće slika u nijansama sive boje (engl. *grayscale*)
 - vrijednost piksela je najčešće predstavljena s 8 bita



[Izvor](#)

Zapis slike na računalu

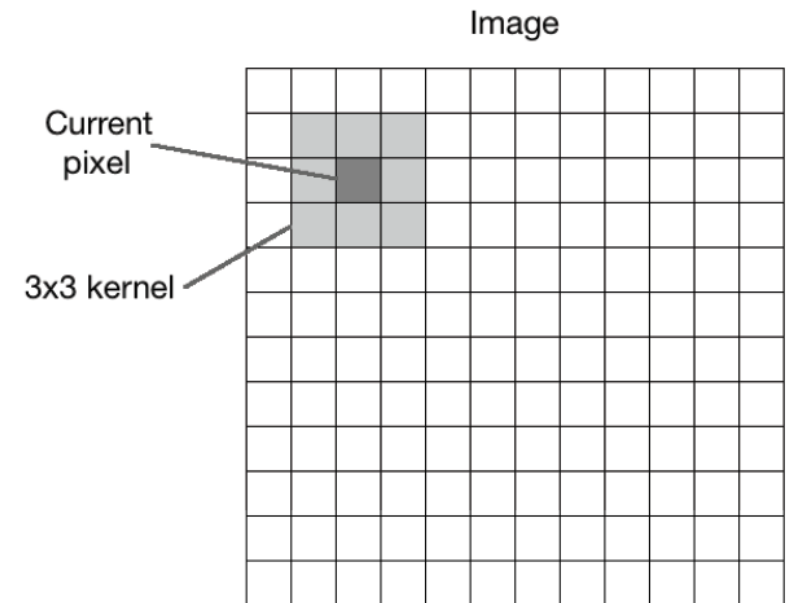
- slika u boji (kromatska slika)
 - za pohranu ovakve slike potrebne su 3 komponente boje, tj. 3 2D matrice
 - svaka piksel ima tri komponente -> najčešće 3 x 8 bita; razni sustavi boja



[Izvor](#)

Linearno filtriranje

- Linearno filtriranje
 - 2D konvolucija
 - vrijednost piksela je određena otežanom sumom ulaznih piksela
 - koristi se kernel s određenim koeficijentima (filar)
 - ovisno o vrijednostima kernela:
 - niskopropusno filtriranje (bluriranje)
 - viskopropusno filtriranje (izoštavanje)
 - pretprocesiranje za detekciju rubova
- Nelinearno filtriranje
 - medijan
 - bilateralni filtar
 - morfološko filtriranje
 - ...



Linearno filtriranje

a	b	c
d	e	f
g	h	i

P _{0,0}	P _{1,0}	P _{2,0}	P _{3,0}	P _{4,0}					
P _{0,1}	P _{1,1}	P _{2,1}	P _{3,1}	P _{4,1}					
P _{0,2}	P _{1,2}	P _{2,2}	P _{3,2}	P _{4,2}					
P _{0,3}	P _{1,3}	P _{2,3}	P _{3,3}	P _{4,3}					

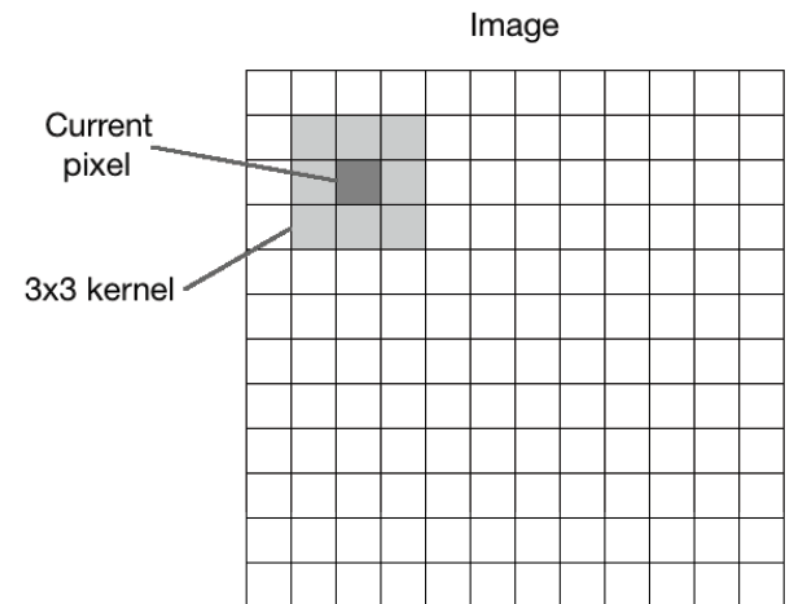
$$P_{2,1}' = a \cdot P_{1,0} + b \cdot P_{2,0} + c \cdot P_{3,0} + d \cdot P_{1,1} + e \cdot P_{2,1} + f \cdot P_{3,1} + g \cdot P_{1,2} + h \cdot P_{2,2} + i \cdot P_{3,2}$$

Pitanje

- Što ako na sliku (u sivim tonovima) primijenimo ovakav kernel?

$$I = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

- Što kada bi u sredini bio broj 2?



Niskopropusno filtriranje

- Niskopropusno filtriranje – guši visoke prostorne frekvencije (detalji i rubovi u slici), propušta samo niske prostorne frekvencije
- **Omekšava oštre rubove, uklanja dio detalja iz slike (zamuti ih)**
- Primjeri:

- ***Moving average***

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

- **Gaussov filter**

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Visokopropusno filtriranje

- postoje različiti visokopropusni filtri
- Koristi se za izoštravanje slike → primjer maske filtra (konvolucijske matrice)

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 9 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

- Što se događa ako zamijenimo središnji element s brojem 8?

Filtriranje primjeri

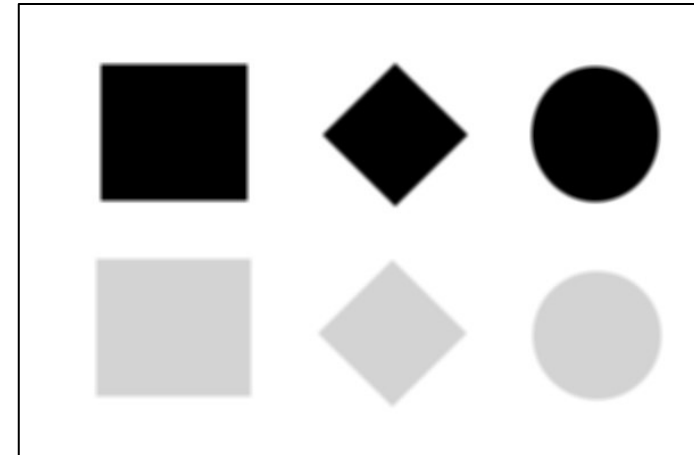
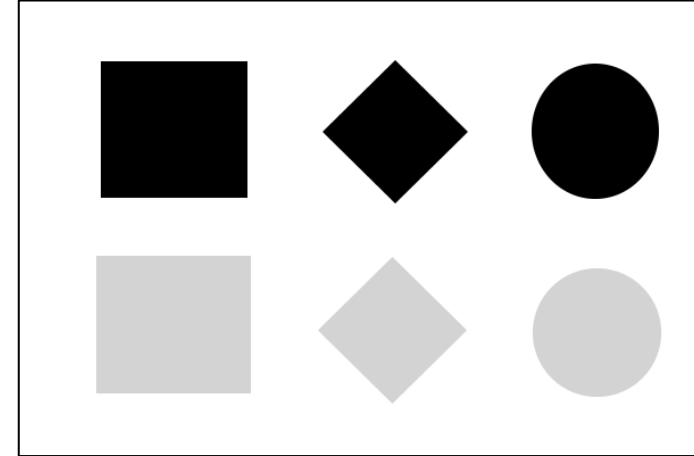
```
import numpy as np
import cv2

# učitaj sliku
img_src = cv2.imread('primjer.jpg')

# niskopropusni (box) filter – moving average
kernel = np.array( [[1, 1, 1, 1, 1],
                    [1, 1, 1, 1, 1],
                    [1, 1, 1, 1, 1],
                    [1, 1, 1, 1, 1],
                    [1, 1, 1, 1, 1]], np.float32)
kernel = kernel/25.0

# filtriraj sliku
img_rst = cv2.filter2D(img_src,-1,kernel)

# spremi rezultat
cv2.imwrite('rezultat_np.jpg',img_rst)
```



Filtriranje primjeri

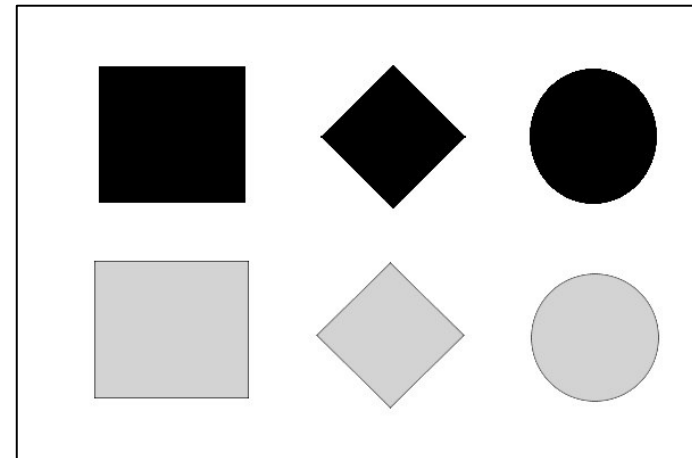
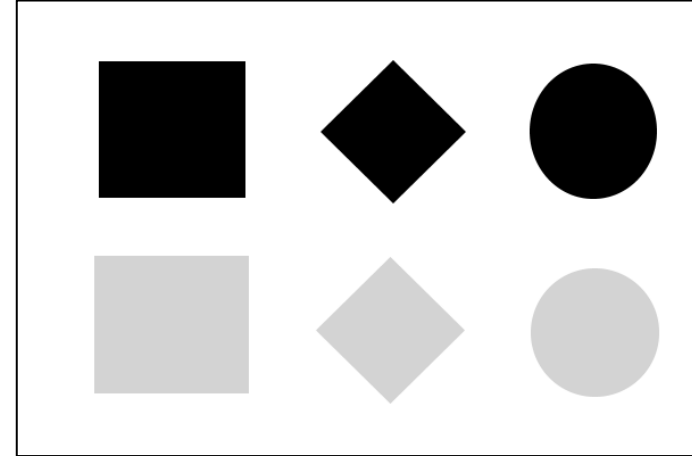
```
import numpy as np
import cv2

# učitaj sliku
img_src = cv2.imread('primjer.jpg')

# visokopropusni filter
kernel = np.array([[[-1.0, -1.0, -1.0],
                    [-1.0, 9.0, -1.0],
                    [-1.0, -1.0, -1.0]]])

# filtriraj sliku
img_rst = cv2.filter2D(img_src, -1, kernel)

# spremi rezultat
cv2.imwrite('rezultat_np.jpg', img_rst)
```



Filtriranje primjeri

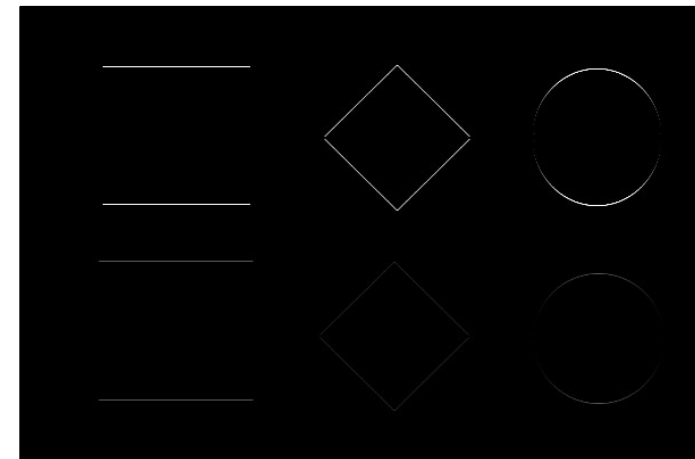
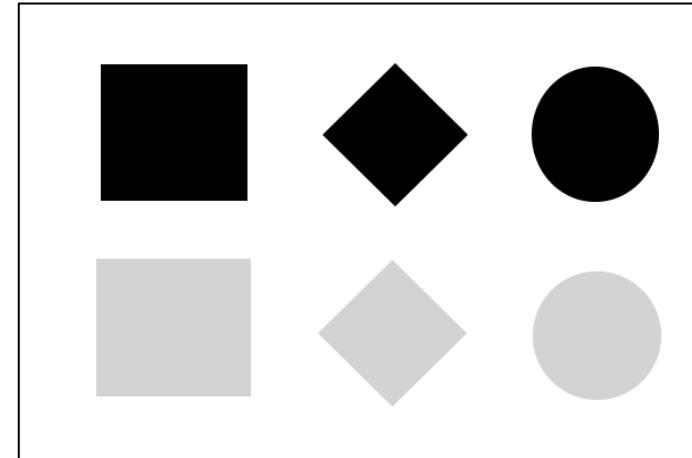
```
import numpy as np
import cv2

# učitaj sliku
img_src = cv2.imread('primjer.jpg')

#kernel osjetljiv na horizontalne linije
kernel = np.array([[-1.0, -1.0],
                   [2.0, 2.0],
                   [-1.0, -1.0]])

# filtriraj sliku
img_rst = cv2.filter2D(img_src, -1, kernel)

# spremi rezultat
cv2.imwrite('rezultat_np.jpg', img_rst)
```



Filtriranje primjeri

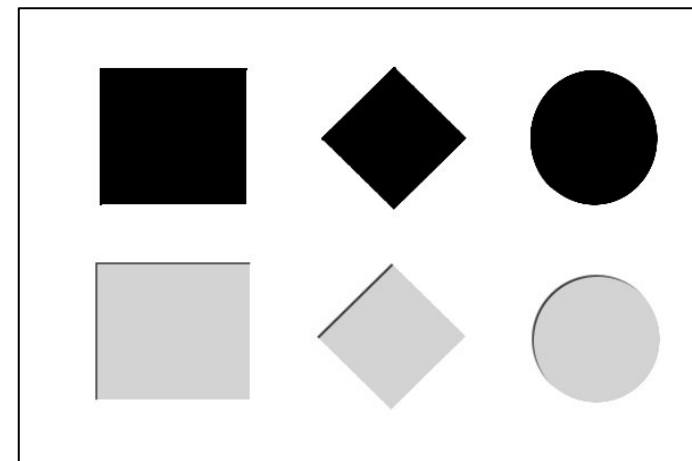
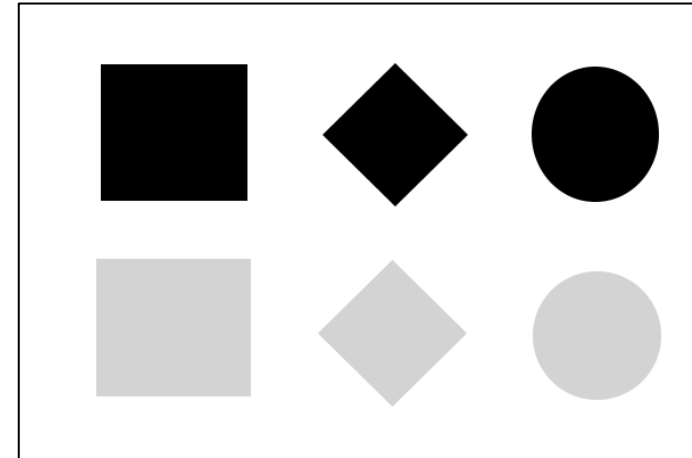
```
import numpy as np
import cv2

# učitaj sliku
img_src = cv2.imread('primjer.jpg')

#embossed efekt
kernel = np.array([[[-2.0, -1.0, 0.0],
                    [-1.0, 1.0, 1.0],
                    [0.0, 1.0, 2.0]]])

# filtriraj sliku
img_rst = cv2.filter2D(img_src,-1,kernel)

# spremi rezultat
cv2.imwrite('rezultat_np.jpg',img_rst)
```



Detekcija rubova

- Odnosi se na proces identifikacije i lociranja naglih diskontinuiteta na slici
- Rubovi čuvaju bitan oblik i strukturu scene – čak i ako velik dio detalja nedostaje, na temelju rubova često još uvijek možemo prepoznati što je na slici.

$$\nabla f \equiv grad(f) = \begin{bmatrix} g_x \\ g_y \end{bmatrix} = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix}$$

- Detekcija ruba u prostornoj domeni provodi se konvolucijom slike s **operatorima ruba**, tj. 2D filtrima koji su osjetljivi na velike promjene gradijenta intenziteta.
- Točke u slici gdje postoji značajna promjena svjetline se obično organiziraju u set zakrivljenih linijskih segmenata koji se nazivaju rubovi
- Osnovni alat u obradi slike i računalnom vidu

Detekcija rubova - motivacija



Čak i bez boja i tekstura, sam raspored rubova nosi dovoljno informacija da prepoznamo scenu.
Možemo zaključiti da se nalazimo u prostoriji (prozori, TV, stol, stolice...)."

Detekcija rubova, zahtjevi i problemi

- Operatori su definirani prema sljedećim zahtjevima:
 - **Niska pogreška detekcije ruba**
 - Pronađeni rub mora biti što bliži pravom rubu, odnosno ne bi trebalo biti lažnih rubova
 - **Dobra lokalizacija ruba**
 - Točke koje operator označi kao dio ruba trebaju biti što bliže stvarnom položaju ruba u slici.
 - Udaljenost između pronađenog ruba i “pravog” ruba treba biti minimalna.
 - **Informacija o orijentaciji ruba**
 - Geometrija operatora (tj. oblik kernela) treba omogućiti određivanje smjera ruba (horizontalni, vertikalni, kosi rubovi itd.).

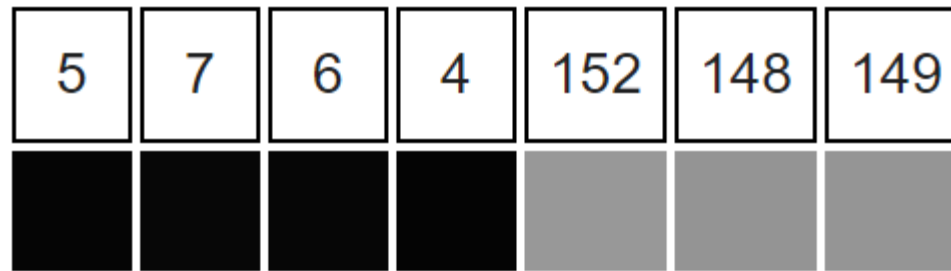
Detekcija rubova, zahtjevi i problemi

- Netočnost detekcije ruba može nastati zbog:
- **Utjecaja šuma**
 - Detektiranje rubova je otežano ako je šum jako izražen – šum može izgledati kao “lažni” mali rubovi.
 - Pokušaji smanjenja šuma (filtiranje, zaglađivanje) često dovode do zamućenih ili iskrivljenih rubova.
 - Operator koji može djelomično zanemariti utjecaj šuma obično koristi veću okolinu (više piksela) i “preskače” elemente slike gdje je šum jak, ali se u tom slučaju često smanjuje ukupna preciznost lokalizacije rubova.
- **Strukture rubova**
 - Ne uključuju svi rubovi jednostavnu, stepenastu promjenu intenziteta (tipa nagli skok s tamnog na svijetlo).
 - Zbog efekata poput refrakcije, sjena, poluprozirnosti i sl., rub može biti definiran postupnom promjenom intenziteta kroz više piksela.
 - U takvim slučajevima operator koji koristimo mora biti prilagođen tipu promjene (npr. širini ruba), inače će detekcija biti netočna ili nepotpuna.

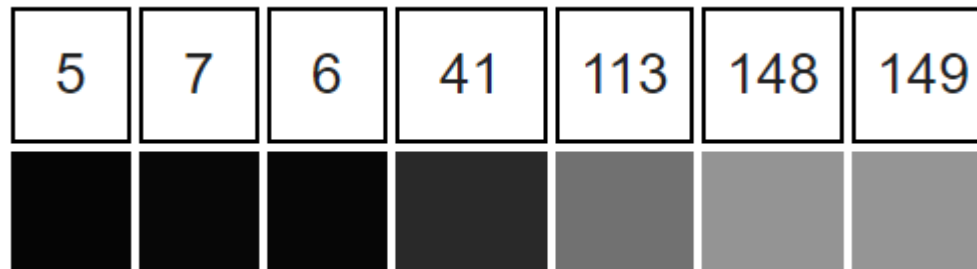
Detekcija rubova

- nije trivijalan zadatak – mora se definirati prag u smislu kolika mora biti promjena intenziteta da bi se nešto smatralo rubom

Gdje je rub?



Gdje je rub?



Detekcija rubova

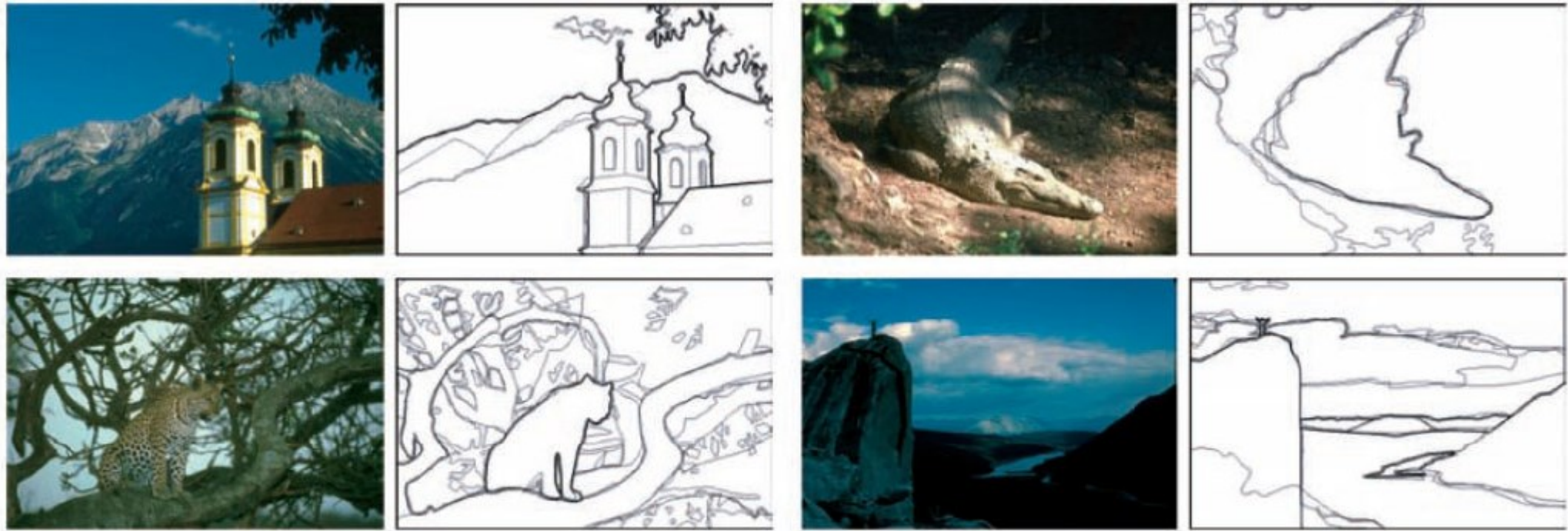


Figure 4.31 Human boundary detection (Martin, Fowlkes, and Malik 2004) © 2004 IEEE. The darkness of the edges corresponds to how many human subjects marked an object boundary at that location.

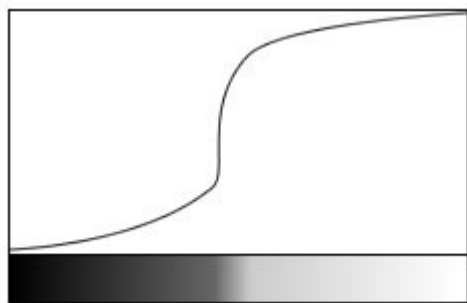
Detekcija rubova

- rezultat algoritma za detekciju rubova je binarna slika gdje su upravo istaknuti rubovi
- generalno algoritmi se klasificiraju na:
 - **algoritmi koji detektiraju „zero crossing” druge derivacije**
 - Laplace
 - **algoritme zasnovane na gradijentu** gdje se računa gradijent uzastopnih piksela u x i y smjeru (eng. *gradient based*) i detektiraju se lokalni maksimumi ili minimumi
 - Sobel, Prewitt, Canny

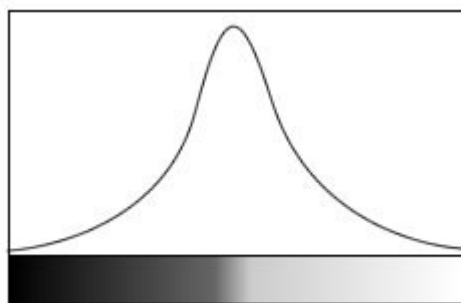
Detekcija rubova

- primjer u jednoj dimenziji

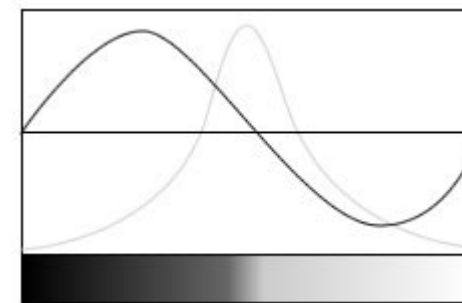
[Izvor](#)



Krivulja
predstavlja
intenzitet piksela



Prva derivacija
krivulje
intenziteta



Druga derivacija
krivulje
intenziteta

gradient based edge detection

- traži peakove u prvoj derivaciji
- što je rub, odnosno što je peak, a što nije?

zero crossing based edge detection

- traži prijelaz kada krivulja prve derivacije mijenja nagib
- osjetljivost na šum!

Detekcija rubova objekata

- Metoda gradijenta - određivanje minimuma i maksimuma prve derivacije signala

- Prva derivacija

$$\frac{\partial f(x, y)}{\partial x} = f(x+1, y) - f(x, y)$$

$$\frac{\partial f(x, y)}{\partial y} = f(x, y+1) - f(x, y)$$

- Centralna derivacija

$$\frac{\partial f(x, y)}{\partial x} = \frac{1}{2} [f(x+1, y) - f(x-1, y)]$$

$$\frac{\partial f(x, y)}{\partial y} = \frac{1}{2} [f(x, y+1) - f(x, y-1)]$$

Zapravo filtri s maskama
(aproksimacija)

$$\Rightarrow \begin{bmatrix} -1 & 1 \\ 0 & 0 \end{bmatrix}$$

$$\Rightarrow \begin{bmatrix} -1 & 0 \\ 1 & 0 \end{bmatrix}$$

$$\Rightarrow \frac{1}{2} \cdot \begin{bmatrix} 0 & 0 & 0 \\ -1 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}$$

$$\Rightarrow \frac{1}{2} \cdot \begin{bmatrix} 0 & -1 & 0 \\ 0 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

Detekcija rubova

- **Prewitt-ov operator**
 - Koristi centralnu derivaciju i konvolucijsku masku 3x3
- **Sobel-ov operator**
 - Koristi centralnu derivaciju i konvolucijsku masku veličine 3x3, ali propuštenu kroz nisko-propusni filter [1 2 1] (blago zaglađivanje u smjeru okomitom na rub)
- **Laplace-ov operator**
 - Računa drugu derivaciju; jednako reagira na rubove bilo koje orijentacije

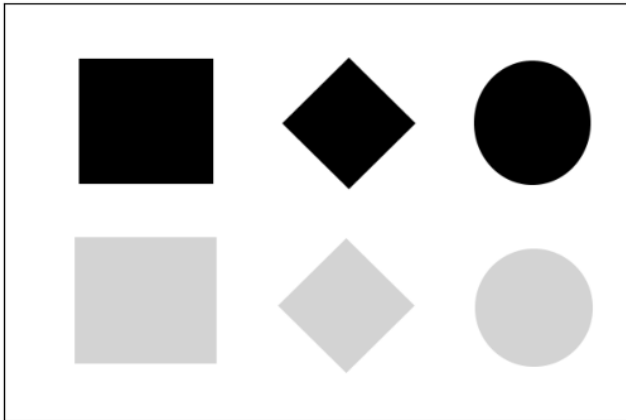
$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} * I \quad G_y = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix} * I$$

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} * I \quad G_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} * I$$

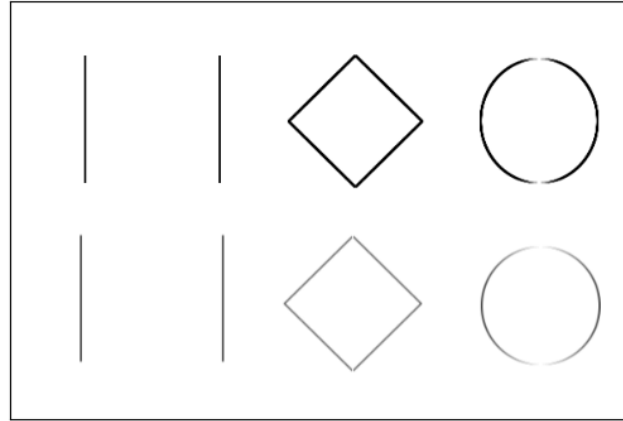
$$G = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} * I$$

Primjer Sobel

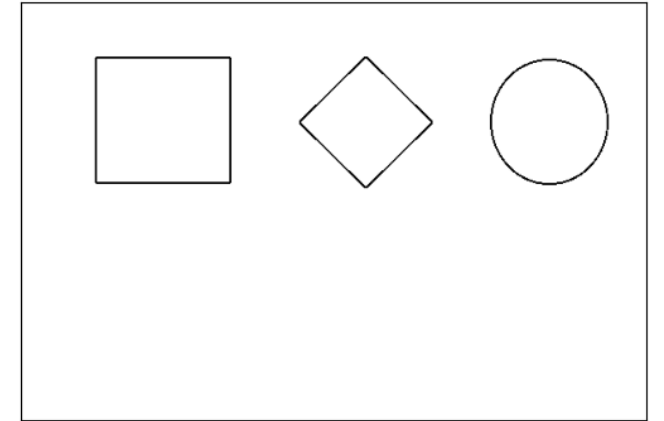
Original



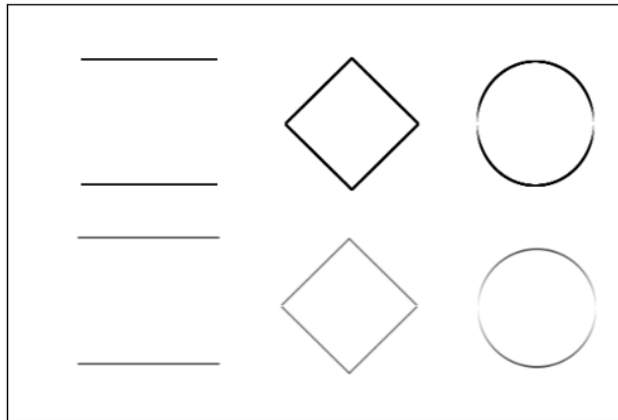
Sobel X



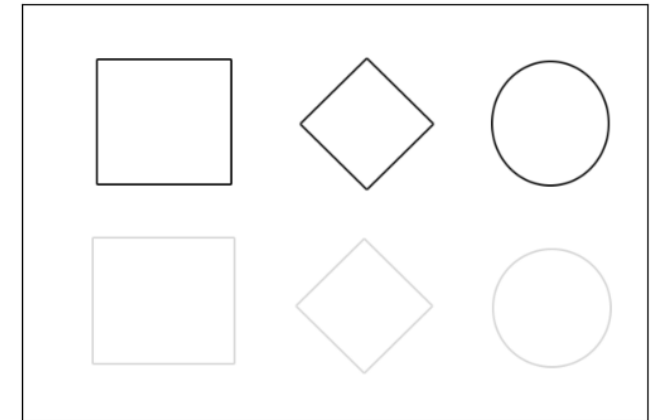
G threshold=120



Sobel Y

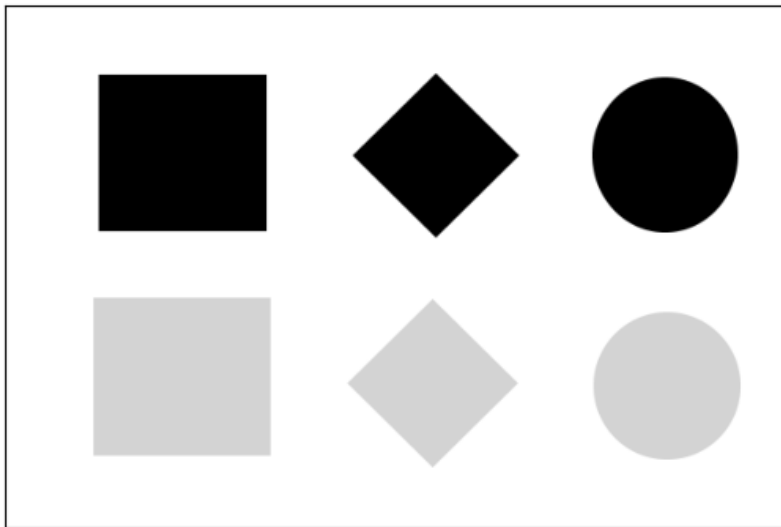


$G = \sqrt{\text{SobelX}^2 + \text{SobelY}^2}$

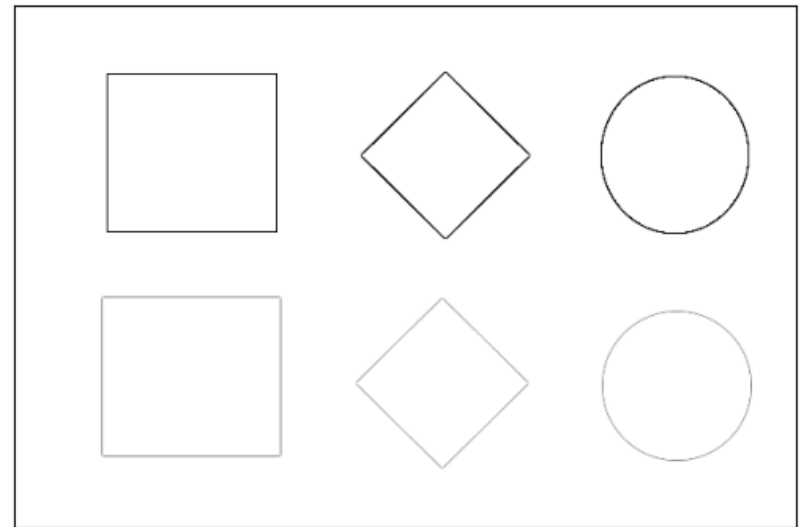


Primjer Laplace

Original



Laplacian



Sobel-ov operator

Ulazna
slika



Primjena
 G_x

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} * I$$



Primjena
 G_y

$$G_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} * I$$



Laplace-ov operator

Ulazna slika



Primjena Laplace operatora



Detekcija rubova jednostavnom metodom praga

- Jednom kada se izračuna mjera jačine ruba (kao npr. gradijent) primjenjuje se metoda praga kako bismo izdvojili stvarne rubove.
- Metoda praga je postupak u kojem za svaki piksel uspoređujemo vrijednost (jakost gradijenta) s odabranim pragom T te ga klasificiramo kao rub ili ne.

Laplace



Prag 50



Prag 200



Detekcija rubova jednostavnom metodom praga

- Problemi:
 - **Šum**
 - Šum često daje lokalne male “peakove” u gradijentu
 - ako je prag prenizak → dobiješ hrpu lažnih rubova (šum)
 - ako je previsok → izgubiš slabe, ali stvarne rubove
 - **Varijabilni kontrast u slici**
 - Neki rubovi su jako kontrastni (npr. crno–bijeli prijelaz) drugi su slabi, ali važni (npr. rub sjene, mek prijelaz)
 - jedan globalni prag ne može dobro odvojiti sve to od šuma u cijeloj slici
 - **Debeli rubovi**
 - Gradijent je velik u više susjednih piksela
 - Nakon primjene praga često se dobiju “debele” rubne zone, a ne tanke linije za rubove
 - To otežava daljnju obradu (npr. detekciju linija, kontura)

Canny detektor rubova

- Vrlo popularan i efikasan detektor rubova; zahtijeva dva parametra
 1. Pretprocesiranje slike (pretvorba u sliku sa sivim tonovima, uklanjanje šuma)
 2. Izračunavanje gradijenata
 3. Odbacivanje ne-maksimalnih vrijednosti (eng. non-maximum suppression)
 4. Primjena dvostrukog praga (eng. double thresholding)
 5. Praćenje ruba histerezom



Canny detektor rubova

1. Pretprocesiranje

- kromatska slika se najprije pretvori u sliku sa sivim tonovima
- uobičajeno se koristi Gaussov blur (dimenzije kernela 3x3, 5x5) kako bi se smanjila količina detalja na slici

2. izračunavanje gradijenata

- koriste se Sobelovi filtri u x i y smjeru te se izračunava ukupni gradijent i smjer gradijenta za svaki piksel

$$Edge_Gradient (G) = \sqrt{G_x^2 + G_y^2}$$

$$Angle (\theta) = \tan^{-1} \left(\frac{G_y}{G_x} \right)$$

- smjer gradijenta je uvijek okomit na rub koji je prisutan na slici (sjetite se što predstavlja gradijent – intenzitet promjene u osvjetljenju) → smjer se diskretizira na 8 vrijednosti (gore, dolje, lijevo, desno i četiri dijagonale)

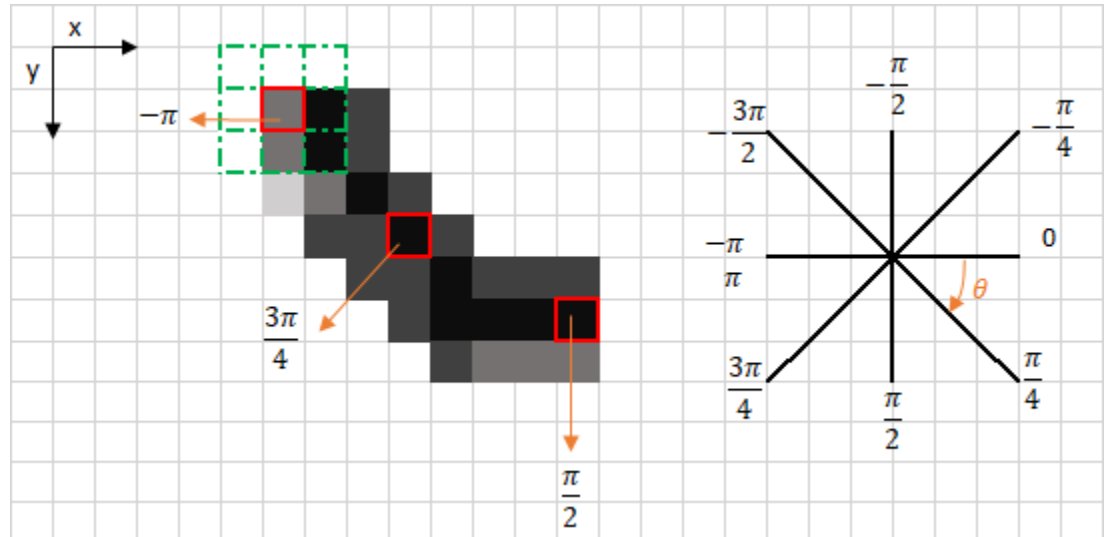
Canny detektor rubova

3. Potiskivanje ne-maksimalnih vrijednosti (eng. non-maximum suppression)

- idealno htjeli bismo da su rubovi u slici predstavljeni tankim „linijama” piksela u izlaznoj slici → maksimalnom vrijednosti na nekom rubu
- stoga je potrebno „stanjiti” rubove tj. zadržati samo lokalne maksimume

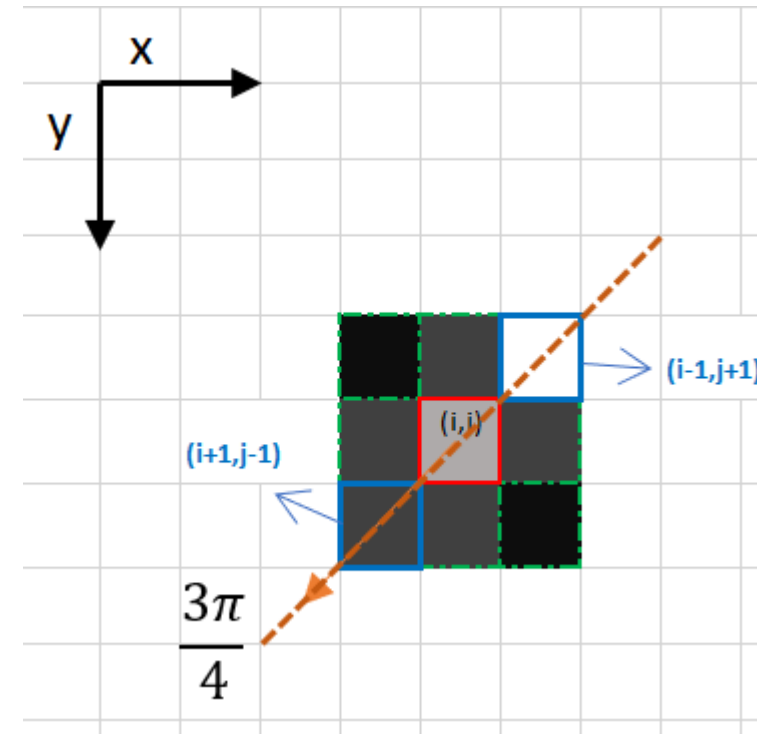
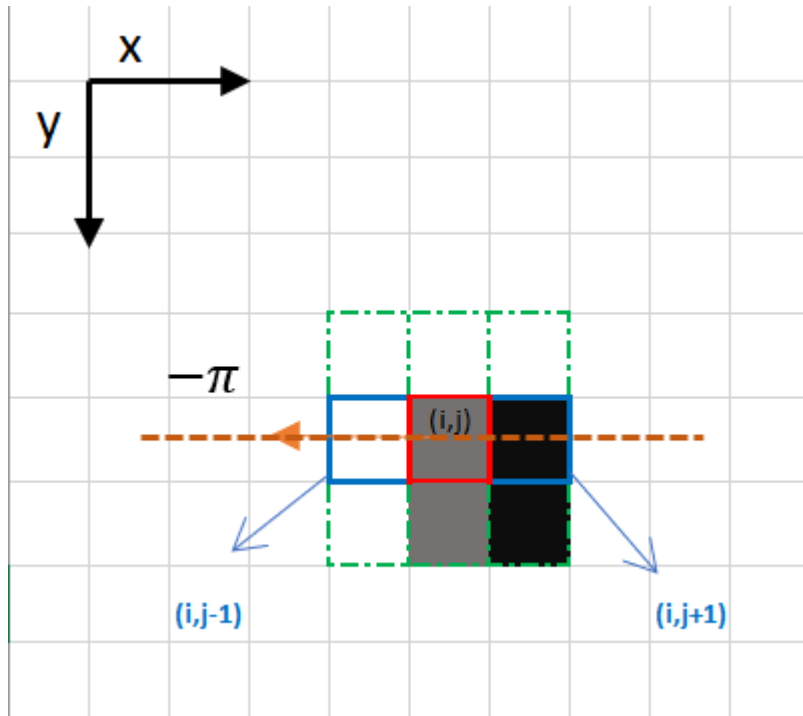
Ideja je proći kroz matricu koja sadrži ukupni gradijent i pronaći elemente s maksimalnom vrijednosti u smjeru ruba

U izlaznu matricu spremiti samo lokalne maksimume; u suprotnom potisnuti vrijednost (postaviti izlaz na nulu)



Canny detektor rubova

Primjeri potiskivanja ne-maksimalnih vrijednosti



Canny detektor rubova

Dobivamo sliku s tanjim rubovima. Ali još uvijek postoje varijacije u jačini rubova – neki pikseli su izraženiji od drugih!



Edge_Gradient



Potiskivanje ne-maksimalnih
vrijednosti

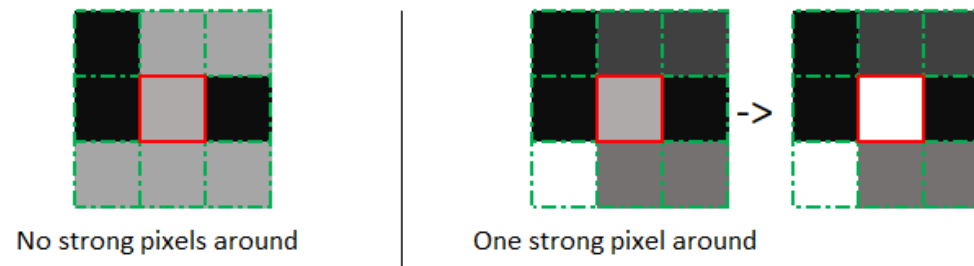
Canny detektor rubova

4. Primjena dvostrukog praga

- ovim postupkom se identificiraju tri vrste piksela u slici dobivenoj potiskivanjem ne-maksimalnih vrijednosti:
 - jaki (strong) – imaju vrijednost iznad gornjeg praga i za njih smo sigurni da pripadaju rubu
 - nevažni – imaju vrijednost ispod donjeg praga i odbacuju se
 - slabi (weak) – imaju vrijednost između gornjeg i donjeg praga te će se odluka pripadaju li rubu ili ne odrediti u zadnjem koraku pomoću mehanizma histereze

5. Praćenje ruba histerezom

- pikseli koji su slabi, a povezani su s jakim pikselima smatramo da su rub



Canny detektor rubova



Primjena dvostrukog praga



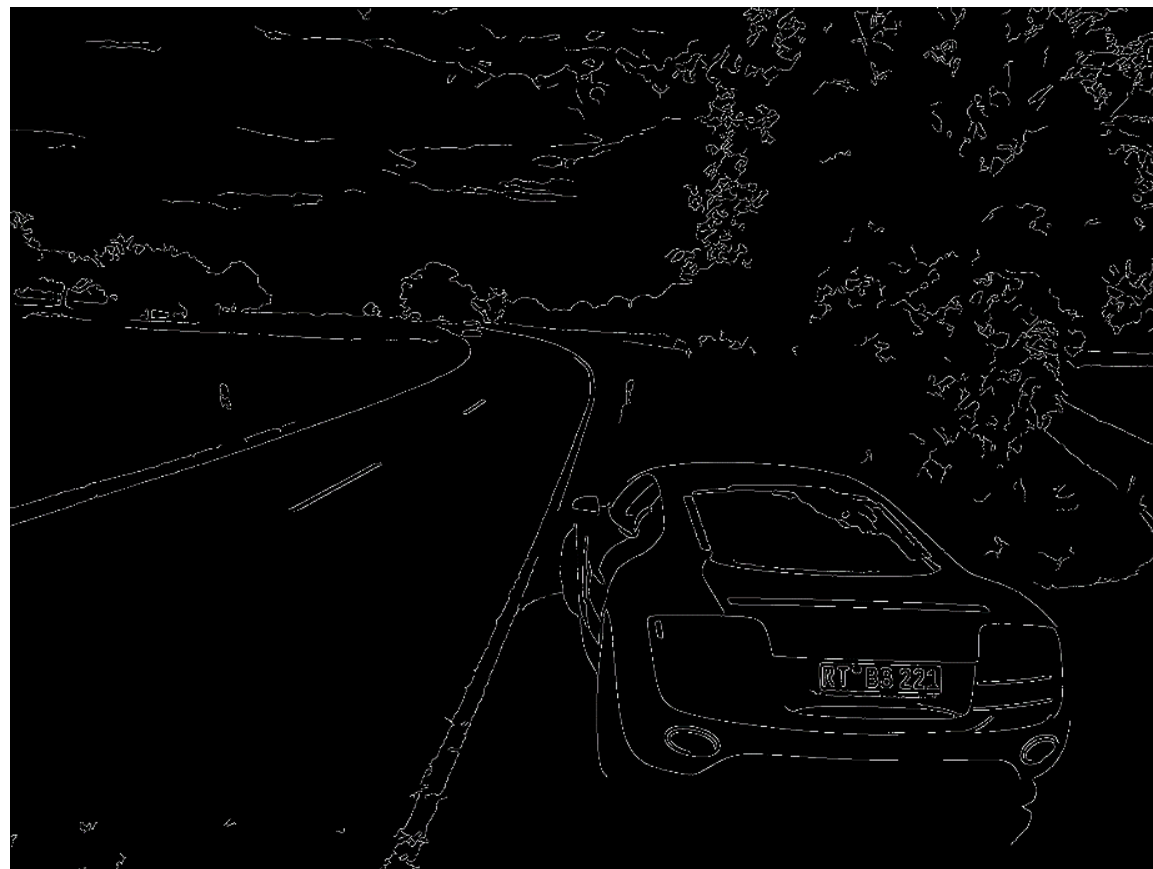
Praćenje ruba

Detekcija rubova uz pretprocesiranje

Bez filtriranja, Canny, 30, 150



Gaussov filter (15x15) + Canny, 30, 150



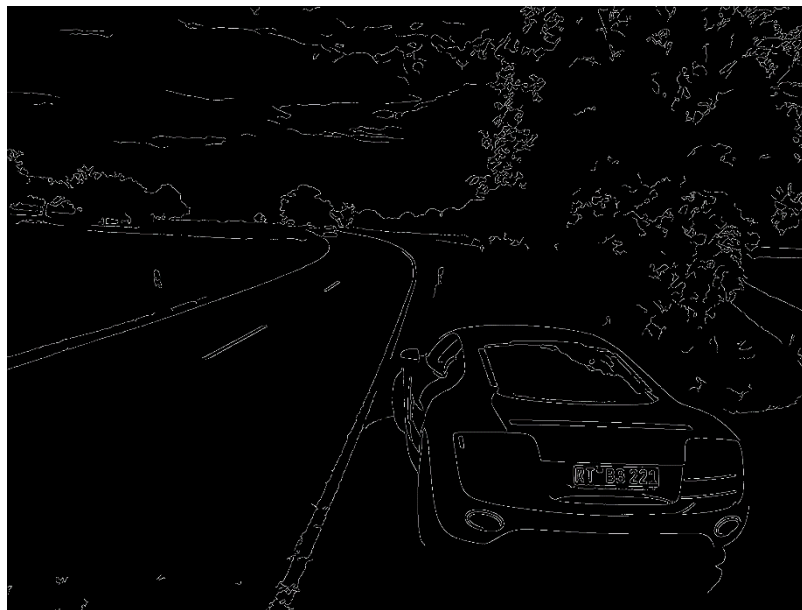
OpenCV funkcija: `cv2.Canny`

Detekcija rubova objekata uz pretprocesiranje

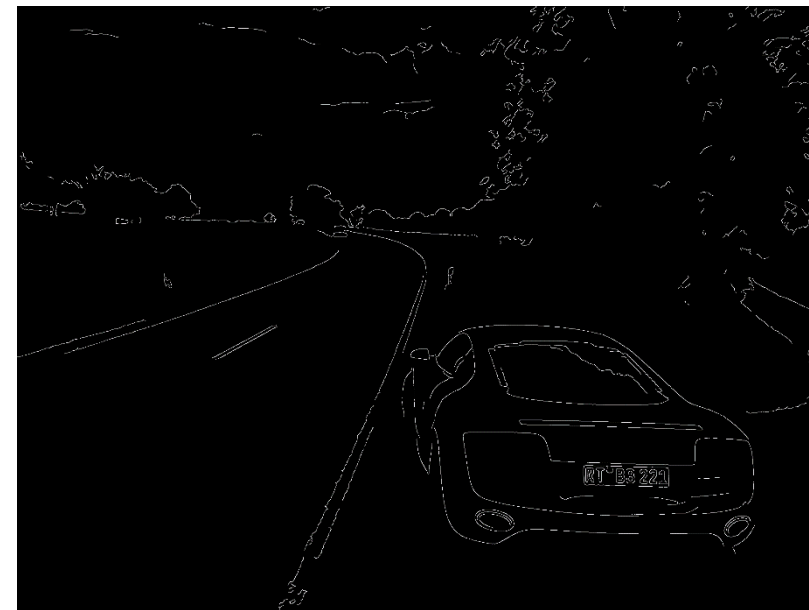
Gaussov filter 15 x 15



Canny, 30, 150

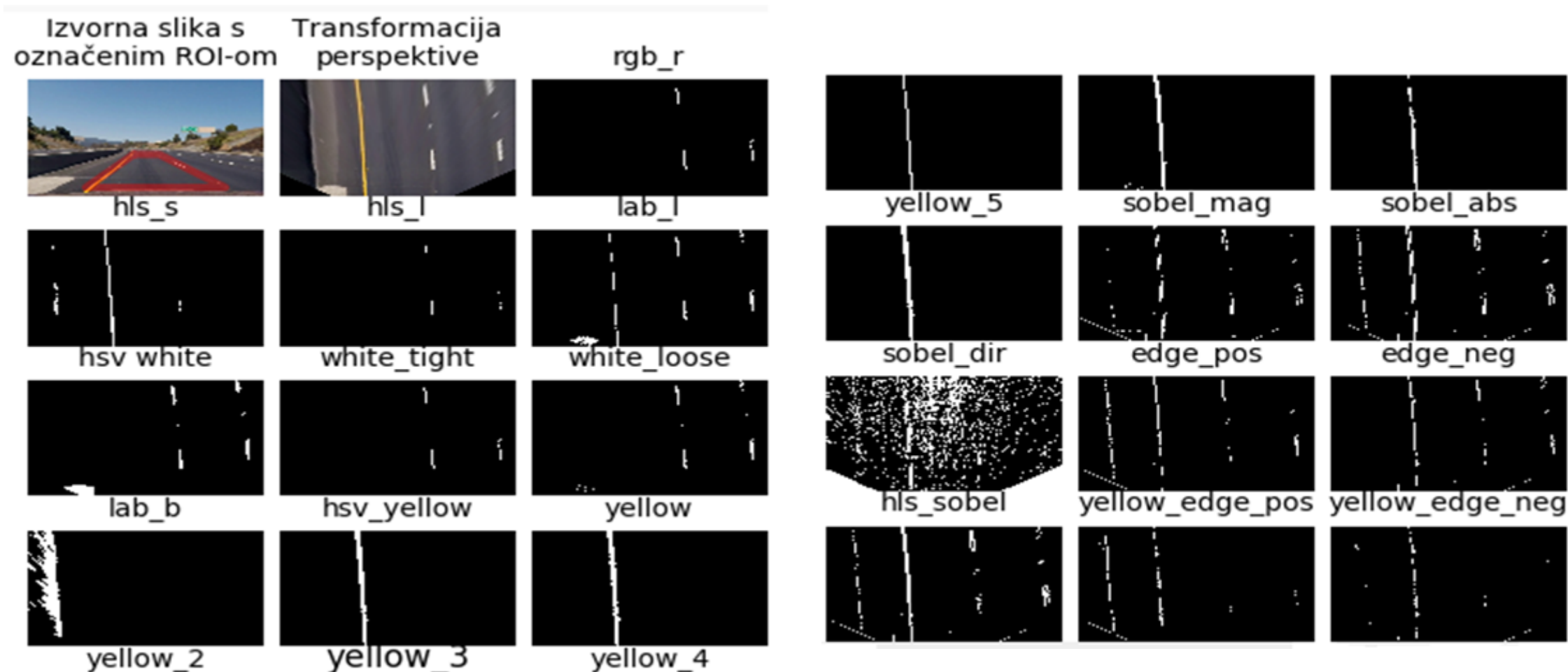


Canny, 100, 200



Detekcija rubova – primjeri primjene

- Detekcija linija vozne trake (prethodi korištenju ranije spomenutog histograma)



Detekcija rubova – primjeri primjene

Original



Sobel



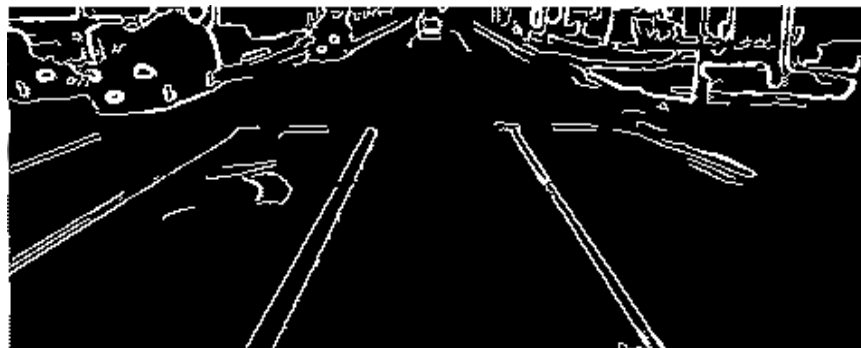
Prewitt



Laplace



Canny



Detekcija rubova – primjeri primjene

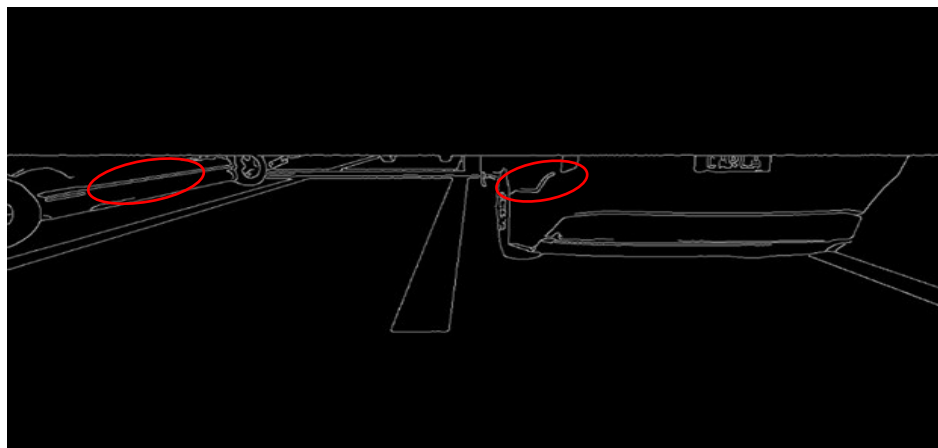
Okvir s
bočne
kamere



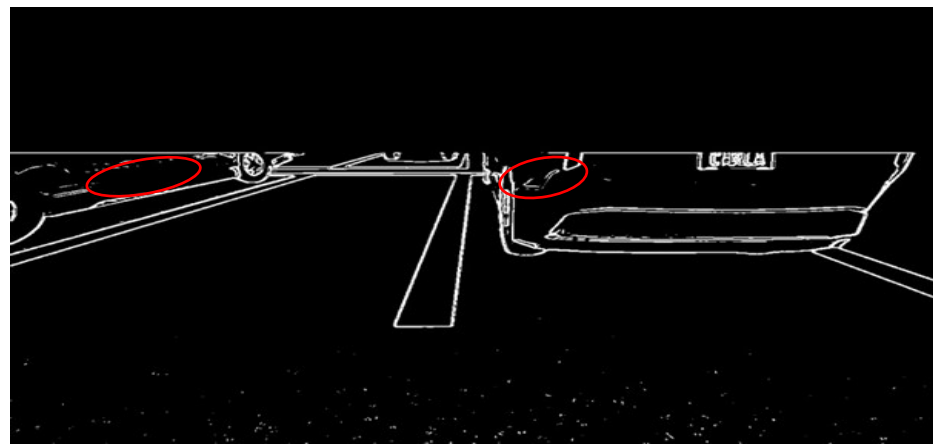
ROI +
median
filter



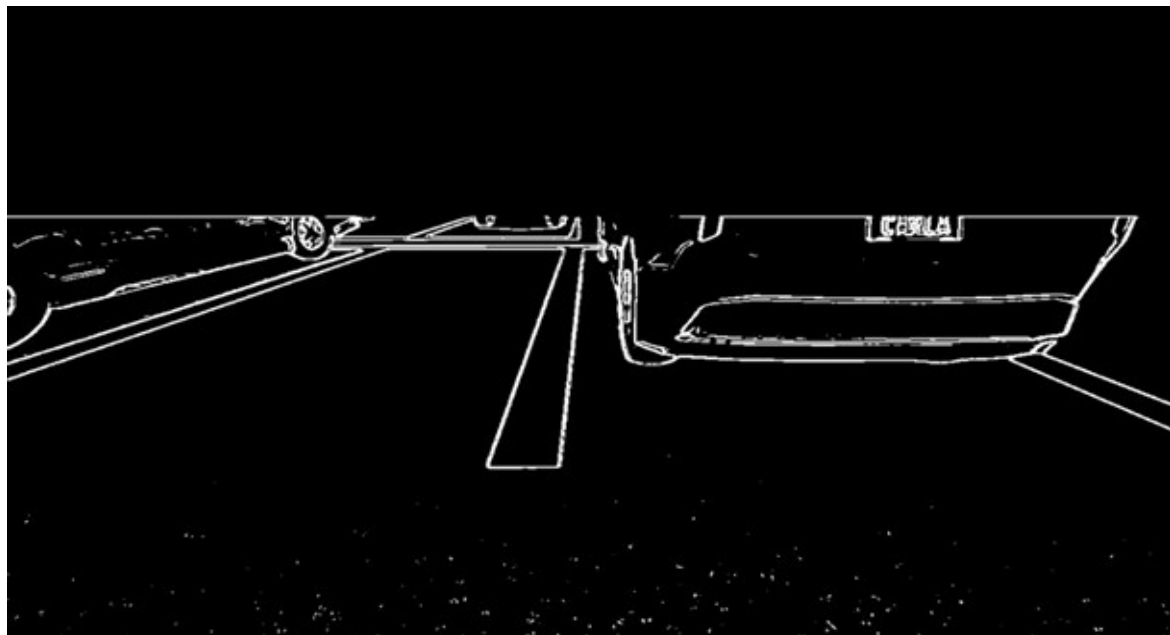
Canny



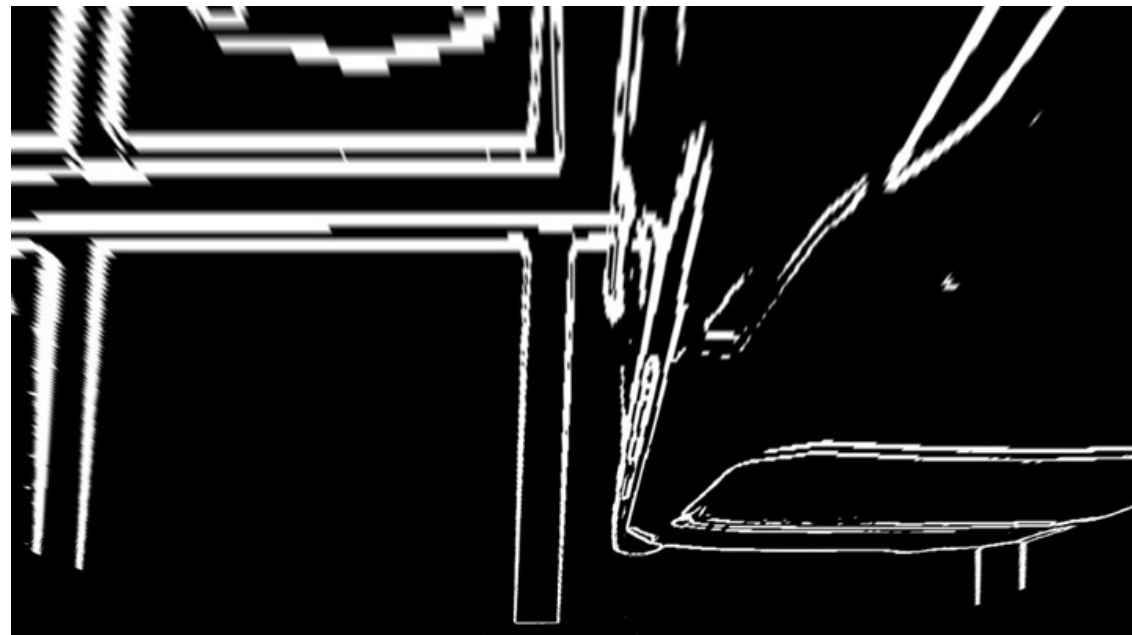
Sobel



Detekcija rubova – primjeri primjene



Sobel



Sobel + perspektivna transformacija

Detekcija rubova objekata uz preprocesiranje

```
import cv2
import numpy as np

image = cv2.imread('images/car_and_road.jpg',0)

height, width = image.shape

# Extract Sobel Edges
sobel_x = cv2.Sobel(image, cv2.CV_64F, 0, 1, ksize=5)
sobel_y = cv2.Sobel(image, cv2.CV_64F, 1, 0, ksize=5)

cv2.imshow('Original', image)
cv2.waitKey(0)
cv2.imshow('Sobel X', sobel_x)
cv2.waitKey(0)
cv2.imshow('Sobel Y', sobel_y)
cv2.waitKey(0)

sobel_OR = cv2.bitwise_or(sobel_x, sobel_y)
cv2.imshow('sobel_OR', sobel_OR)
cv2.waitKey(0)

laplacian = cv2.Laplacian(image, cv2.CV_64F)
cv2.imshow('Laplacian', laplacian)
cv2.waitKey(0)
```

```
# We need to provide two values: threshold1 and
# threshold2. Any gradient value larger than threshold2
# is considered to be an edge. Any value below threshold1
# is considered not to be an edge. Values in between threshold1
# and threshold2 are either classified as edges or non-edges
# based on how their intensities are "connected". In this case,
# any gradient values below 50 are considered non-edges,
# whereas any values above 120 are considered edges.

# Canny Edge Detection uses gradient values as thresholds
# The first threshold gradient
canny = cv2.Canny(image, 50, 120)
cv2.imshow('Canny', canny)
cv2.waitKey(0)

cv2.destroyAllWindows()
```

Morfološko filtriranje

Morfološko filtriranje

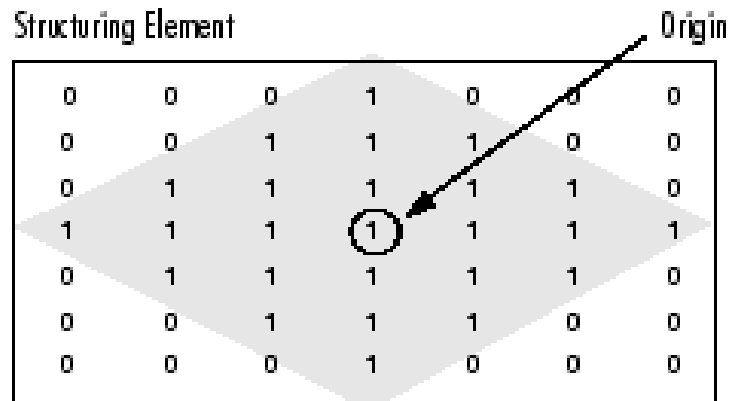
- Morfološko filtriranje/transformacije → to znači da se geometrijske strukture na slici mijenjaju
- Niz transformacija koje utječu na oblik objekata na slici
- Morfološke transformacije
 - Primjenjuju strukturni element (element strukturiranja) na sliku
- Najčešće se primjenjuju na binarnim slikama, ali se mogu primijeniti i na slikama u sivim tonovima
- **Primjeri**
 - **Erozija**
 - **Dilatacija**
 - **Otvaranje**
 - **Zatvaranje**

Često se primjenjuju kao korak pred-obrađe za OCR (Optical Character Recognition) algoritam, detekciju/korekciju barkoda, QR koda, registarskih oznaka ...

Element strukturiranja

`cv2.getStructuringElement(shape, ksize[, anchor])` -> retval

- **shape** – oblik elementa koji može biti jedan od MorphShapes:
`cv2.MORPH_RECT`
`cv2.MORPH_CROSS`
`cv2.MORPH_ELLIPSE`
- **ksize** – veličina elementa strukturiranja
- Vraća strukturirajući element tražene veličine i oblika za morfološku operaciju



Morfološko filtriranje - Erozija

- Ona erodira („nagriza”) granice obilježja na slici
- Osnovna ideja erozije → erodira granice objekta u prednjem planu (uvijek nastojte da prednji plan bude bijeli)
- `cv2.erode()`
- **Dakle, što ona radi?**
 - Kernel prolazi kroz sliku (kao u 2D konvoluciji)
 - Piksels na izvornoj slici (bilo 1 ili 0) smatrat će se 1 samo ako su svi pikseli ispod kernela 1, u suprotnom, on se erodira (postavlja na 0)
 - Svi pikseli u blizini granice bit će odbačeni ovisno o veličini kernel
 - Debljina ili veličina objekta u prednjem planu se smanjuje ili se jednostavno smanjuje bijelo područje na slici



5x5 kernel

Morfološko filtriranje - Eroziija

- Korisno je za uklanjanje šuma na slici
- Također se koristi za razdvajanje dvaju povezanih objekata



Morfološko filtriranje - Dilatacija

- Upravo suprotno od erozije
- Piksel je '1' ako je barem jedan piksel ispod kernela '1'
- Povećava bijelo područje na slici ili se povećava veličina objekta u prednjem planu
- Također je korisna za spajanje odvojenih dijelova predmeta
- `cv2.dilate()`



5x5 kernel

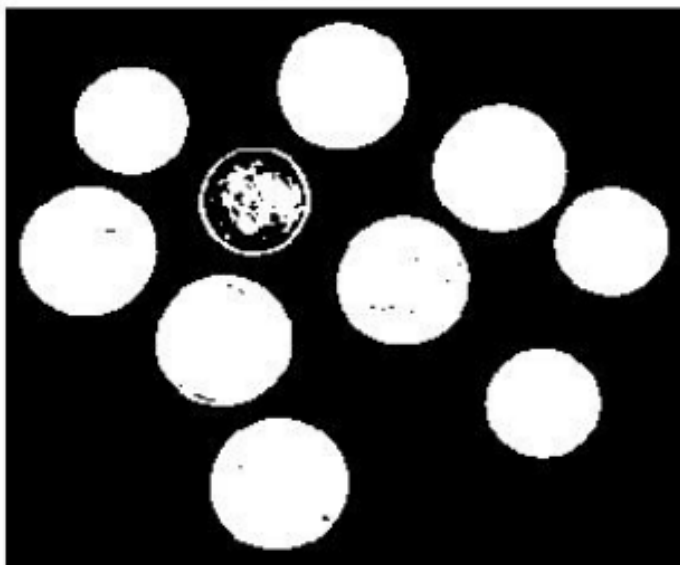
Morfološko filtriranje - Dilatacija

- Primjer primjene dilatacije

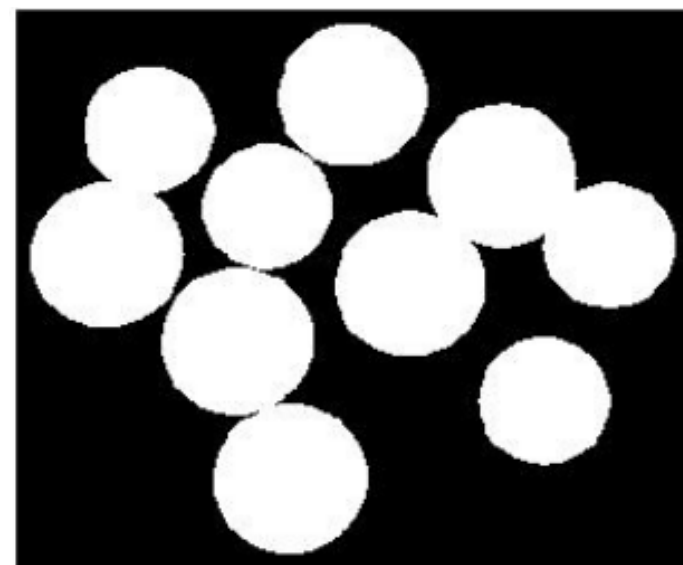
Originalna slika



Binarna slika

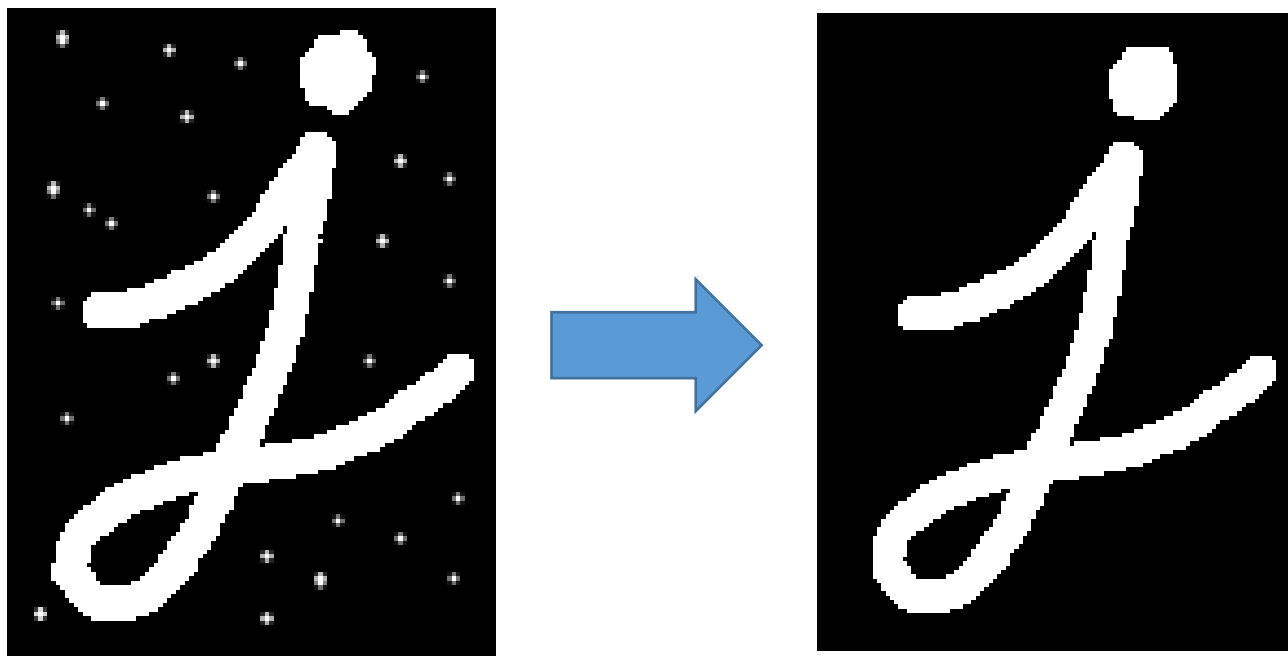


Dilatirana binarna slika



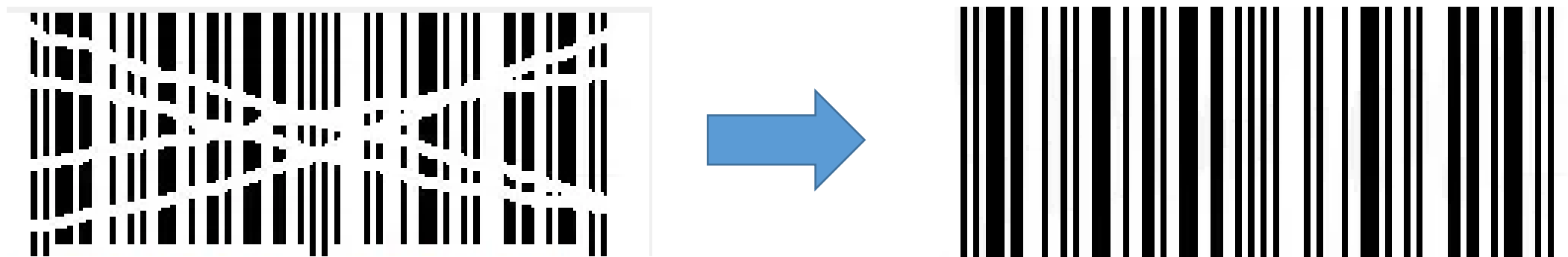
Morfološko filtriranje - Otvaranje

- **Otvaranje** → primjena erozije pa potom dilatacije
 - Ova operacija nastoji proširiti male otvore i ukloniti slabo povezane piksele
- **Erozija** → uklanja šum, ali i smanjuje objekt u prednjem planu → **Dilatacija**



Morfološko filtriranje - Otvaranje

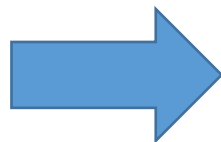
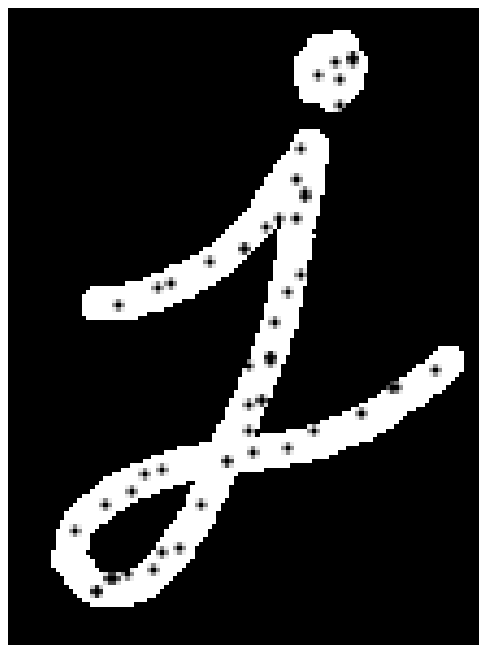
- Otvaranje → npr. kod oporavka bar koda



- Primijenjen kernel 1x5, uz 5 iteracija otvaranja

Morfološko filtriranje - Zatvaranje

- Zatvaranje → primjena dilatacije pa potom erozije
- Korisno je za zatvaranje malih rupa unutar objekata u prednjem planu ili malih crnih točkica na objektu

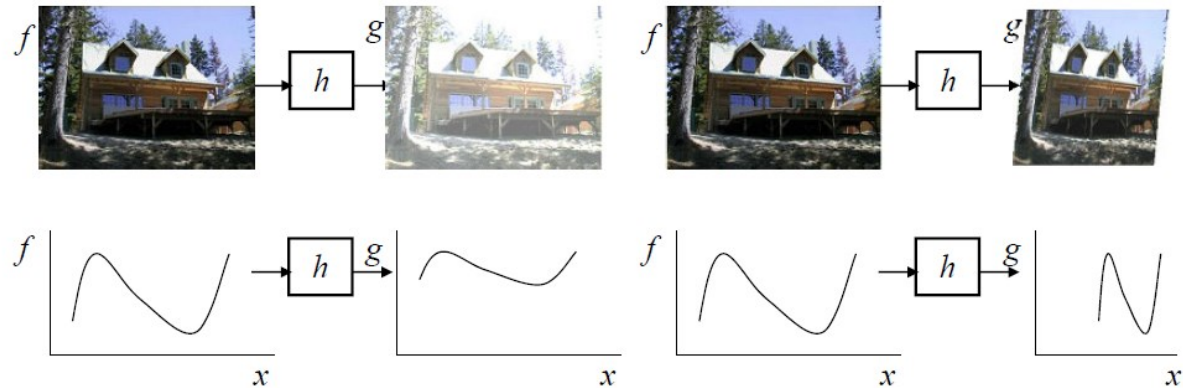


dilatacija prvo popuni male rupe i pukotine, erozija vrati objekt na približnu izvornu veličinu.

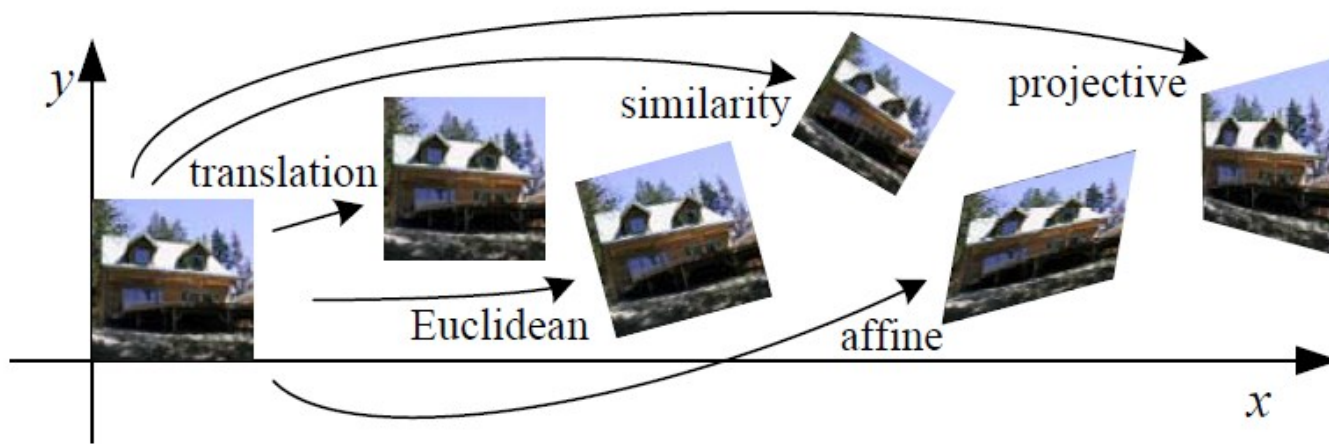
Geometrijske transformacije

Geometrijske transformacije slike

- Do sada je bilo govora o mijenjaju vrijednosti piksela filtriranjem slike
- Pod geometrijskim transformacijama podrazumijeva se promjena pozicije pojedinog piksela dok se vrijednost piksela ne mijenja
 - translacija slike
 - rotacija slike
 - povećanje/smanjenje slike
 - projekcija na različite ravnine
- Dije se na:
 - affine transformacije
 - nonaffine transformacije → perspektivna transformacija
- Geometrijska transformacija općenito opisuje odnos između koordinata piksela u dvije slike iste scene



Primjeri



Affine transformacije - matrica je 2×3

Non affine transformacija (perspektivna transformacija) – ne zadržava ni kuteve, ni dužinu ni paralelnost linija; matrica je 3×3

Osnova je množenje sa matricom

Transformation	Matrix	# DoF	Preserves	Icon
translation	$\begin{bmatrix} \mathbf{I} & \mathbf{t} \end{bmatrix}_{2 \times 3}$	2	orientation	
rigid (Euclidean)	$\begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix}_{2 \times 3}$	3	lengths	
similarity	$\begin{bmatrix} s\mathbf{R} & \mathbf{t} \end{bmatrix}_{2 \times 3}$	4	angles	
affine	$\begin{bmatrix} \mathbf{A} \end{bmatrix}_{2 \times 3}$	6	parallelism	
projective	$\begin{bmatrix} \tilde{\mathbf{H}} \end{bmatrix}_{3 \times 3}$	8	straight lines	

Transformacija slike

- affina transformacija je funkcijsko mapiranje između dva geometrijska prostora koje zadržava točke, ravne i paralelne linije kao i odnose među točkama
- affina transformacija predstavlja se matricom M koja je dimenzija 2×3

$$A = \begin{bmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \end{bmatrix}_{2 \times 2} \quad B = \begin{bmatrix} b_{00} \\ b_{10} \end{bmatrix}_{2 \times 1}$$

$$M = [A \quad B] = \begin{bmatrix} a_{00} & a_{01} & b_{00} \\ a_{10} & a_{11} & b_{10} \end{bmatrix}_{2 \times 3}$$

- ako se želi piksel X slike preslikati na novu lokaciju T u izlaznoj slici:

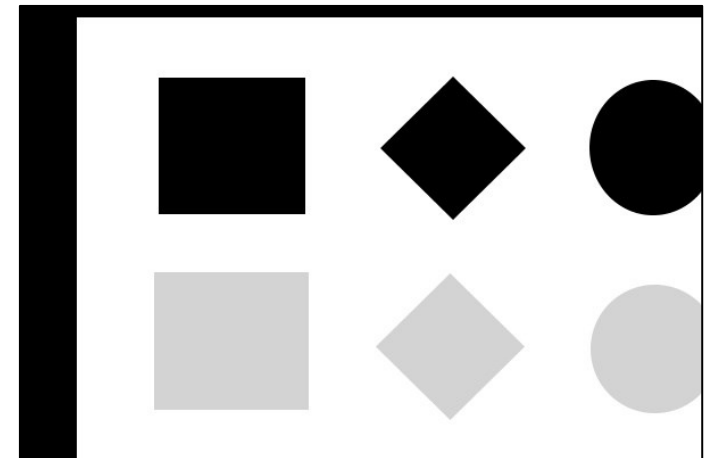
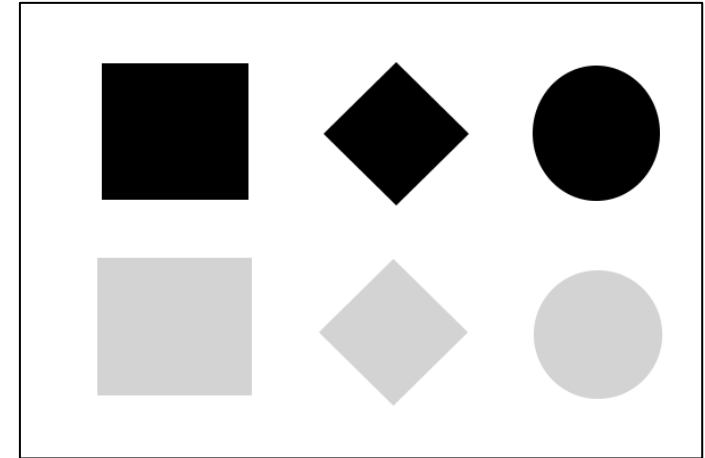
$$X = \begin{bmatrix} x \\ y \end{bmatrix} \quad T = A \cdot \begin{bmatrix} x \\ y \end{bmatrix} + B \quad T = M \cdot [x, y, 1]^T$$

Translacija slike

- Translacija – pomak slike na određenu lokaciju iz originalne pozicije
- Transformacijska matrica je oblika:

$$M = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \end{bmatrix}$$

- OpenCV funkcija
`cv2.warpAffine(img, M, (cols, rows))`

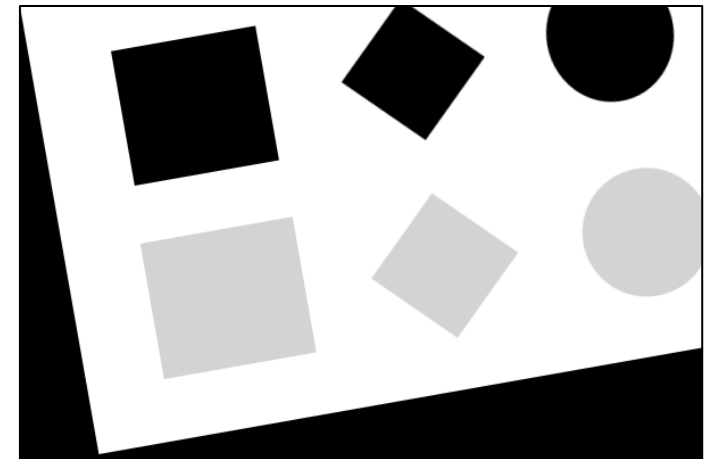
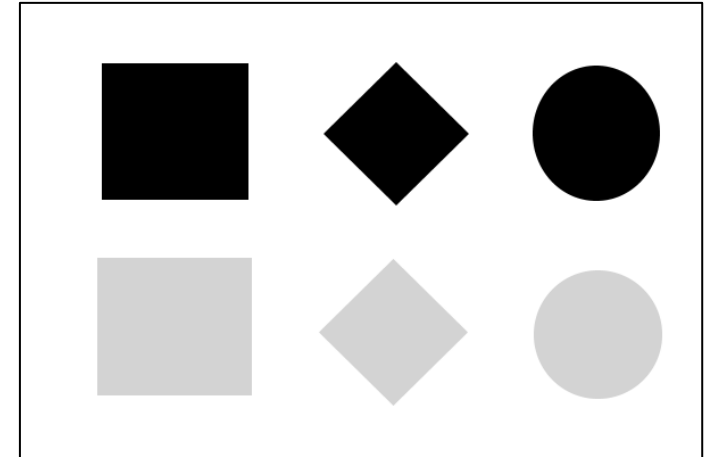


Rotacija slike

- Rotacija – rotacija slike za određeni kut (oko ishodišta)
- Transformacijska matrica je oblika:

$$M = \begin{bmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \end{bmatrix}$$

- kut je definiran u smjeru suprotnom od kazaljke na satu; oko lijevog gornjeg ugla
- OpenCV funkcije:
`cv2.warpAffine`
`cv.getRotationMatrix2D`

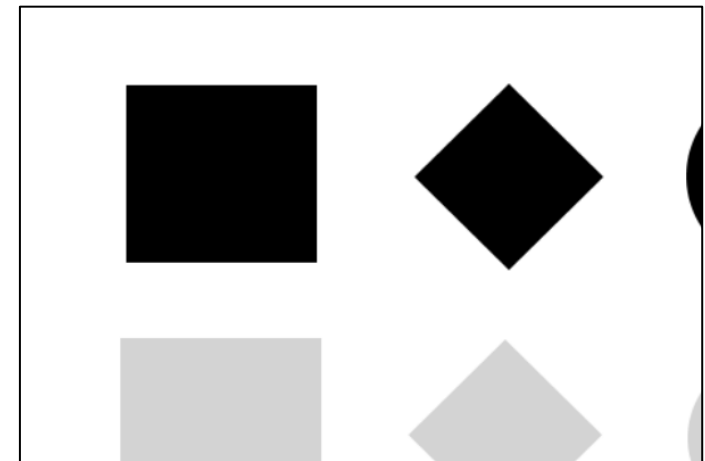
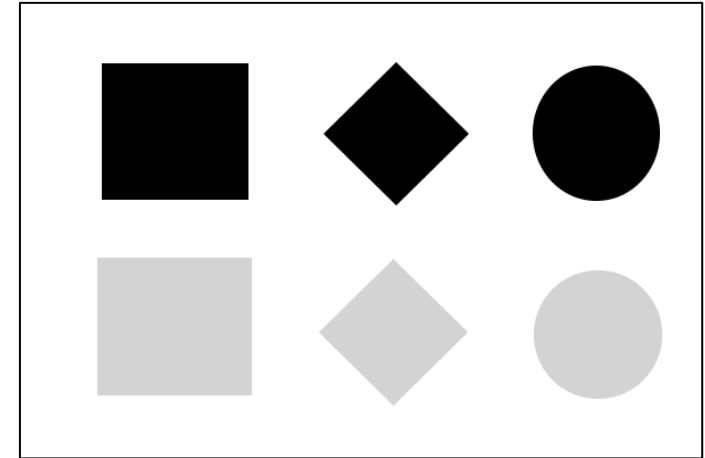


Skaliranje slike

- Skaliranje – uvećanje ili smanjivanje slike
- Transformacijska matrica je oblika:

$$M = \begin{bmatrix} c_x & 0 & 0 \\ 0 & c_y & 0 \end{bmatrix}$$

- međutim specijalno su na raspolaganju funkcije `cv2.resize()` gdje se definira veličina izlazne slike ili skalirajući faktor
- potrebno je napraviti interpolaciju piksela – postoje različite metode



Geometrijske transformacije OpenCV

```
import numpy as np
import cv2

# učitaj sliku
img_src = cv2.imread('primjer.png', 0)
rows = img_src.shape[0]
cols = img_src.shape[1]

# kreiraj transformacijsku matricu
theta = -10.0 * np.pi/180.0
M_rot = np.float32([[np.cos(theta), -np.sin(theta), 0],
                    [np.sin(theta), np.cos(theta), 0]])

# transformiraj sliku
img_rst = cv2.warpAffine(img_src, M_rot, (cols, rows))

# spremi rezultat
cv2.imwrite('rezultat_rotacija.png', img_rst)
```

Kombinacija OpenCV

- rotacija slike, s pomakom i promjenom skale:
`cv2.getRotationMatrix2D()`

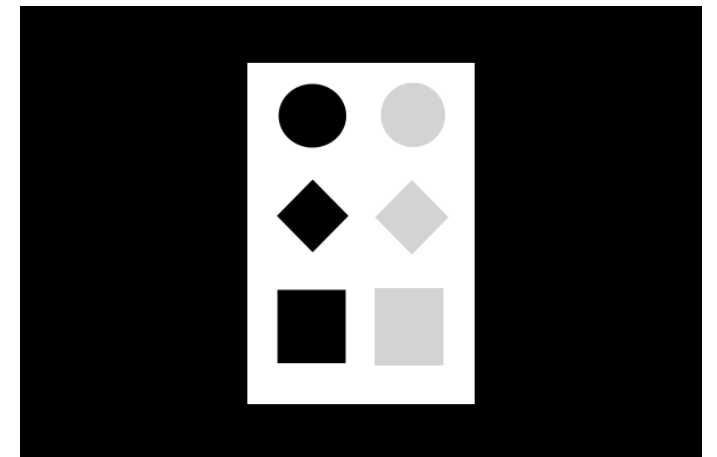
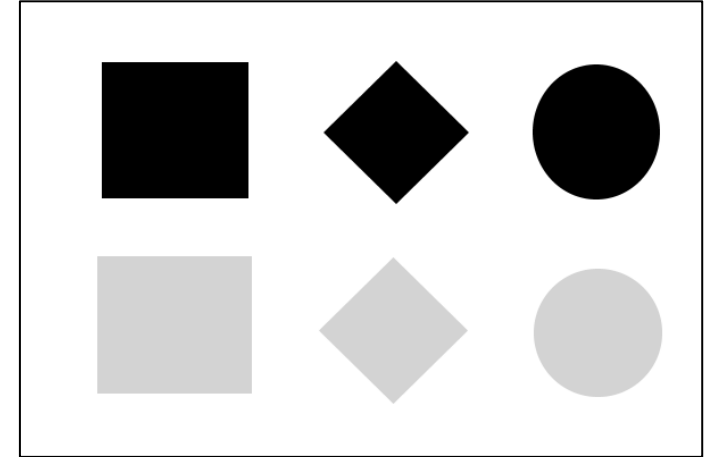
$$M = \begin{bmatrix} \alpha & \beta & (1 - \alpha)t_x - \beta t_y \\ -\beta & \alpha & \beta t_x + (1 - \alpha)t_y \end{bmatrix}$$

$$\alpha = scale \cos(\theta)$$

$$\beta = scale \sin(\theta)$$

- na primjer: rotacija za 90 stupnjeva, pomak u centar slike i smanjivanje za faktor 0.5:

`M = cv2.getRotationMatrix2D(((cols-1)/2.0,(rows-1)/2.0),90,0.5)`



Pitanje

- kakvu geometrijsku transformaciju rade sljedeće matrice?

$$M = \begin{bmatrix} 1 & s_h & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

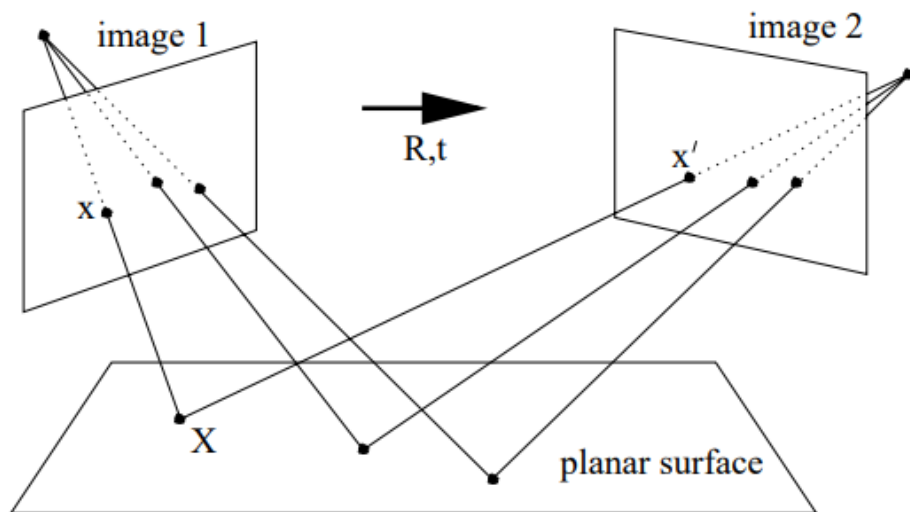
$$M = \begin{bmatrix} 1 & 0 & 0 \\ s_v & 1 & 0 \end{bmatrix}$$

Perspektivna transformacija

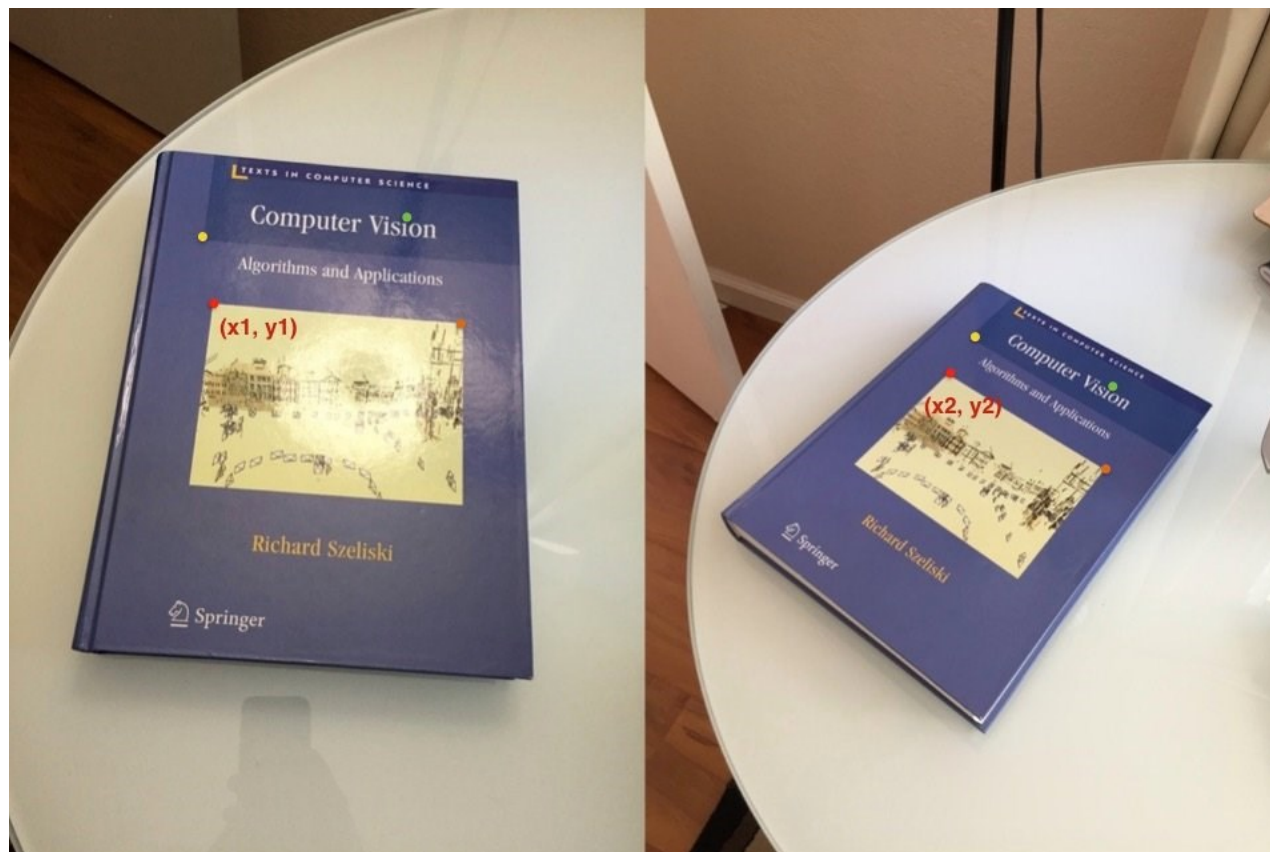
- Perspektivna transformacija je poseban slučaj koji modelira promjene nastale zbog perspektive.
- Nakon ove operacije ravne linije u originalnoj slici će i dalje biti ravne u izlaznoj slici; međutim, ako su bile paralelne, to više ne mora biti slučaj!
- Često se naziva i **homografija**.
- OpenCV funkcije:
 - `cv2.warpPerspective()` – primjena 3x3 transformacijske matrice
 - `cv2.getPerspectiveTransform()` – dobivanje transformacijske matrice iz 4 odgovarajuće točke na slikama (ne smiju ležati na pravcu)

Perspektivna transformacija

“Kako bi ista ravna površina (plakat, stranica, pod...) izgledala iz **drugog pogleda** kamere?”

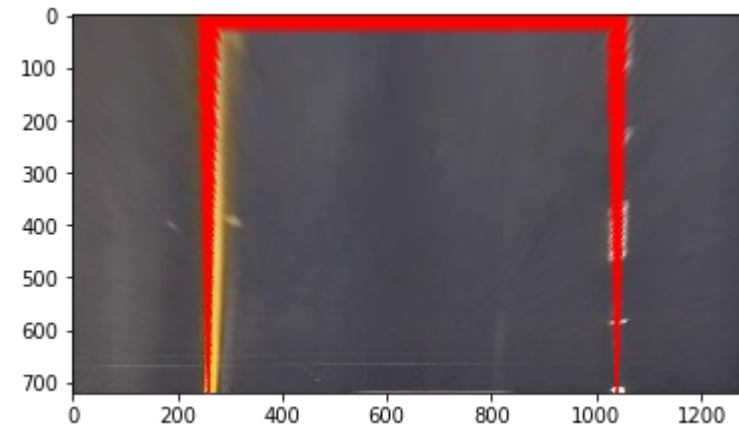
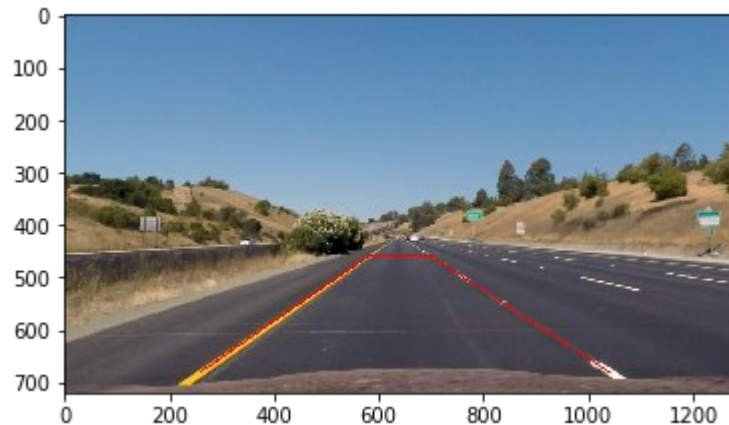


$$\begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix} = H \begin{bmatrix} x_2 \\ y_2 \\ 1 \end{bmatrix} = \begin{bmatrix} h_{00} & h_{01} & h_{02} \\ h_{10} & h_{11} & h_{12} \\ h_{20} & h_{21} & h_{22} \end{bmatrix} \begin{bmatrix} x_2 \\ y_2 \\ 1 \end{bmatrix}$$



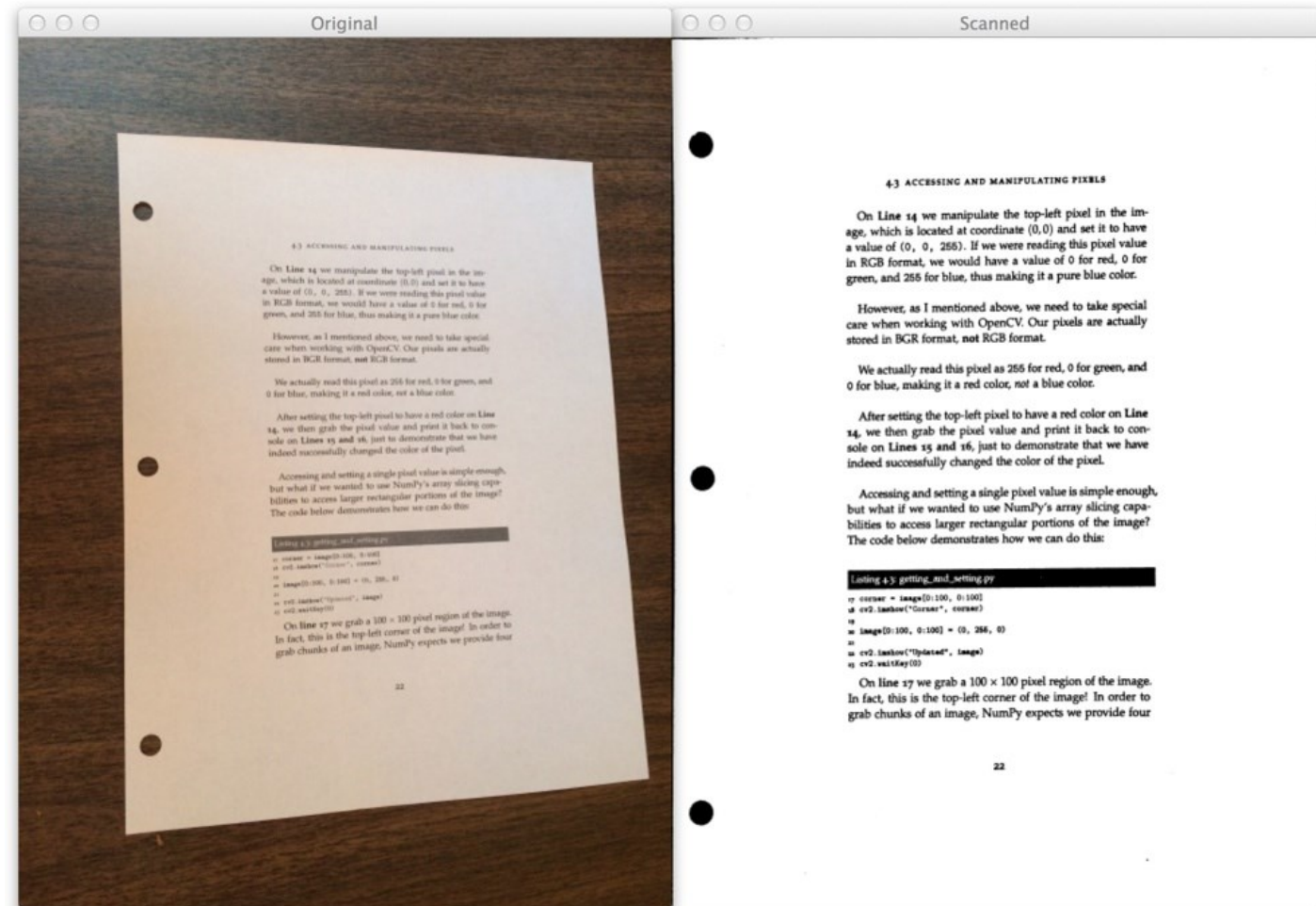
Primjer upotrebe – „pogled odozgor”

- detekcija vozne trake - lakše je iz pogleda odozgor
 - moguće koristiti samo filtre koji su osjetljivi na vertikalne linije u slici
 - linije su paralelne
 - moguća detekcija putem histograma i primjena naprednijih metoda



Primjer upotrebe – „poravnavanje” slike

Aplikacije na smartphoneima
za skeniranje dokumenata



Segmentacija slike

Što je segmentacija slike?

- Segmentacija slike je podjela slike u regije prema nekom pravilu
- Segmentacija je postupak podjele slike u povezane grupe elemenata slike:
 - U područja (regije)
 - U linearne strukture kao što su linije
 - U 2D oblike kao što su krugovi, elipse
- Segmentirana područja (regije) moraju zadovoljavati sljedeće uvjete:
 - Područja moraju biti homogena u odnosu na neko svojstvo (npr. iste boje ili teksture)
 - Granice područja trebaju biti jednostavne
 - Susjedna područja se trebaju značajno razlikovati
 - Područja ne smiju imati male otvore

Što je segmentacija slike?

- Segmentirana područja (regije) P moraju zadovoljavati sljedeće uvjete:

1. $\cup P_i = \text{Slika}$

Unija svih područja je cjelokupna slika

2. $P_i \cap P_j = \emptyset, i \neq j$

Područja se ne preklapaju

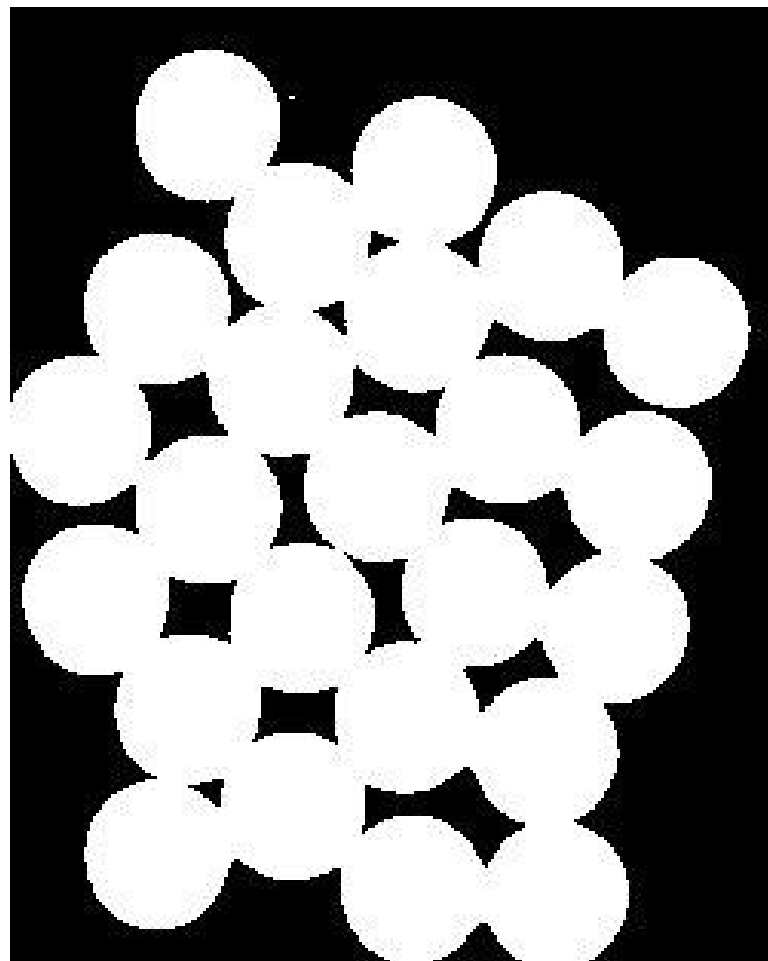
3. $\forall_{P_i} Q(P_i) = \text{TRUE}$

Svako područje ima homogeni sadržaj po nekom kriteriju

4. $Q(P_i \cup P_j) = \text{FALSE},$
 $i \neq j, P_i \text{ je susjed } P_j.$

Unija susjednih područja nema homogen sadržaj

Primjeri segmentacije slike



Primjer semantičke segmentacije slike

- tradicionalna segmentacija ne nastoji „razumjeti” što pojedini segment slike predstavlja
- semantička segmentacija – particioniranje slike u semantički smislene cjeline



Postupci za segmentaciju slike

- Postupci za segmentaciju se mogu podijeliti u sljedeće kategorije
 - Amplitudna segmentacija – postavljanjem praga (ručno/adaptivno)
 - Segmentacija na temelju rubova
 - Segmentacija na temelju rasta područja
 - Segmentacija grupiranjem
 - Segmentacija ekspertnim sustavima
 - Segmentacija neuronskim mrežama

Primjer semantičke segmentacije slike neuronskom mrežom

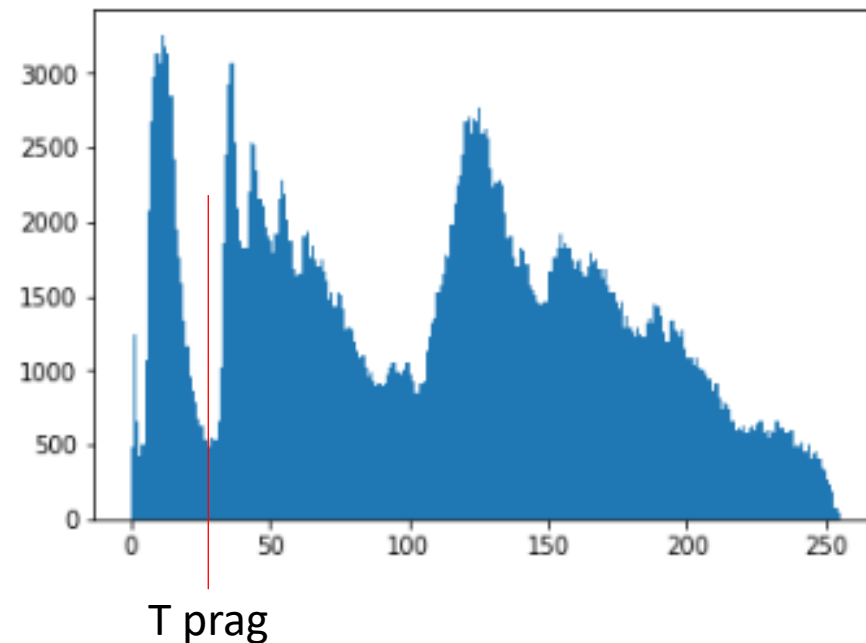
- danas posebno aktivno područje, razvijaju se različite metode temeljene na dubokom učenju
- primjer Mask RCNN → ali o tome nešto više kasnije



Amplitudna segmentacija

Amplitudna segmentacija (thresholding)

- Koristi se kada amplitudne značajke dovoljno precizno definiraju područja na slici
- Za postavljanje praga koristi se najčešće histogram



Amplitudna segmentacija (thresholding) u OpenCV

- Segmentacija s manualnim postavljanjem praga u OpenCV – često se koristi
`thresh1 = cv2.threshold(image, Threshold Value, Max. Value, Threshold Type)`
- Threshold Type – način segmentacije (5 metoda, sve mogu biti korisne u određenim slučajevima)
 - **cv2.THRESH_BINARY** - sve vrijednosti amplitude ispod Threshold Value postavljaju na 0, a one iznad Threshold Value na Max. Value
 - **cv2.THRESH_BINARY_INV** – sve vrijednosti amplitude iznad Threshold Value postavlja na 0, a one ispod na Max. Value
 - **cv2.THRESH_TRUNC** - vrijednosti amplitude ispod Threshold Value ne mijenja, a one iznad postavlja na Threshold Value
 - **cv2.THRESH_TOZERO** - vrijednosti amplitude iznad Threshold Value ne mijenja, a one ispod postavlja na nulu
 - **cv2.THRESH_TOZERO_INV** - vrijednosti amplitude ispod Threshold Value ne mijenja, a one iznad postavlja na nulu

Amplitudna segmentacija (thresholding) u OpenCV

- Segmentacija s manualnim postavljanjem praga u OpenCV
thresh1 = cv2.threshold(image, Threshold Value, Max. Value, Threshold Type)
- 5 različitih metoda

```
import cv2
import numpy as np

# Load our image as greyscale
image = cv2.imread('images/gradient.jpg',0)
cv2.imshow('Original', image)

# Values below 127 goes to 0 (black, everything above goes to 255 (white)
ret,thresh1 = cv2.threshold(image, 127, 255, cv2.THRESH_BINARY)
cv2.imshow('1 Threshold Binary', thresh1)

# Values below 127 go to 255 and values above 127 go to 0 (reverse of above)
ret,thresh2 = cv2.threshold(image, 127, 255, cv2.THRESH_BINARY_INV)
cv2.imshow('2 Threshold Binary Inverse', thresh2)

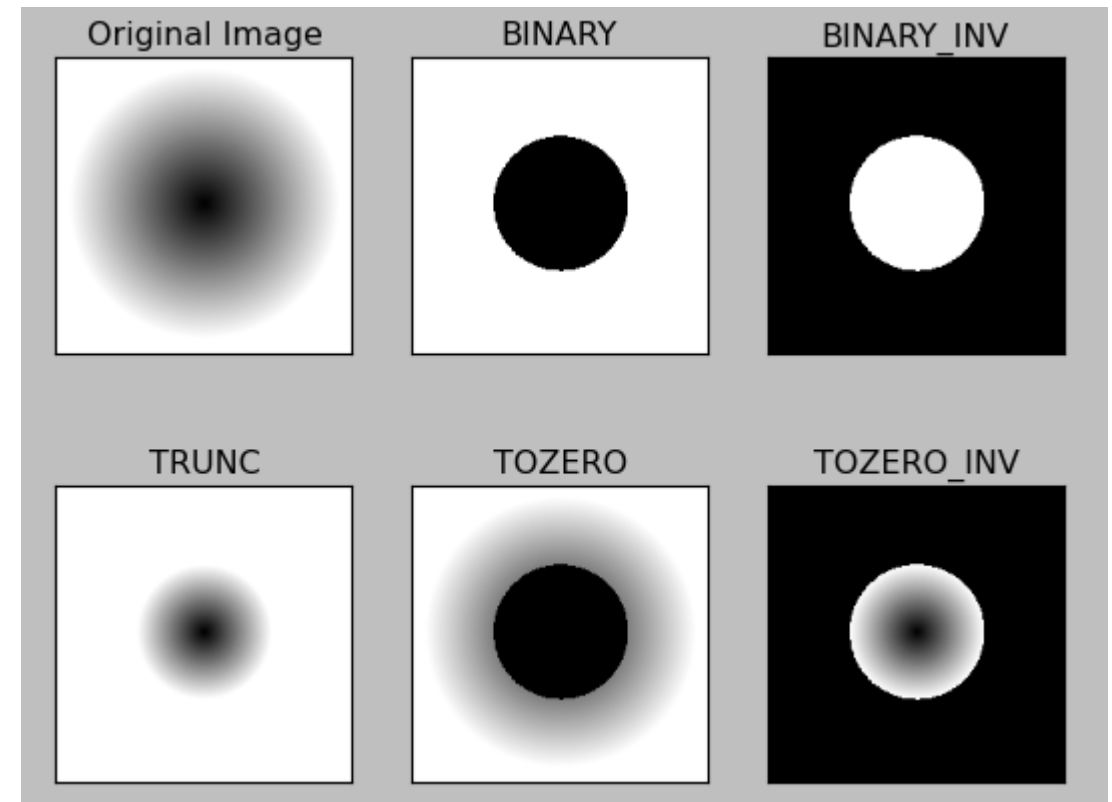
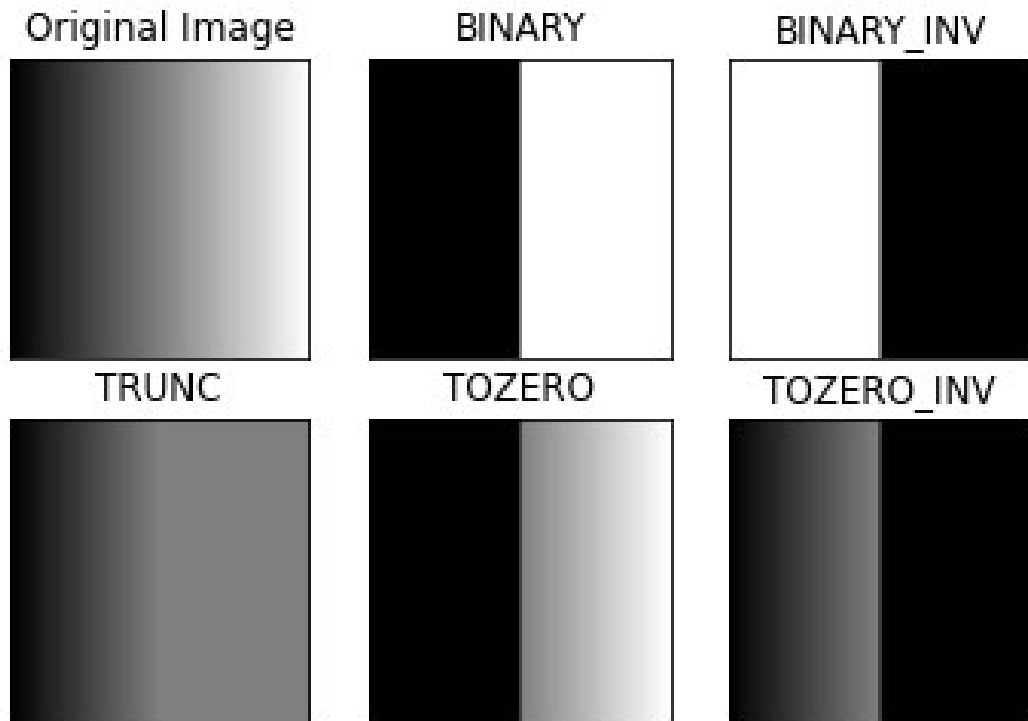
# Values above 127 are truncated (held) at 127 (the 255 argument is unused)
ret,thresh3 = cv2.threshold(image, 127, 255, cv2.THRESH_TRUNC)
cv2.imshow('3 THRESH TRUNC', thresh3)

# Values below 127 go to 0, above 127 are unchanged
ret,thresh4 = cv2.threshold(image, 127, 255, cv2.THRESH_TOZERO)
cv2.imshow('4 THRESH TOZERO', thresh4)

# Reverse of above, below 127 is unchanged, above 127 goes to 0
ret,thresh5 = cv2.threshold(image, 127, 255, cv2.THRESH_TOZERO_INV)
cv2.imshow('5 THRESH TOZERO INV', thresh5)
cv2.waitKey(0)

cv2.destroyAllWindows()
```

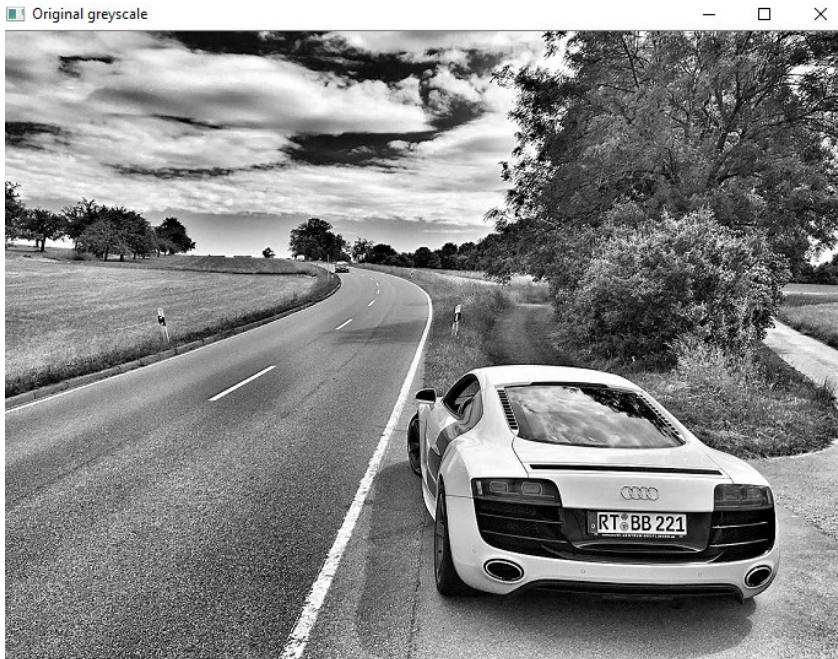
Amplitudna segmentacija (thresholding) u OpenCV – jednostavni primjer



Amplitudna segmentacija (thresholding) u OpenCV – primjer

- Segmentacija s manualnim postavljanjem praga u OpenCV

thresh1 = cv2.threshold(image, Threshold Value, Max. Value, Threshold Type)



```
#SIMPLE THRESHOLDING car_and_road
```

```
import cv2
import numpy as np
```

```
# Load our image
```

```
image = cv2.imread('images/car_and_road.jpg')
cv2.imshow('Original color', image)
```

```
# Load our image as greyscale
```

```
image = cv2.imread('images/car_and_road.jpg',0)
cv2.imshow('Original greyscale', image)
```

```
# Values below 200 goes to 0 (black, everything above goes to 255 (white)
```

```
ret,thresh1 = cv2.threshold(image, 200, 255, cv2.THRESH_BINARY)
cv2.imshow('1 Threshold Binary', thresh1)
```

```
# Values below 200 go to 255 and values above 127 go to 0 (reverse of above)
```

```
ret,thresh2 = cv2.threshold(image, 200, 255, cv2.THRESH_BINARY_INV)
cv2.imshow('2 Threshold Binary Inverse', thresh2)
```

```
# Values above 200 are truncated (held) at 127 (the 255 argument is unused)
```

```
ret,thresh3 = cv2.threshold(image, 200, 255, cv2.THRESH_TRUNC)
cv2.imshow('3 THRESH TRUNC', thresh3)
```

```
# Values below 200 go to 0, above 127 are unchanged
```

```
ret,thresh4 = cv2.threshold(image, 200, 255, cv2.THRESH_TOZERO)
cv2.imshow('4 THRESH TOZERO', thresh4)
```

```
# Reverse of above, below 200 is unchanged, above 127 goes to 0
```

```
ret,thresh5 = cv2.threshold(image, 200, 255, cv2.THRESH_TOZERO_INV)
cv2.imshow('5 THRESH TOZERO INV', thresh5)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```


Amplitudna segmentacija (thresholding) u OpenCV – primjer



- Zaključak – za ovakvu sliku obični thresholding nije baš prikladan
- Potencijalno poboljšanje dodatnim podešavanjem praga

Adaptivna amplitudna segmentacija

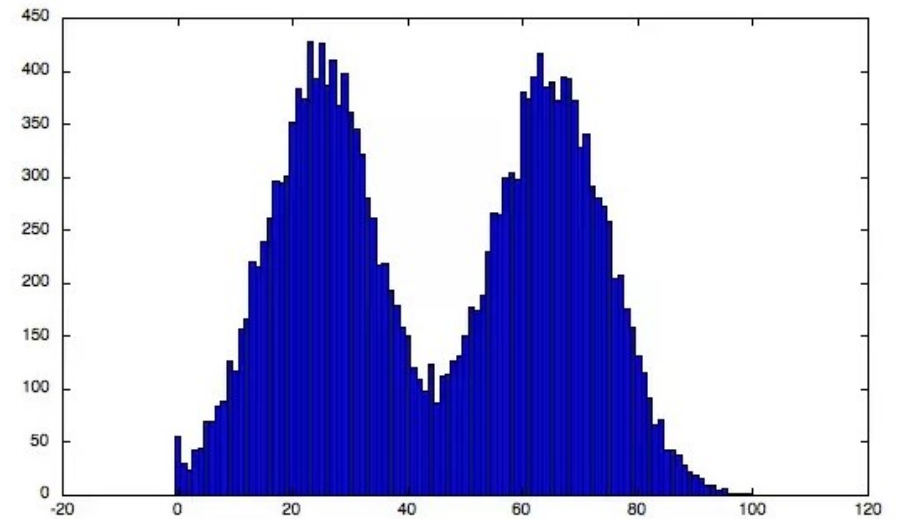
- Adaptivna amplitudna segmentacija
 - Mana „običnog” thresholding postupka – sami moramo odrediti i postaviti prag
 - Zbog toga se češće (kad na slici postoji promjena intenziteta) koristi adaptivni thresholding

thresh = cv2.adaptiveThreshold(image, maxValue, adaptiveMethod, ThresholdType, blocksize, Constant)

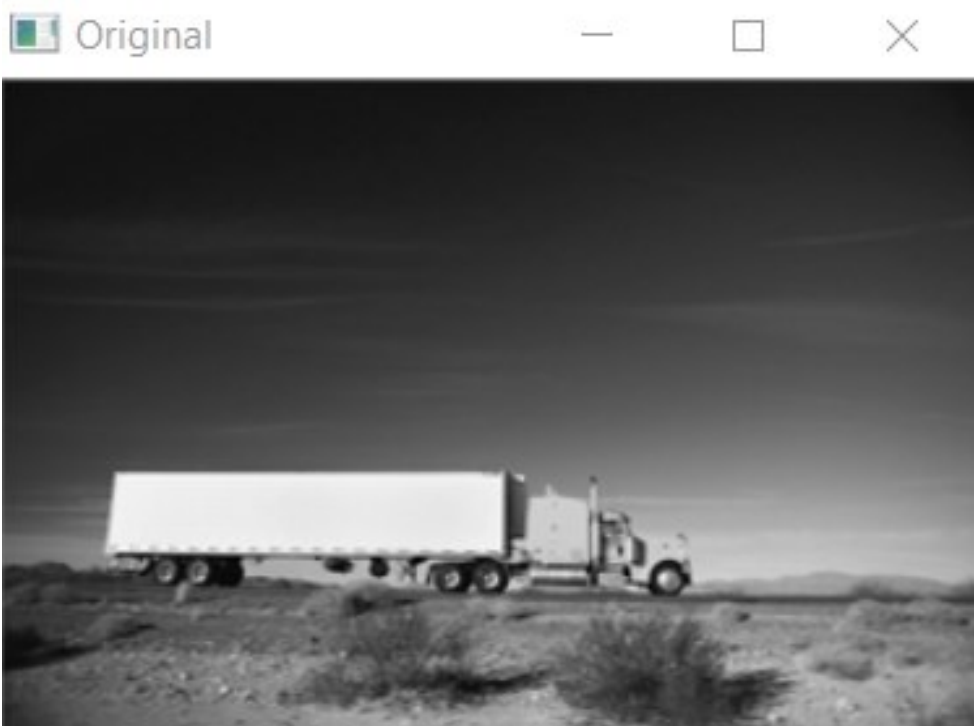
- Image – ulazna slike (grayscale)
- maxValue - maksimalna vrijednost na koju se postavljaju pikseli koji zadovoljavaju postavljeni uvjet
- adaptiveMethod – adaptivni algoritam odabira praga koji se koristi (ADAPTIVE_THRESH_MEAN_C ili ADAPTIVE_THRESH_GAUSSIAN_C)
- ThresholdType – mora biti THRESH_BINARY ili THRESH_BINARY_INV
- blocksize – veličina prozora koji se koristi za proračun praga određenog piksela (3,5,7,...) – neparan broj
- Constant - vrijednost koja se oduzima od srednje vrijednosti ili srednje vrijednosti uz težinske parametre (obično se postavlja na 5)

Adaptivna amplitudna segmentacija

- Adaptivna amplitudna segmentacija OTSU metodom
 - OTSU metoda je optimalna metoda za određivanje praga za slike s histogramima s dva maksimuma (bi-modalni histogram)
 - Ovom metodom se razdvajaju elementi slike na dvije klase, pri čemu je disperzija intenziteta elemenata slike unutar pojedine klase minimalna, a disperzija između klasa maksimalna
 - automatski se računa prag iz histograma
 - NAPOMENA: Ne koristiti funkciju `cv2.adaptiveThreshold` nego `cv2.threshold`



Adaptivna amplitudna segmentacija



```
import cv2
import numpy as np

# Load our new image
image = cv2.imread('images/truck.jpg', 0)
#image = cv2.imread('images/car_and_road.jpg', 0)

cv2.imshow('Original', image)
cv2.waitKey(0)

# Values below 127 goes to 0 (black, everything above goes to 255 (white)
ret, thresh1 = cv2.threshold(image, 200, 255, cv2.THRESH_BINARY)
cv2.imshow('Threshold Binary', thresh1)
cv2.waitKey(0)

# It's good practice to blur images as it removes noise
image = cv2.GaussianBlur(image, (5, 5), 0)

# Using adaptiveThreshold
thresh = cv2.adaptiveThreshold(image, 255, cv2.ADAPTIVE_THRESH_MEAN_C,
                              cv2.THRESH_BINARY, 3, 5)
cv2.imshow("Adaptive Mean Thresholding", thresh)
cv2.waitKey(0)

_, th2 = cv2.threshold(image, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
cv2.imshow("Otsu's Thresholding", th2)
cv2.waitKey(0)

# Otsu's thresholding after Gaussian filtering
blur = cv2.GaussianBlur(image, (5,5), 0)
_, th3 = cv2.threshold(blur, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
cv2.imshow("Gaussian Otsu's Thresholding", th3)
cv2.waitKey(0)

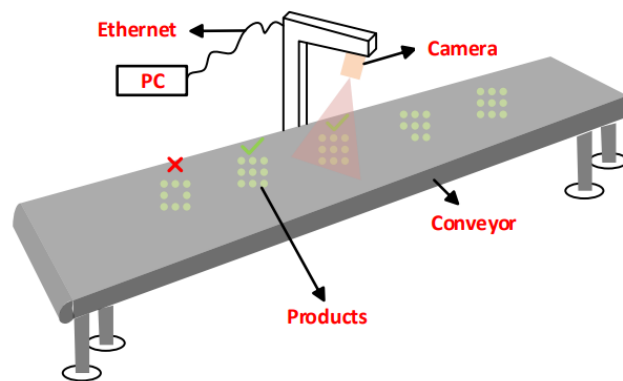
cv2.destroyAllWindows()
```

Adaptivna amplitudna segmentacija OpenCV



Amplitudna segmentacija (thresholding)

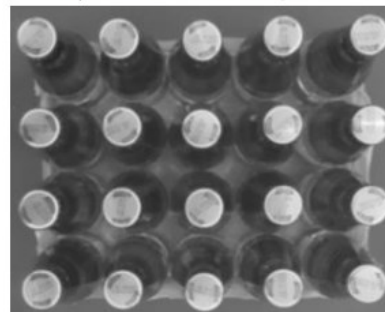
- Obični thresholding dobro funkcionira za slike u kojima su objekti isti/slični i gdje osvjetljenje ne varira – pa se prag može odrediti ručno vizualnim putem ili pomoću Otsu
- čest slučaj u industriji (npr. brojanje proizvoda na traci, kontrolirana rasvjeta)
 - brojanje boca
 - brojanje jaja



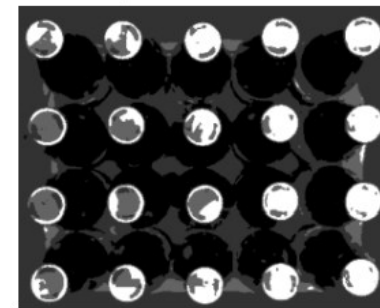
a) Source Image



b) Gaussian Filter



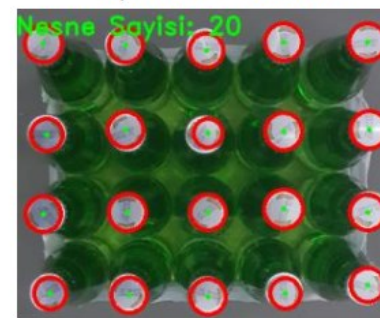
c) S Channel



d) Otsu Threshold



e) Sobel Edge Detection



f) Hough Transform

Segmentacija na temelju rubova

Segmentacija na temelju rubova objekata

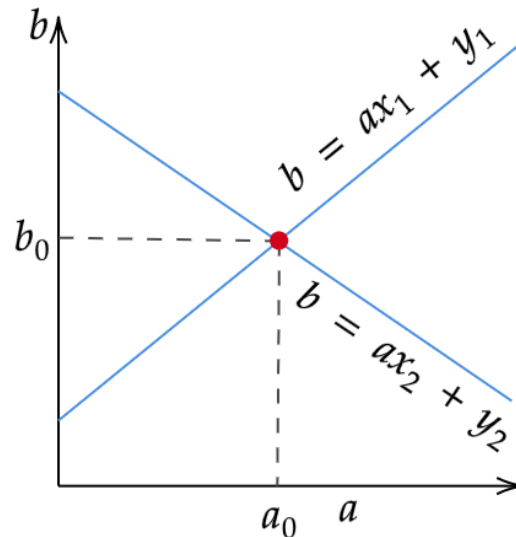
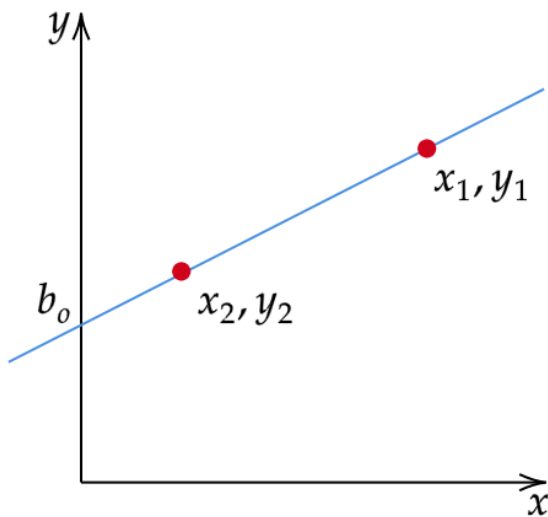
- Pretpostavke
 - Objektni su dovoljno različiti jedni od drugih i razdvajaju ih jasne granice
 - Objektni su dovoljno različiti od pozadine i od nje ih razdvajaju jasne granice
- Ako su te pretpostavke zadovoljene → moguće je pronaći granice objekata na osnovu rubova, tj. naglih diskontinuiteta u intenzitetu (svjetlini)
- Detekcija rubova se provodi nekom od metoda detekcije kao što su Canny, Sobel,...
- Za segmentaciju se nakon toga najčešće koriste metode
 - Lokalno procesiranje
 - Interpolacija krivulja
 - Hough-ova transformacija, RANSAC

Ravne linije

- iako su rubovi i zakrivljene linije prikladne za opis kontura objekata, naš svijet je izgrađen od mnogo „ravnih linija”
- detekcija ravnih linija u slici jedna je osnovnih operacija mnogih naprednijih algoritama u području računalnog vida
 - npr. algoritam detekcije vozne trake
 - algoritam za kalibraciju kamere
- ekstrakciju ravnih linija moguće je napraviti na temelju slika u kojim su detektirani rubovi
 - Houghova transformacija
 - RANSAC

Houghova transformacija

- Koristi se za prvenstveno za detekciju linija, ali i drugih kontura u slici (npr. krugovi, elipse,...)
- Prethodno moraju biti detektirani rubovi – potrebna nam je binarna ulazna slika
- Houghov prostor je 2D prostor gdje horizontalna koordinatna os predstavlja nagib pravca, a vertikalna koordinatna os odsječak pravca



Jednadžba pravca: $y = ax + b$

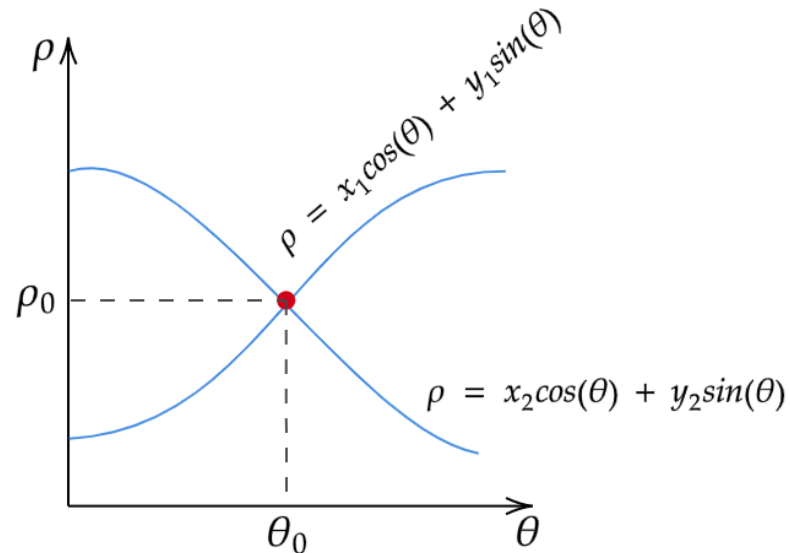
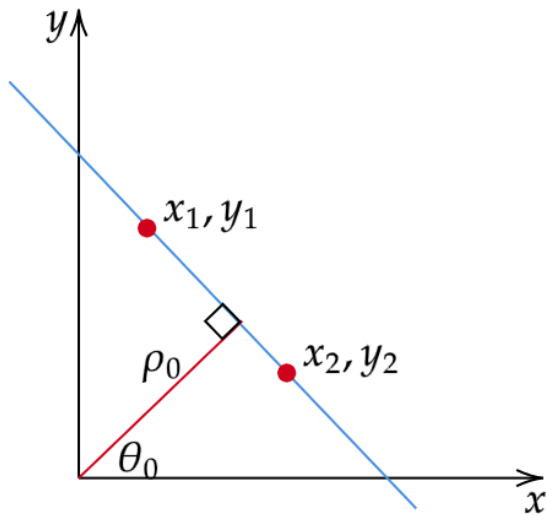
Međutim, problem s ovakvim zapisom je što vertikalni pravci imaju beskonačan nagib a

Houghova transformacija

- Stoga, Hough-ova transformacija koristi drugačiji oblik jednadžbe pravca:

$$\rho = x \cos\theta + y \sin\theta$$

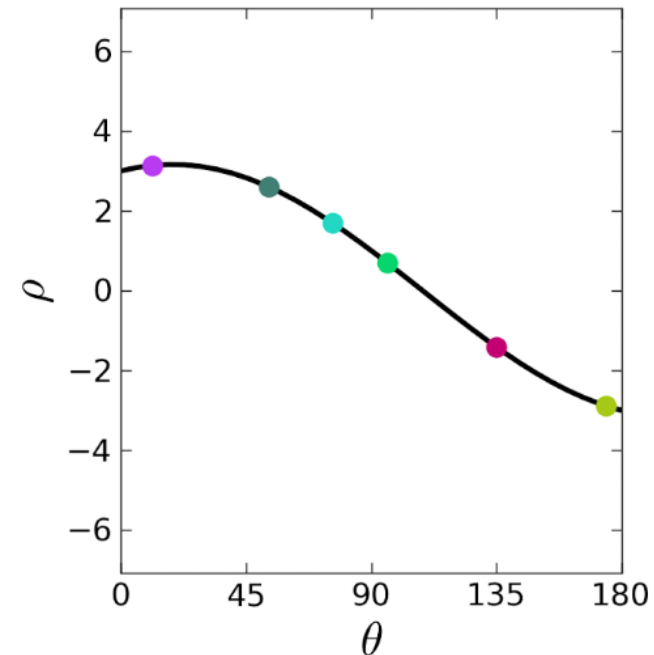
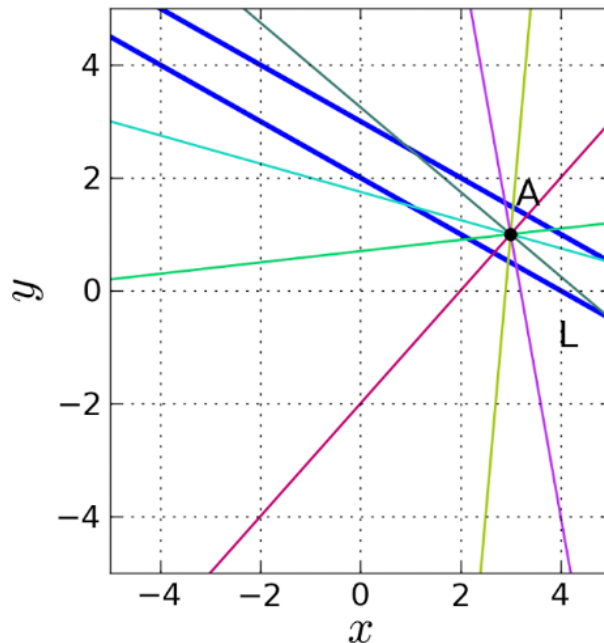
gdje je ρ udaljenost pravca od ishodišta, a θ je kut nagiba normale pravca s obzirom na x os (jednadžba pravca u polarnom obliku)



Preslikava pravac u točku – **Hough-ova transformacija pravca je točka u koordinatnom sustavu (ρ, θ)**

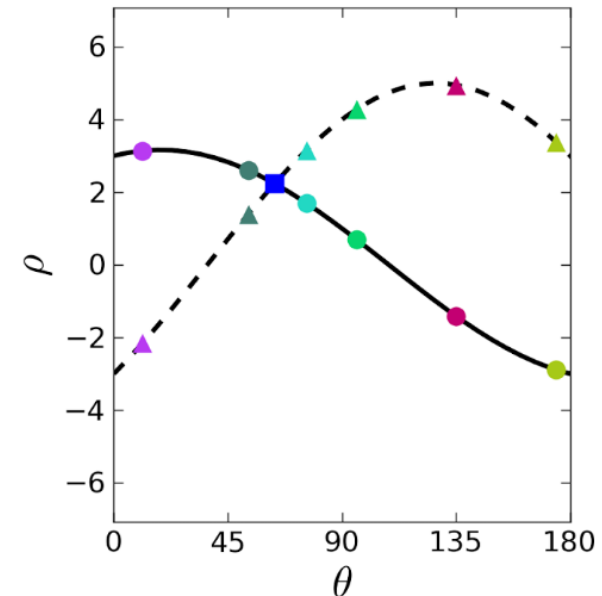
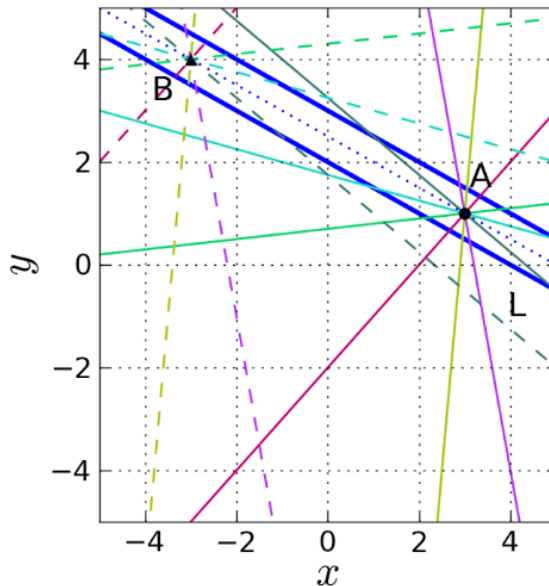
Houghova transformacija

- Kako sada ovo primijeniti na detekciju linije?
- odaberemo jednu točku ulazne slike i predstavimo sve moguće pravce kroz tu točku u Houghovom prostoru – dobiva se sinusoida
- princip primijenimo na sve točke u ulaznoj binarnoj slici



Houghova transformacija

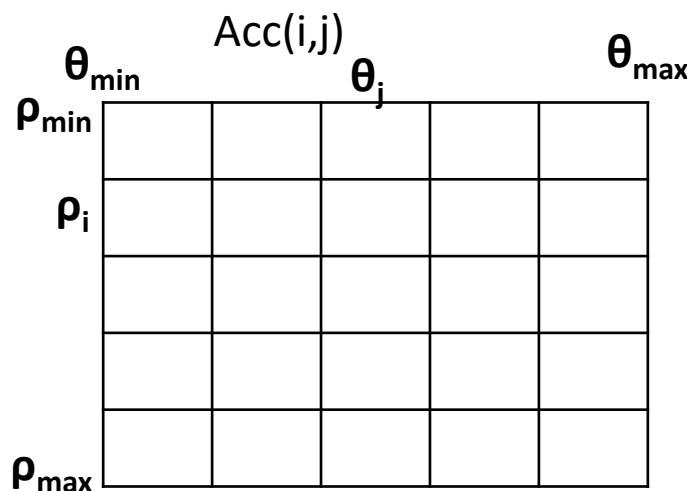
- Svi pravci koji prolaze kroz točku A preslikavaju se u sinusoidu (puna linija na desnoj slici) u KS (ρ, θ)
- Svi pravci koji prolaze kroz točku B preslikavaju se u sinusoidu (isprekidana linija na desnoj slici) u KS (ρ, θ)
- Ako točke A i B leže na istom pravcu (kolinearne su), tada će se sinusoide dobivene Hough-ovom transformacijom sjeći u jednoj točki $H(\theta, \rho)$ koja definira taj pravac



Houghova transformacija - akumulator

- Houghova transformacija koristi polje koje se naziva akumulator za spremanje rezultata
- njegovu dimenziju određuje broj nepoznanica – u slučaju detekcije linije to je matrica jer su dvije nepoznanice (ρ , θ)
- budući da je ovo računalni algoritam, potrebno je diskretizirati obje nepoznanice

Diskretizacija određuje kompromis između brzine, memorijskih zahtjeva i preciznosti detekcije



U akumulator se spremaju *voteovi*

Houghova transformacija – algoritam za detekciju ravnih linija (pravaca)

1. Provedi detekciju rubova na ulaznoj slici
2. Pripremi akumulator – odredi način diskretizacije te postavi $\text{Acc}(i,j)=0 \quad \forall (i,j)$
3. Proces glasanja

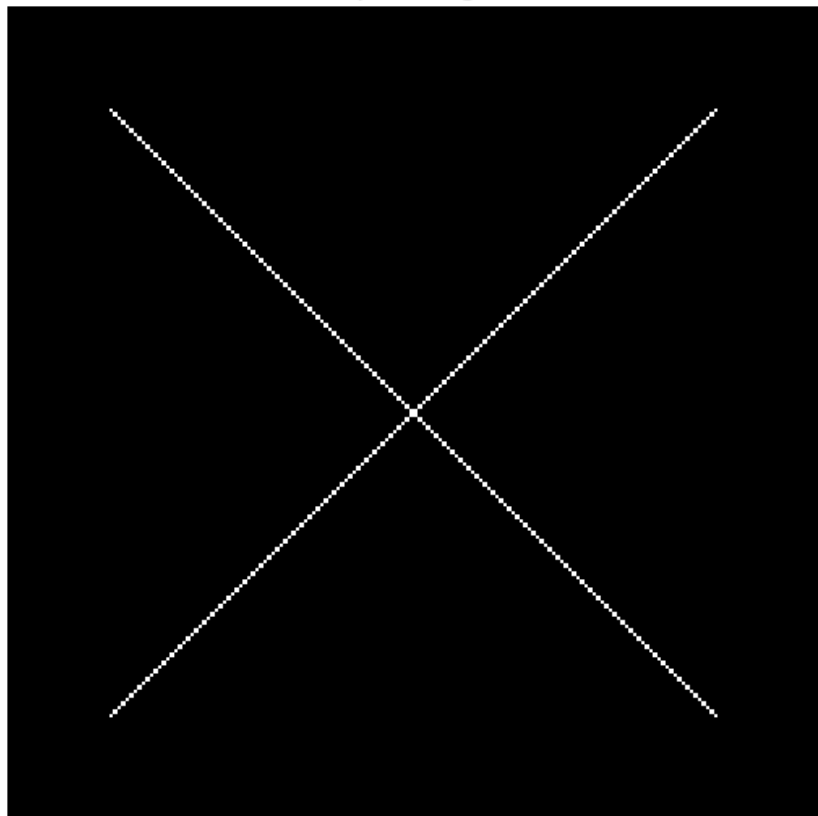
- za svaku točku ulazne slike i za svaki mogući kut θ_j izračunaj ρ

$$\rho = x \cos\theta_j + y \sin\theta_j$$

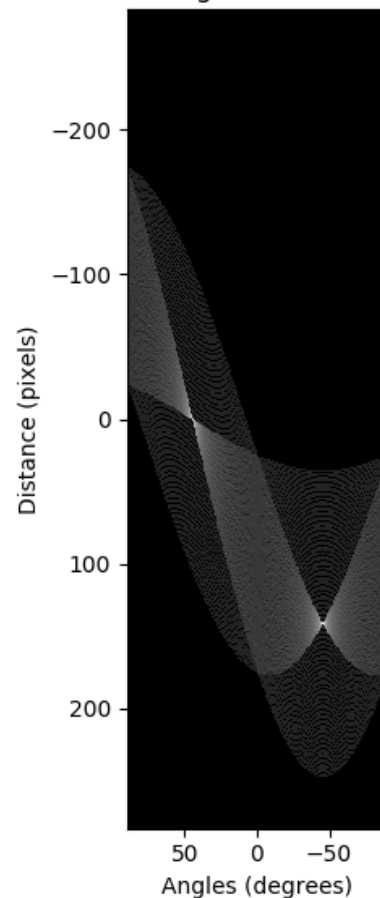
- pronajdi najbliži ρ_i u akumulatoru i inkrementiraj vrijednost $\text{Acc}(i,j)$ u akumulatoru
 - svaki element akumulatora kaže koliko točaka/piksela je „glasalo” za potencijalnu liniju u ulaznoj slici s parametrima (ρ_i, θ_j)
4. Pronajdi peakove u akumulatoru – lokalni maksimumi koji ukazuju na postojanje linije u ulaznoj slici
 - obično se primijeni metoda praga

Houghova transformacija – primjer 1

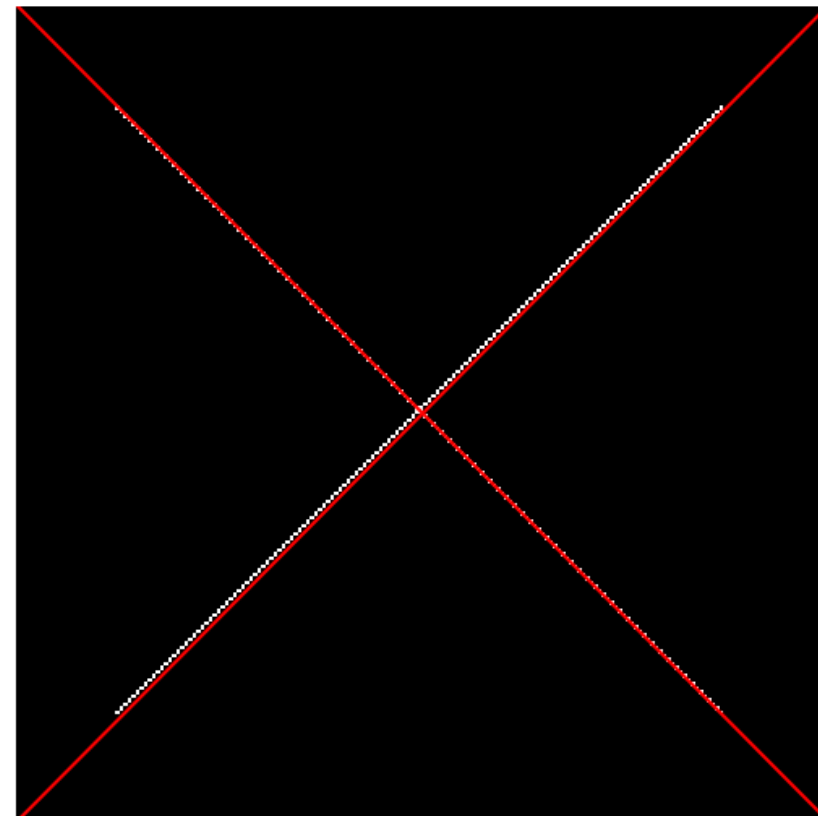
Input image



Hough transform

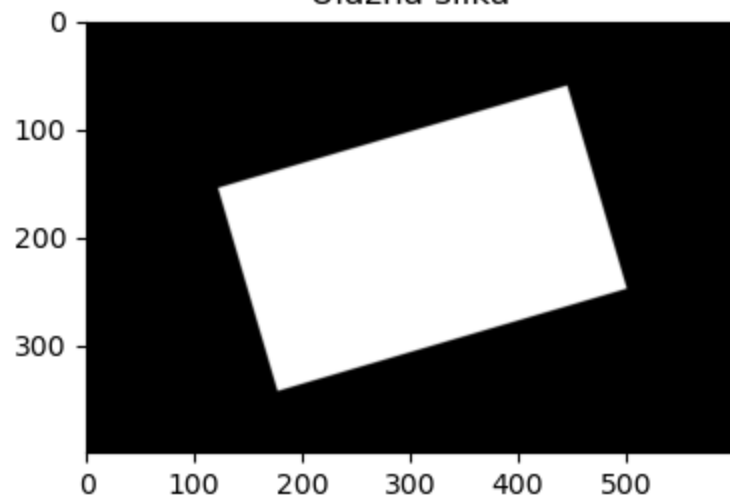


Detected lines

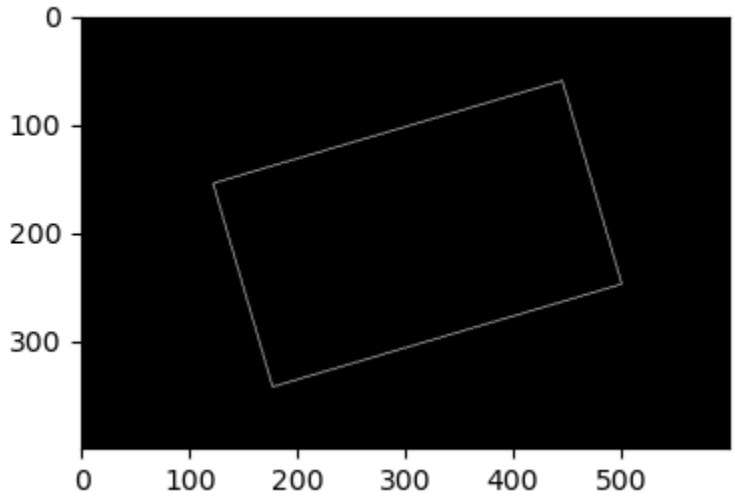


Houghova transformacija - primjer

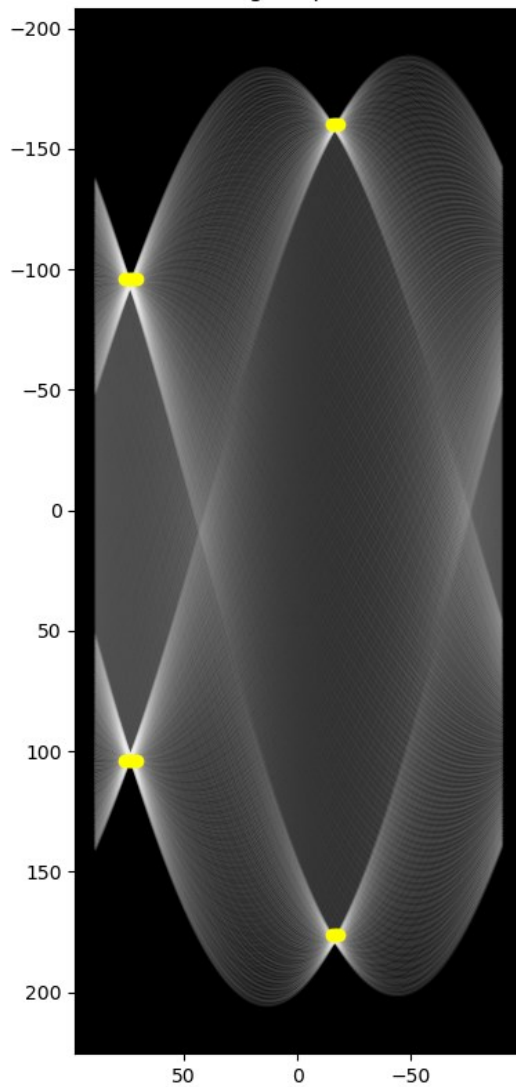
Ulazna slika



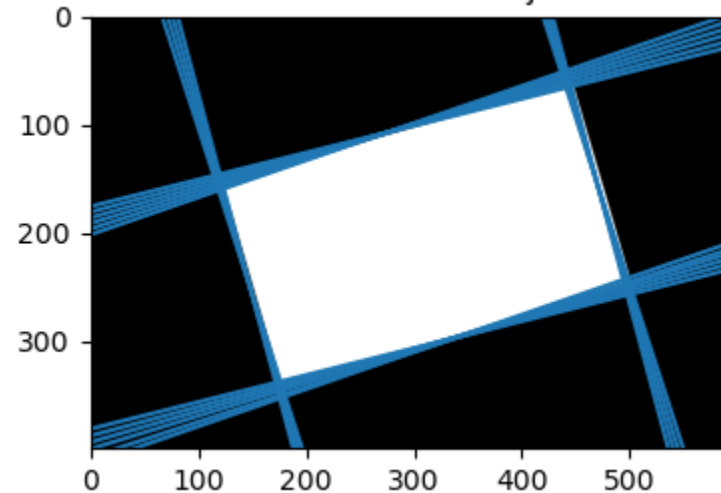
Detektirani rubovi



Houghov prostor



Detektirane linije



Houghova transformacija - primjer



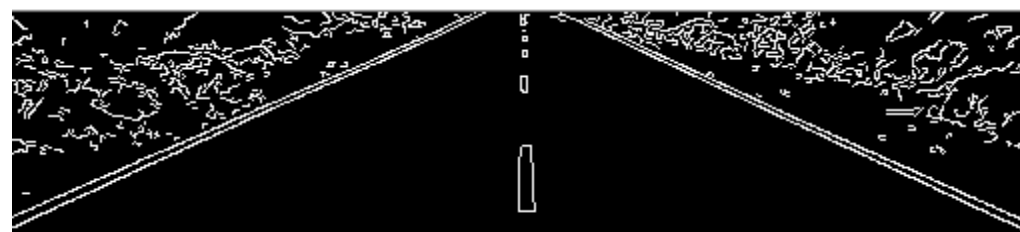
Regija od
interesa (RoI)



Gaussov filter



Canny edge
detektor



Houghova transformacija - primjer



Prikazana su prva dva
maksimuma u akumulatoru

Houghova transformacija u OpenCV

`cv2.HoughLines(binarized/thresholded image,  $\rho$  accuracy,  $\theta$  accuracy, Threshold)`

- Ulaz je binarizirana slika s detektiranim rubovima (najčešće detekcija rubova pa thresholding)
- Potrebno je dati točnost naspram oba parameta Houghove transformacije ρ i θ , npr.
 - ρ accuracy = 1 element slike
 - θ accuracy = $\pi / 180$, što odgovara 1 stupanj
- Threshold je ovdje minimalan broj točaka koji moraju biti na liniji da bi se ona smatrala linijom
 - Npr. 100 $\rightarrow$ parametri (ρ_i, θ_j) se proglašavaju parametrima pravca ako je $\text{Acc}(i,j) > 100$

Houghova transformacija u OpenCV

```
import cv2
import numpy as np

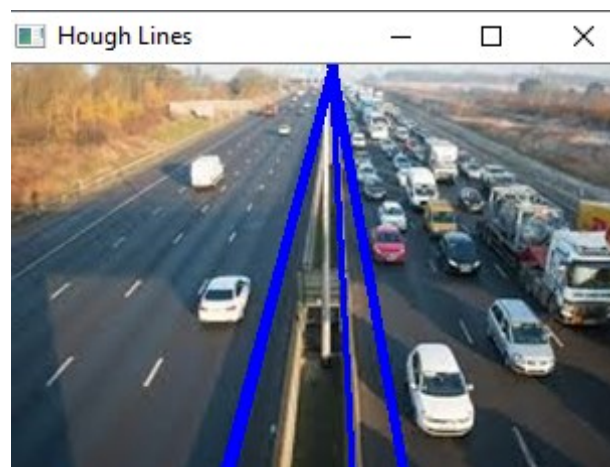
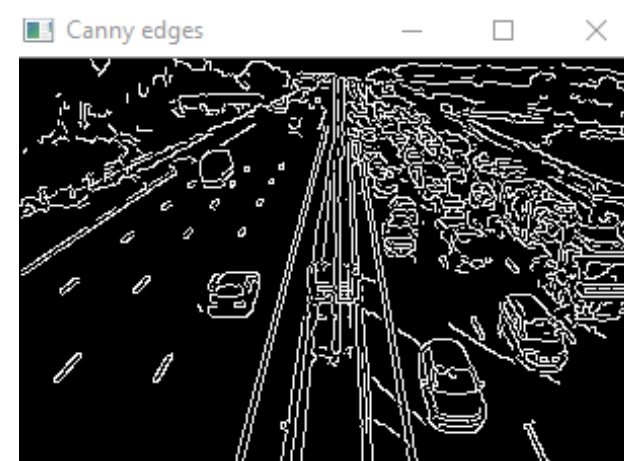
image = cv2.imread('images/autoput.jpg')
cv2.imshow('Original', image)

# Grayscale and Canny Edges extracted
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
edges = cv2.Canny(gray, 100, 170, apertureSize = 3)
cv2.imshow('Canny edges', edges)

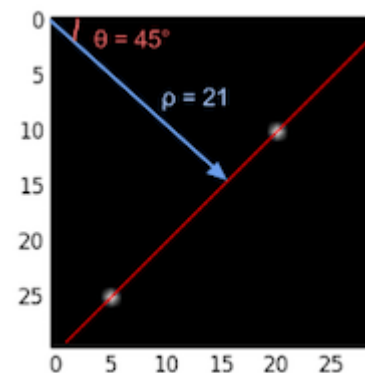
# Run HoughLines using a rho accuracy of 1 pixel
# theta accuracy of np.pi / 180 which is 1 degree
# Our line threshold is set to 110 (number of points on line)
lines = cv2.HoughLines(edges, 1, np.pi / 180, 110)

# We iterate through each line and convert it to the format
# required by cv.lines (i.e. requiring end points)
for rho, theta in lines[0]:
    a = np.cos(theta)
    b = np.sin(theta)
    x0 = a * rho
    y0 = b * rho
    x1 = int(x0 + 1000 * (-b))
    y1 = int(y0 + 1000 * (a))
    x2 = int(x0 - 1000 * (-b))
    y2 = int(y0 - 1000 * (a))
    cv2.line(image, (x1, y1), (x2, y2), (255, 0, 0), 2)

cv2.imshow('Hough Lines', image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



OpenCV ishodište



Houghova transformacija primjer

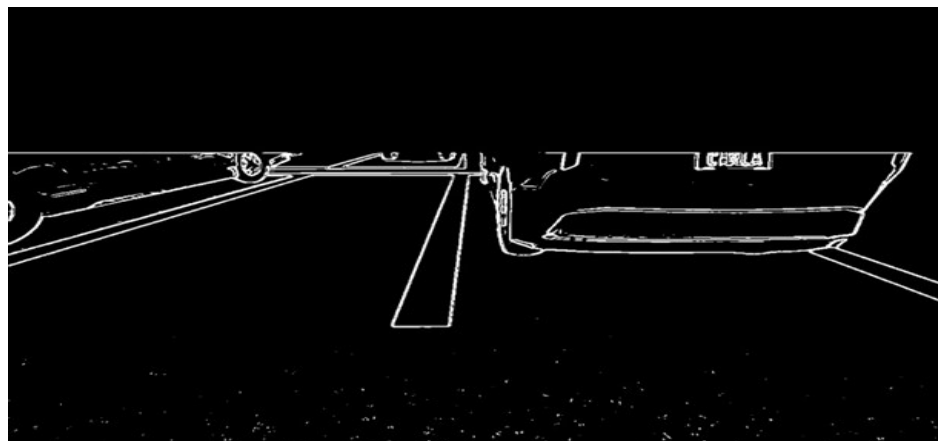
Okvir s
bočne
kamere



ROI +
median
filter



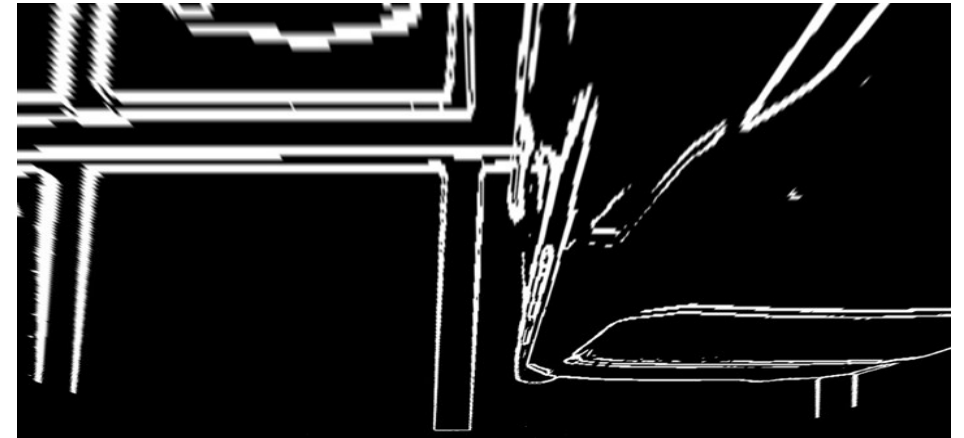
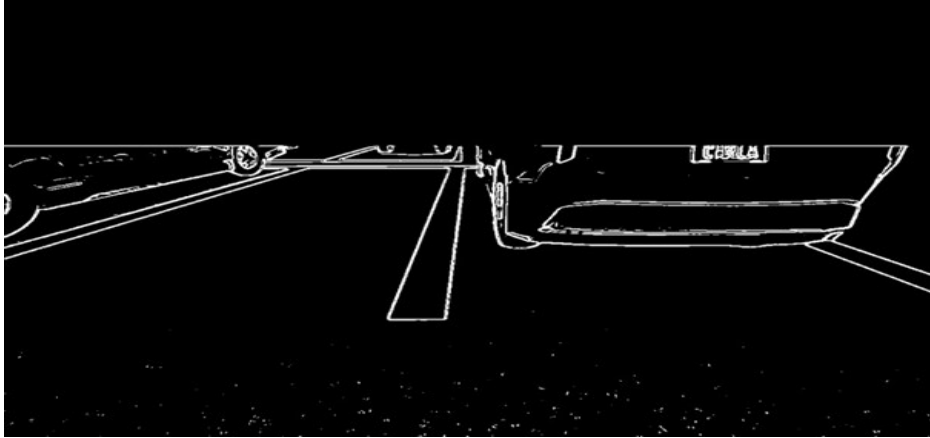
Sobel



Sobel + transformacija perspektive

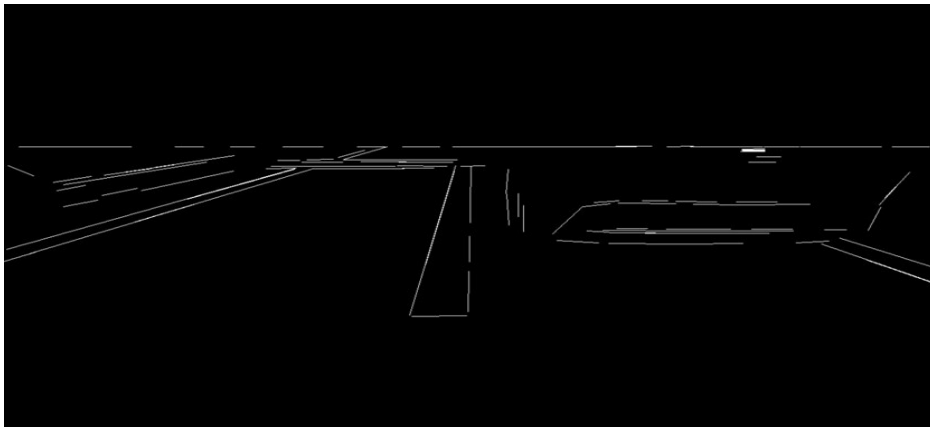
Houghova transformacija primjer

Sobel



Sobel + transformacija perspektive

Hough
transf.
nakon
Sobela



Transformacija perspektive samo linija nakon
Houghove transformacije

Houghova transformacija za detekciju krugova

- osim ravnih linija, u našem svijetu postoji mnogo kružnih ili eliptičnih oblika
- Jednadžba kružnice $(x - a)^2 + (y - b)^2 = R^2$ u parametarskom obliku glasi:

$$x = a + R \cos \theta$$

$$y = b + R \sin \theta$$

- R je polumjer kružnice, a točka C s koordinatama (a,b) je središte kružnice
 - Kako se θ mijenja od 0 do 2π , tako točke (x,y) obilaze kružnicu
- Ako postoje elementi slike koji pripadaju kružnici (ako slika sadrži puno točaka koje pripadaju kružnici), postupkom segmentacije potrebno je odrediti set parametara (a,b,R) za tu kružnicu
- Isti princip kao i kod detekcije linija; razlika je u akumulatoru koji je sada 3D matrica

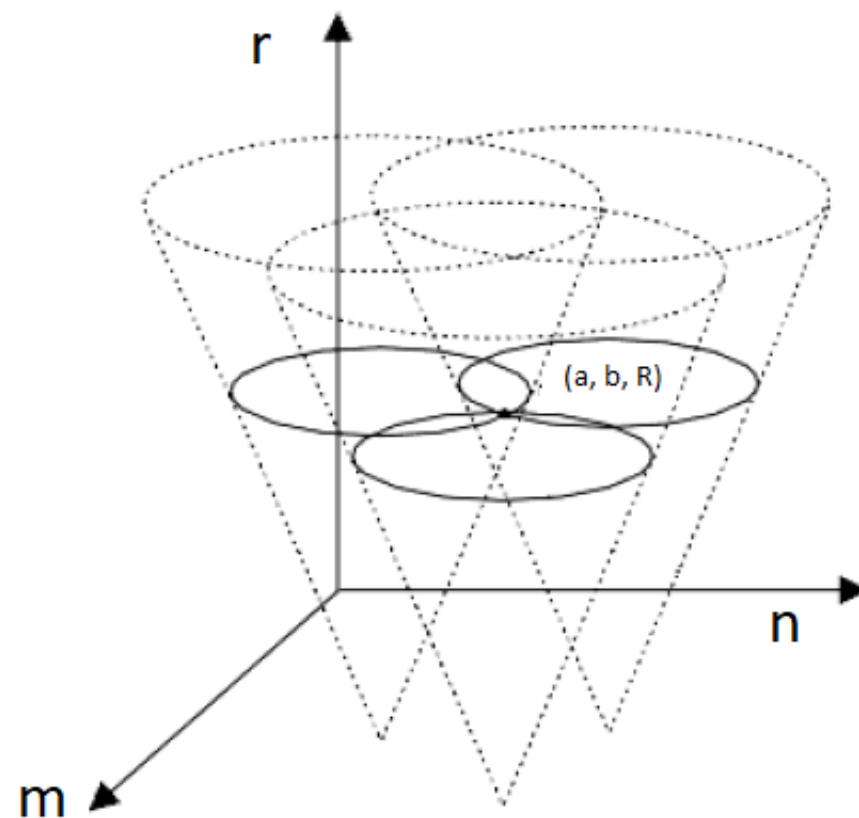
Houghova transformacija za detekciju krugova

- Koordinate u akumulacijskom polju mogu se izračunati kao:

$$a = x - R \cos\theta$$

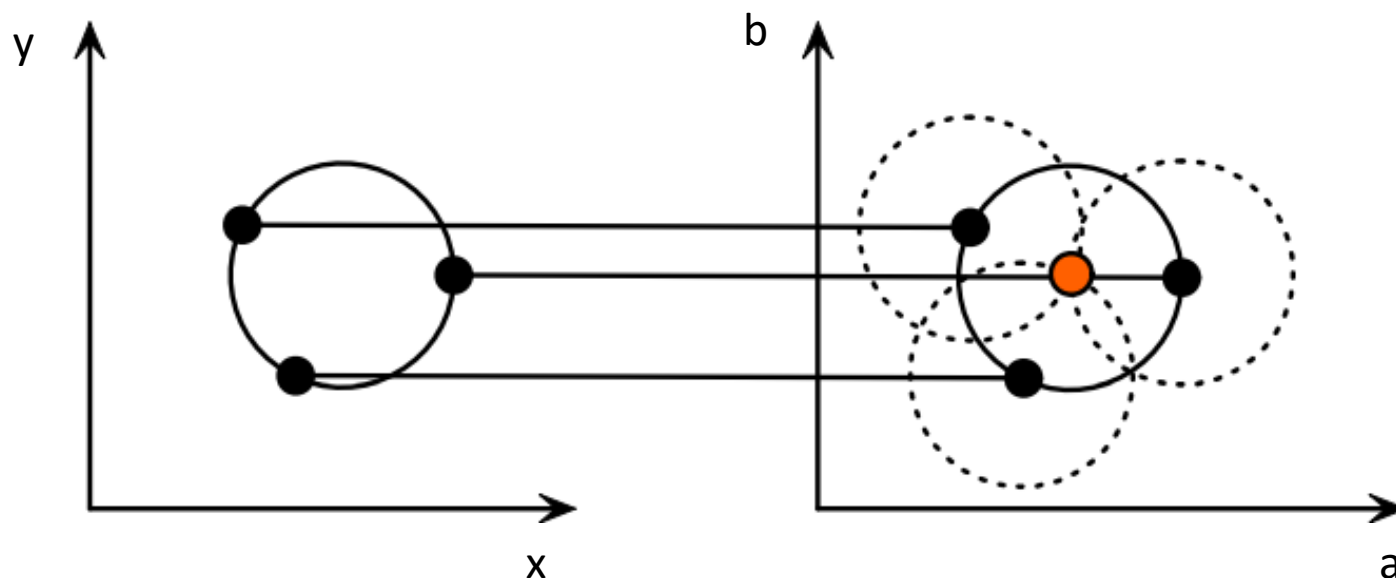
$$b = y - R \sin\theta$$

- Svaki se piksel (x,y) prostora slike pridružuje se plaštu stošca koji u (a,b,R) parametarskom prostoru predstavlja sve kružnice koje prolaze kroz točku (x,y)
- Svakom mogućem radijusu odgovara točno jedna kružnica stožaste površine



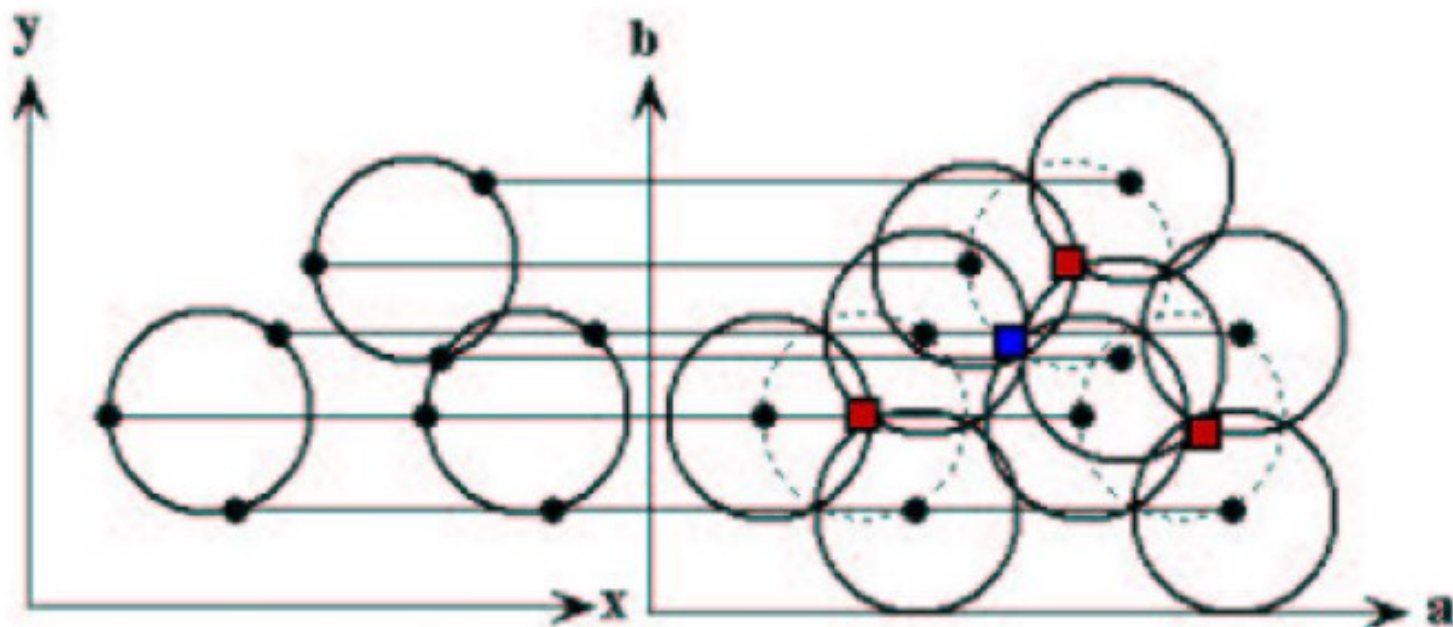
Houghova transformacija za detekciju krugova

- Za poznati R , svaka točka na obodu kruga bit će Hough-ovom transformacijom preslikana u kružnicu u (a,b) prostoru
- Kada se ovo provede za sve točke kružnice, dobiva se skup kružnica istog polumjera koje se sijeku u jednoj točki, čije koordinate određuju koordinate središta početne kružnice



Houghova transformacija za detekciju krugova

- Za poznati R , svaka točka na obodu kruga bit će Hough-ovom transformacijom preslikana u kružnicu u (a,b) prostoru
- Kada se ovo provede za sve točke kružnice, dobiva se skup kružnica istog polumjera koje se sijeku u jednoj točki, čije koordinate određuju koordinate središta početne kružnice



Houghova transformacija za detekciju krugova u OpenCV

```
cv2.HoughCircles(image, method, dp, MinDist, param1,  
param2, minRadius, MaxRadius)
```

- method – podržano samo cv2.HOUGH_GRADIENT
- dp – inverz rezolucije akumulatora $A(i,j)$
- MinDist – minimalni razmak između središta detektiranih kružnica
- param1 – vrijednost gradijenta za detekciju rubova
- param2 – prag akumulatora – manja vrijednost znači da će više kružnica biti detektirano (neki i pogrešno)
- minRadius – najmanji dozvoljeni radius
- MaxRadius – najveći dozvoljeni radius

Houghova transformacija za detekciju krugova u OpenCV

```
import cv2
import numpy as np
import cv2.cv as cv

image = cv2.imread('images/sign.jpg')
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
cv2.imshow('Original', image)
cv2.waitKey(0)
cv2.imshow('Gray', gray)
cv2.waitKey(0)

blur = cv2.medianBlur(gray, 5)
cv2.imshow('Blur', blur)
cv2.waitKey(0)

circles = cv2.HoughCircles(blur, cv.CV_HOUGH_GRADIENT, 1.5, 10)
#circles = cv2.HoughCircles(gray, cv.CV_HOUGH_GRADIENT, 1, 10)

circles = np.uint16(np.around(circles))

for i in circles[0,:]:
    # draw the outer circle
    cv2.circle(image, (i[0], i[1]), i[2], (255, 0, 0), 2)

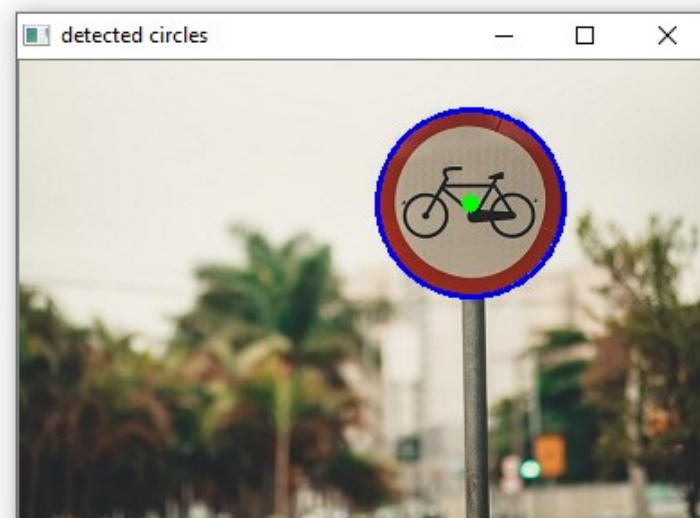
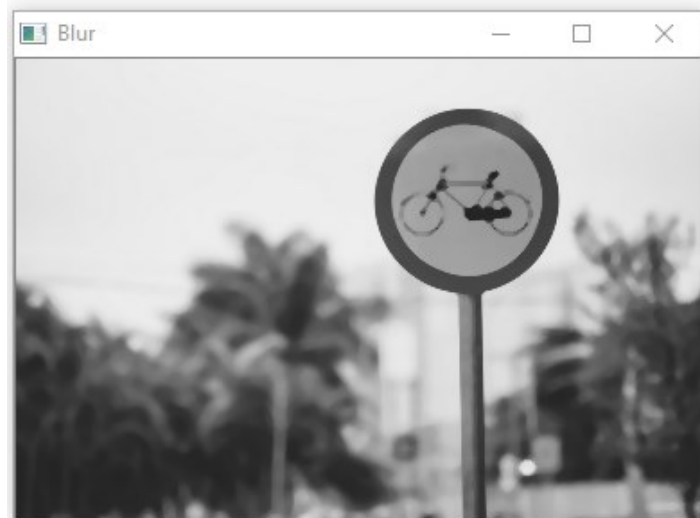
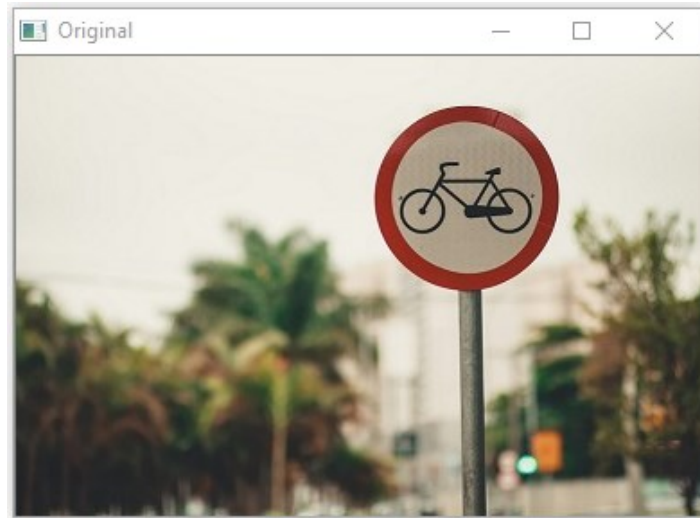
    # draw the center of the circle
    cv2.circle(image, (i[0], i[1]), 2, (0, 255, 0), 5)

cv2.imshow('detected circles', image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Koraci:

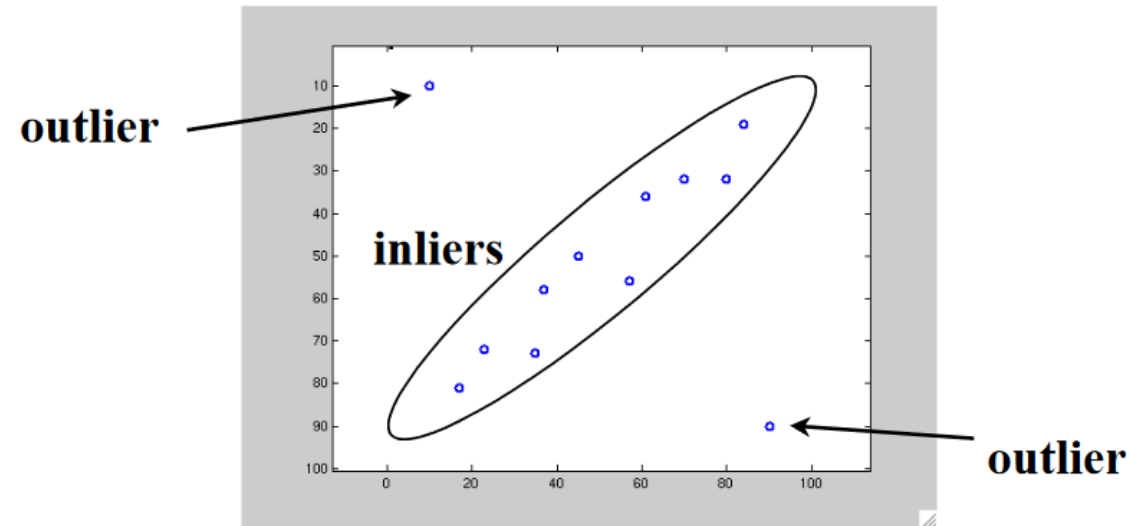
- Pretvaranje BGR slike u monokromatsku (GRAY)
- Primjena median filtra za smanjenje šuma
- Primjena Houghove transformacije s prethodnim proračunom gradijenta za detekciju ruba
- Crtanje kruga i centra kruga

Houghova transformacija za detekciju krugova



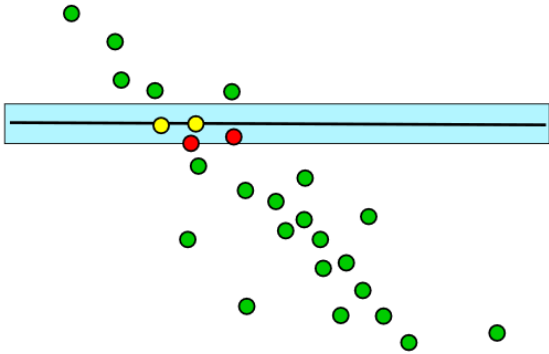
Random Sample Consensus - RANSAC

- RANSAC (engl. RANdom SAmple Consensus)
 - Koristi se za detekciju linija i kontura
 - Potrebno je prethodno provesti detekciju rubova
 - Iterativna metoda procjene parametara unaprijed definiranog matematičkog modela na temelju podataka u kojima su prisutne stršeće vrijednosti (engl. outliers)
 - Npr. metoda najmanjih kvadrata (engl. least squares) je osjetljiva na prisustvo stršećih vrijednosti u podacima

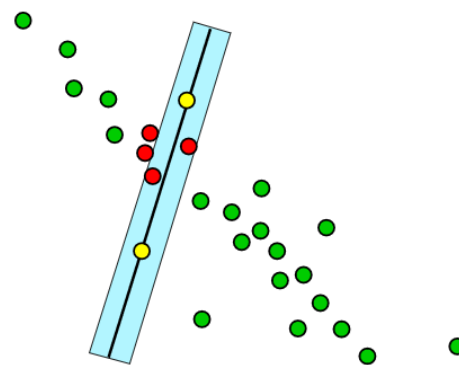


Random Sample Consensus - RANSAC

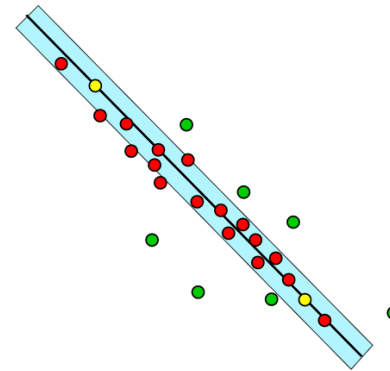
- Primjer: RANSAC + detekcija linije
 - Odabere se N podataka (ovdje $N=2$) iz dostupnih podataka, povuče se pravac kroz njih te se prebroji koliko podataka se nalazi unutar neke granice od dobivenog pravca
 - Postupak se ponavlja



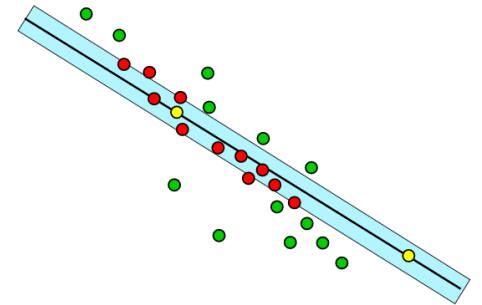
Broj inliera = 4



Broj inliera = 6



Broj inliera = 19



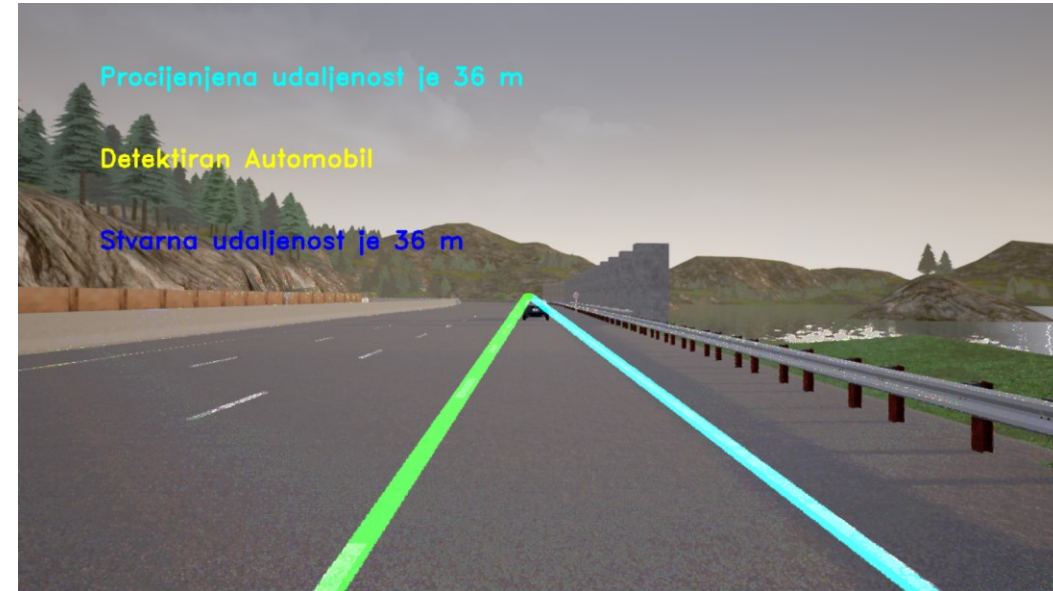
Broj inliera = 13

Random Sample Consensus - RANSAC

- Nakon što se dostigne maksimalni broj iteracija, odabire se rješenje s najviše inliera te se obično procjene parametri LS metodom samo na inlierima
- RANSAC je nedeterministički algoritam u smislu da generira prihvatljiv rezultat s određenom vjerojatnošću (koja se povećava s brojem iteracija algoritma)
- Povećanje broja iteracija → veća vjerojatnost točne detekcija → nakon određenog broja iteracija neće se značajno povećavati preciznost rezultata
- Prednosti algoritma
 - Vrlo je robustan na prisustvo outliera
 - Jednostavan je i efikasan
- Nedostatci
 - Potrebno je definirati parametre (poput broja iteracija i veličinu praga za definiranje inliera),
 - Može potrajati (npr. kod složenijih modela ili velikih skupova podataka)

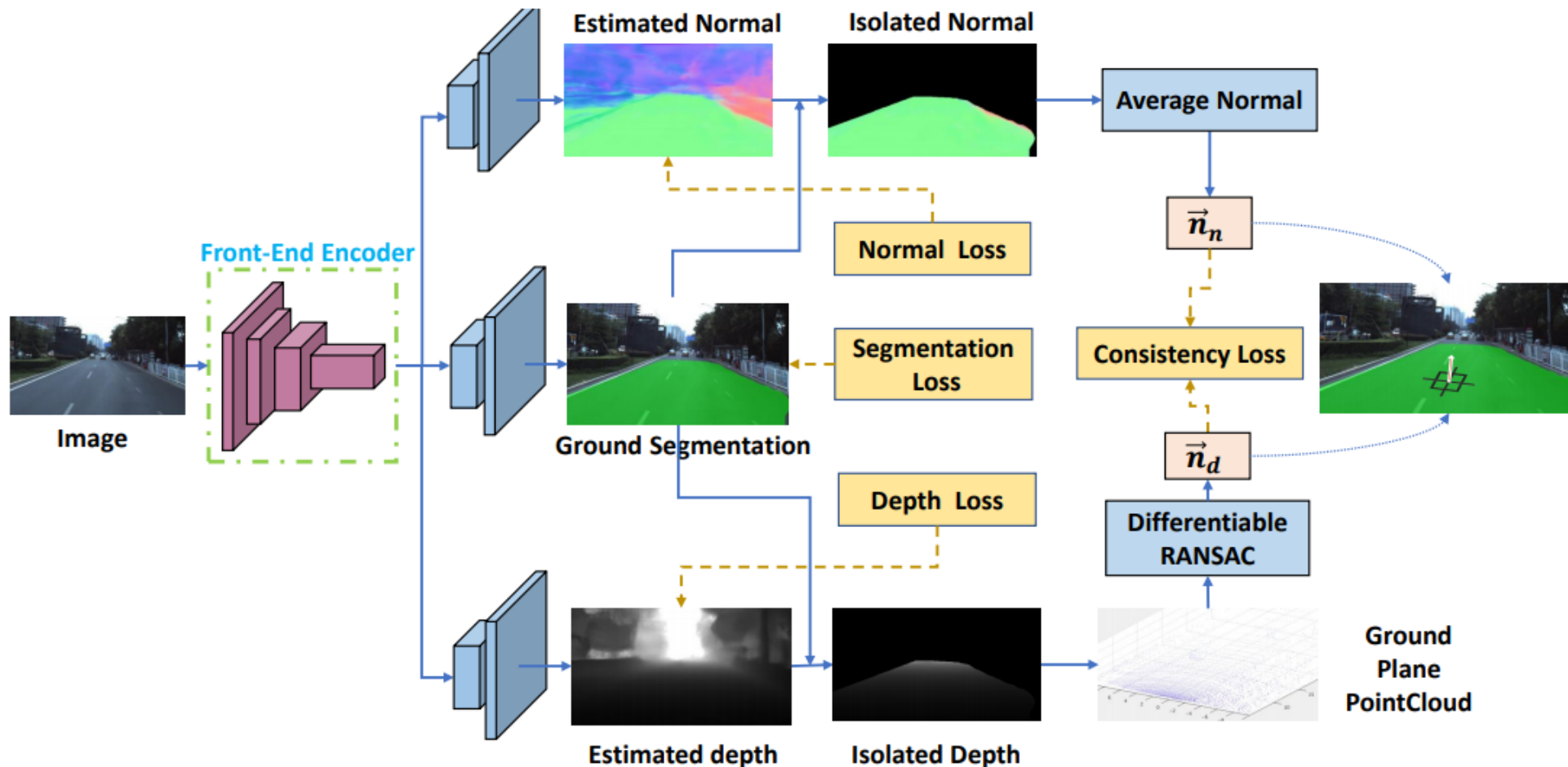
Random Sample Consensus - RANSAC

- Ulaz u RANSAC je binarna slika (i to samo ROI) dobivena detekcijom rubova Canny operatorom
 - Na taj način smanjen je broj ulaznih podataka za RANSAC → manji broj iteracija za detekciju linija
- Dvije linije vlastite vozne trake (dvije oznake)
 - Primjenjuje se RANSAC dva puta (po 250 iteracija)
 - Kad se detektira prva kolnička oznaka tada se pozitivni elementi slike brišu s binarne slike koja je ulaz prilikom drugog izvođenja RANSAC-a
- Tako se sprječava ponovna detekcija iste kolničke oznake, a ujedno se i smanjuje broj ulaznih podataka za drugo izvođenje RANSAC algoritma
- Često na kolniku nalaze isprekidane kolničke oznake
 - koristi se dobivena jednadžba pravca kako bi se detektirana kolnička oznaka produžila do dna slike
 - također, pomoću jednadžbi pravaca pronalazi se točka u kojoj se detektirani pravci sijeku (vanishing point)



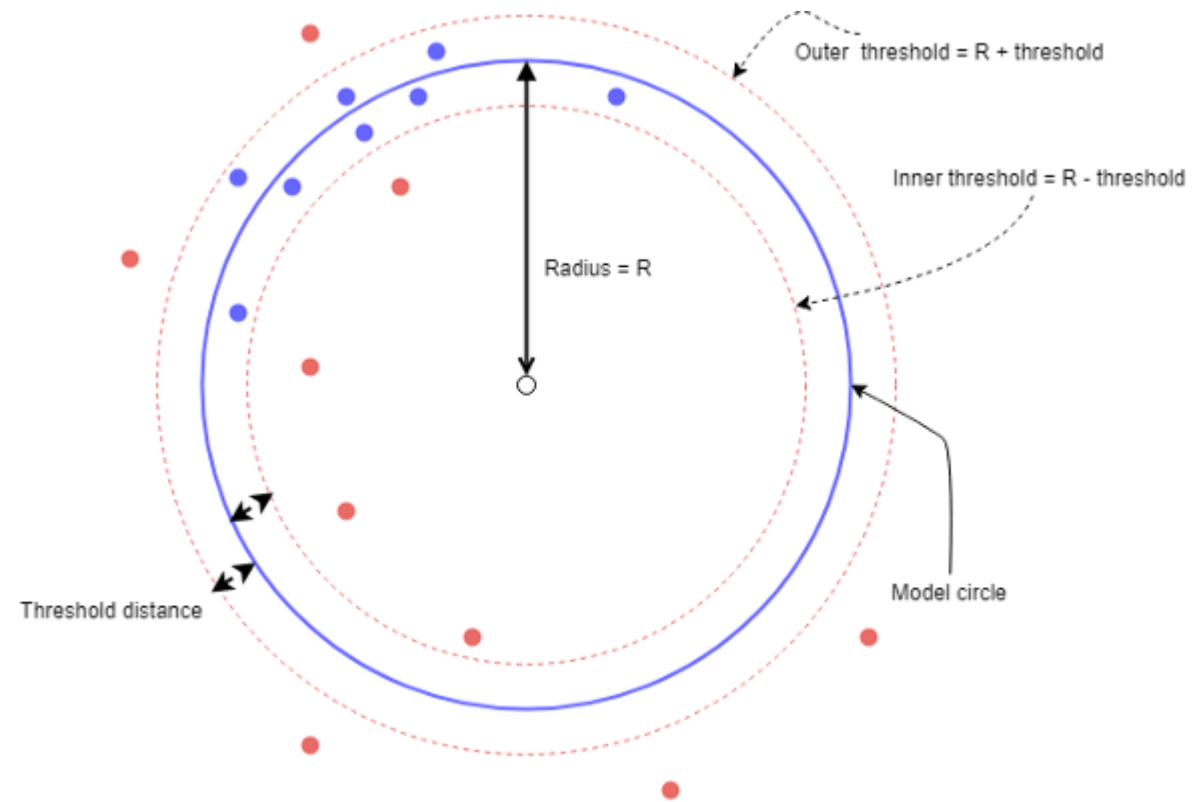
Random Sample Consensus - RANSAC

- česta primjena je detekcija ravnine tla (engl. ground plane) u robotici, autonomnoj vožnji i sl.

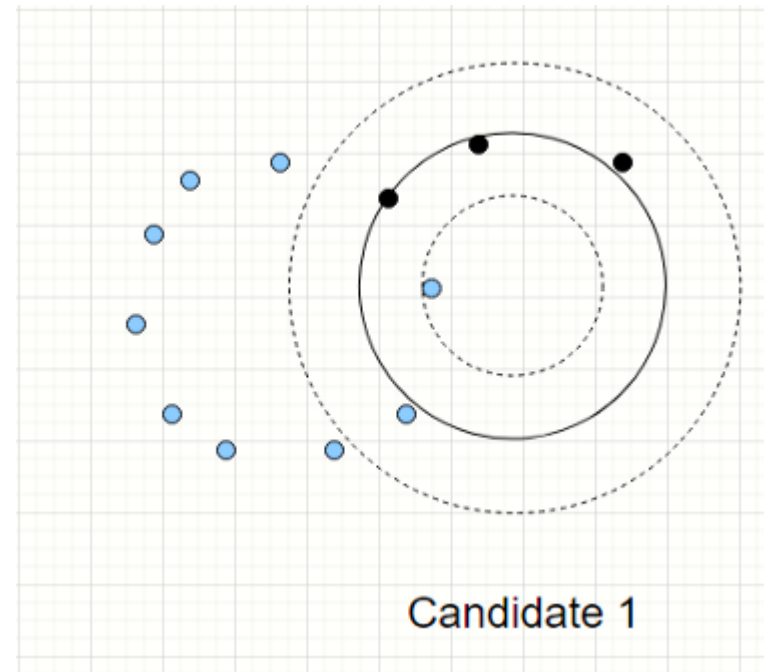
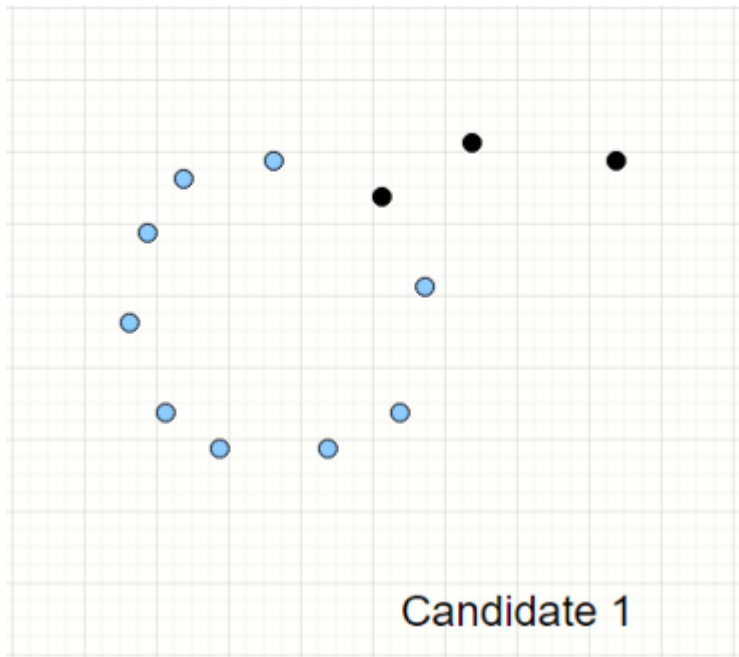


RANSAC za kružnice (krugove)?

- RANSAC se može primijeniti na estimaciju parametara bilo kakvog modela pa tako i za procjenu kružnice
- Kružnica je definirana sa tri parametra: centar(x, y) i radijus (r)
- U svakoj iteraciji potrebno je nasumično odabrati tri podatka
- Slično kao kod pravca, traži se konsenzus set



RANSAC za kružnice(krugove)?

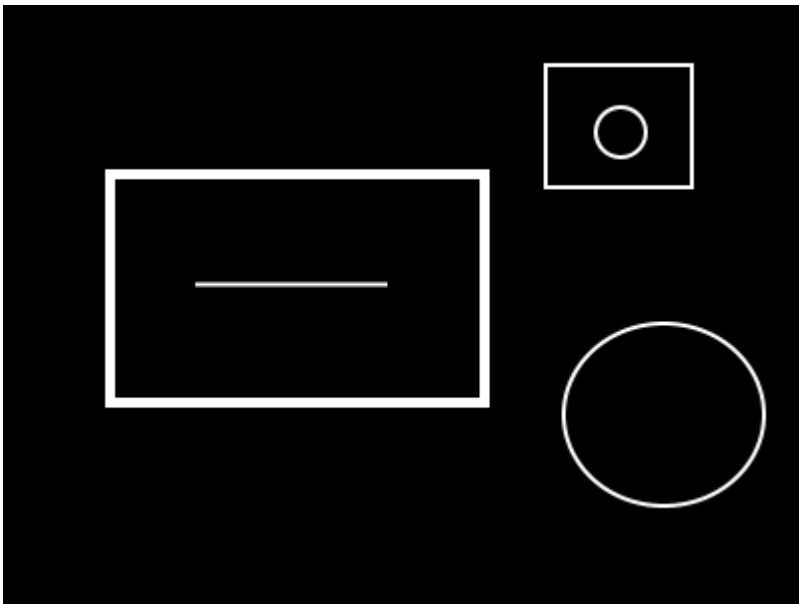


Jedna iteracija

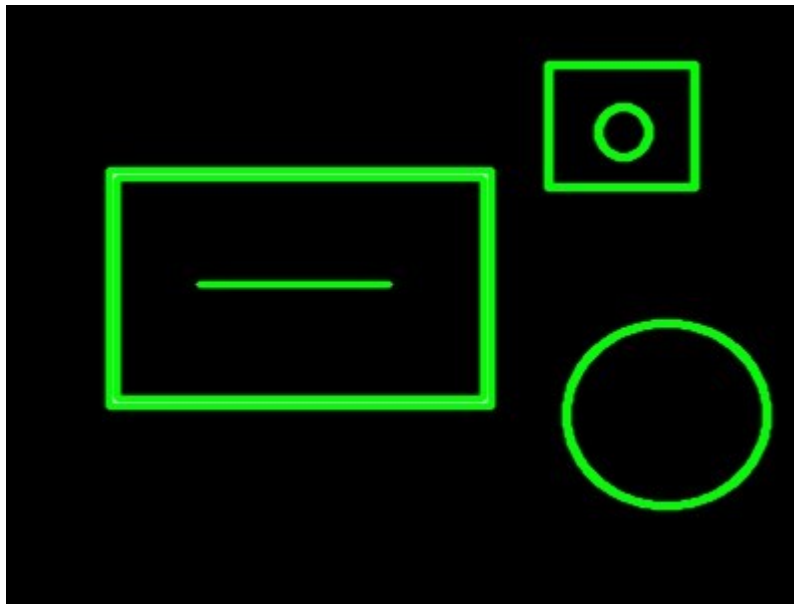
OpenCV detekcija kontura

- konturom se može smatrati krivulja koja povezuje sve točke nekog ruba
- može se dobiti složeniji oblik od pravca odnosno ravne linije
- ulaz je binarna slika (npr. na sliku se najprije primijeni detekcija rubova ili samo thresholding)
- OpenCV pruža dvije funkcije:
 - `cv2.findContours()`
 - `cv2.drawContours()`
- pronađeni rub odnosno kontura se može spremiti:
 - u obliku svih točaka ruba koji pripadaju konturi
 - aproksimirati pomoću horizontalnih, vertikalnih i dijagonalnih linija -> manji memorijski zahtjevi
 - mogući su i različiti modovi traženja konture (sve, samo vanjska, odnosi među konturama)

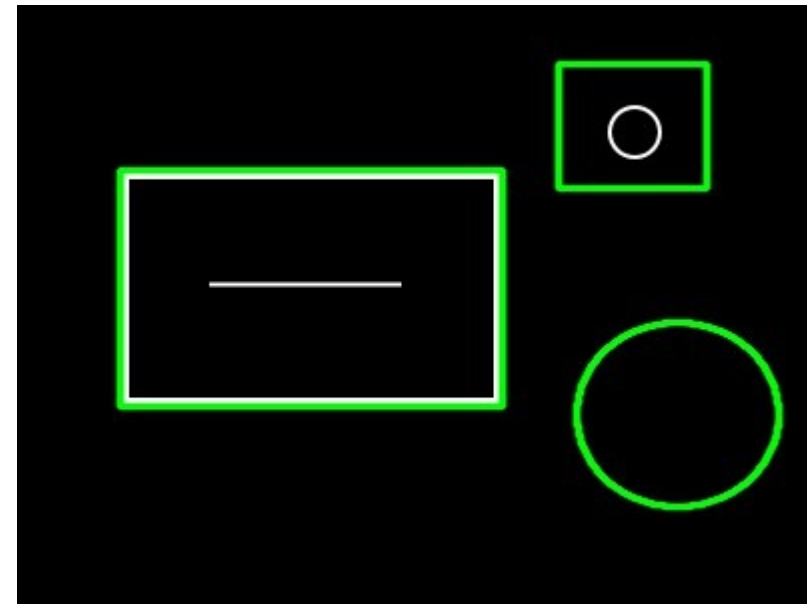
OpenCV detekcija kontura - primjer



Ulazna slika



mode = `cv2.RETR_TREE`



mode = `cv2.RETR_EXTERNAL`

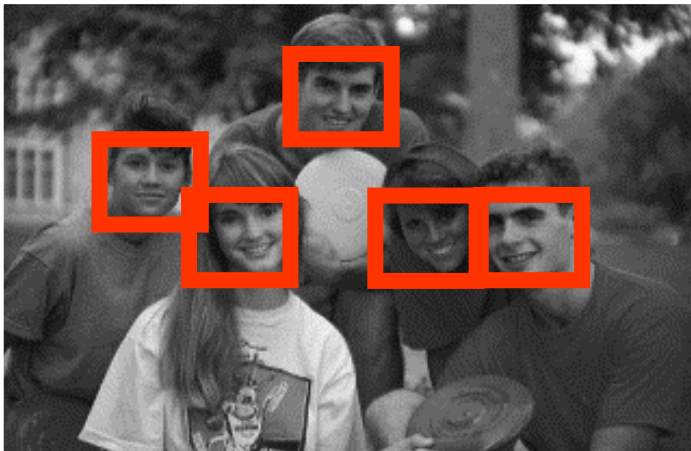
Detekcija objekata

Detekcija vs. identifikacija objekata

Prepoznavanje objekata

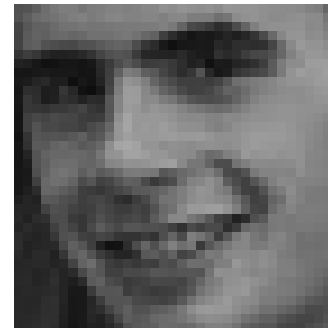
Detekcija objekata

Ima li ljudskih lica na slici?
Gdje je lice?



Identifikacija objekata

Tko je to?
Je li to Ana ili Luka?



Detekcija objekata

- Detekcija objekata je postupak detekcije područja u slici (ili video snimci) koja imaju određena semantička značenja
 - ADAS → prometni znak, semafor, pješak, biciklist, automobil, ...
- Klasifikacija objekata – jedan od osnovnih problema u računalnom vidu
- Što računalo „vidi“ - matricu koja je popunjena brojevima $[0,255]$ - postoji „semantički gap“



16	13	10	15	0
16	16	16	10	0
15	12	0	0	0
0	5	9	15	0
16	3	12	11	0
0	8	14	16	12
15	3	8	13	0
13	0	3	15	1
11	15	16	16	0
1	0	9	15	0
13	15	1	0	0
0	0	11	16	4

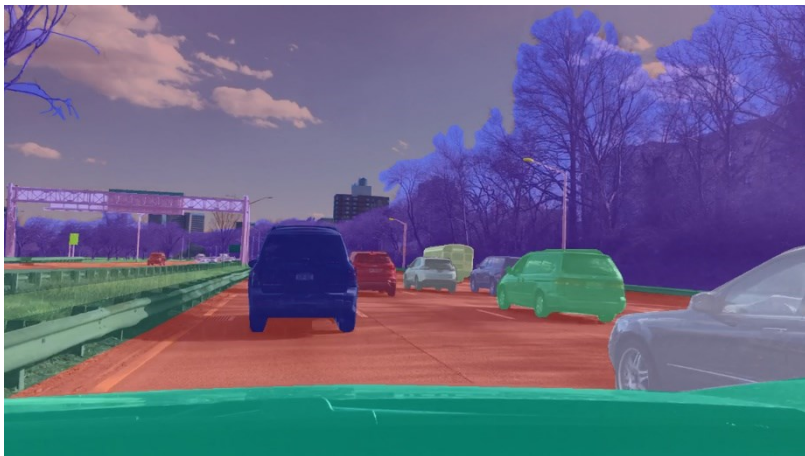
Detekcija objekata

- Izazovi?
 - Varijacije u točki snimanja
 - Varijacije u osvjetljenju i skaliranju
 - Deformacije objekata
 - Okluzija
 - Varijacije unutar izgleda objekta
 - Velik broj različitih objekata
 - ...



Detekcija objekata u autonomnoj vožnji

Segmentacija



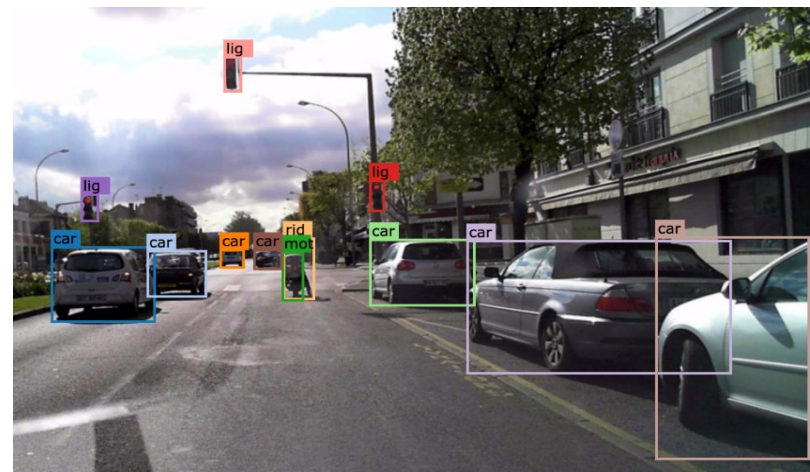
Vozno područje



Detekcija oznaka na kolniku



Detekcija objekata u prometu



Detekcija objekata

- Prethodi mu postupak segmentacije objekata, odnosno lokalizacija objekata i definiranje granica objekata
- Mjera kvalitete detekcije → Intersection over Union (IoU)
 - BBgt - područje objekta od interesa definirano kao ground truth (zeleni pravokutnik na slici)
 - BBp – područje koje je detekcijom određeno kao moguće područje objekta od interesa (crveni pravokutnik na slici)



$\text{IoU} = 0.7$



$\text{IoU} = 0.95$

Detekcija objekata korištenjem predloška

- Usporedba dijelova slike s predloškom (engl. template matching)
- Problemi
 - Skaliranje
 - Rotacija
 - Promjena osvjetljenja
 - Okluzije
- moguće koristiti npr. u industriji – konstantno osvjetljenje, fiksna udaljenost kamere itd.
- u OpenCV funkcija `cv2.matchTemplate`



Detekcija objekata korištenjem predloška

```
import cv2
import numpy as np

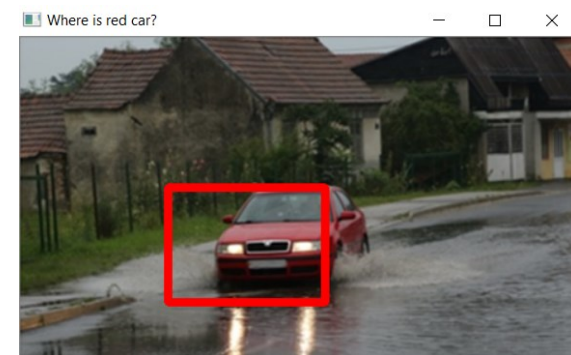
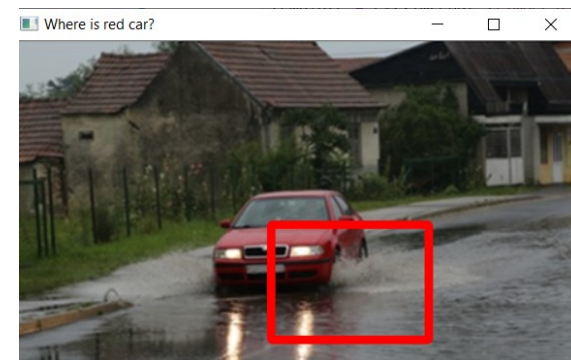
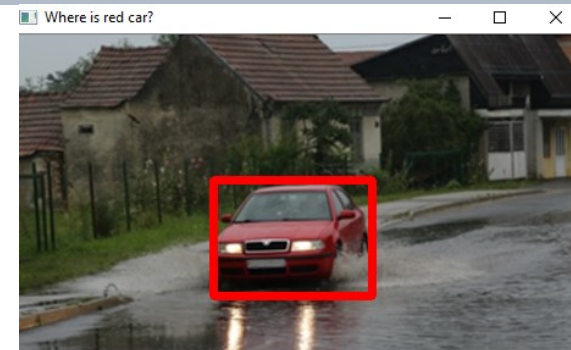
# Load input image and convert to grayscale
image = cv2.imread('./images/find_a_car.jpg')
cv2.imshow('Where is red car?', image)
cv2.waitKey(0)
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Load Template image
template = cv2.imread('./images/red_car.jpg', 0)
# template = cv2.imread('./images/red_car1.jpg', 0)
# template = cv2.imread('./images/red_car_rotated.jpg', 0)

result = cv2.matchTemplate(gray, template, cv2.TM_CCOEFF)
min_val, max_val, min_loc, max_loc = cv2.minMaxLoc(result)

# Create Bounding Box
top_left = max_loc
bottom_right = (top_left[0] + 130, top_left[1] + 95)
cv2.rectangle(image, top_left, bottom_right, (0, 0, 255), 5)

cv2.imshow('Where is red car?', image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



Potrebna je smislenija metoda koja će funkcionirati za razne uvjete → npr. izdvajanje značajki

Izdvajanje značajki

Izdvajanje značajki

- za identifikaciju objekta od interesa prvo se izdvajaju značajke slike koje su karakteristične za objekt koji treba detektirati
- Značajke moraju imati sljedeće karakteristike
 - Ponovljivost – mogu se izračunati za različite slike iste scene
 - Posebnost – dovoljno se razlikuju jedna od druge
 - Učinkovitost – sadrže manje informacija od cjelokupne slike te ih se može koristiti i u vremenski kritičnim aplikacijama
 - Lokalizirane - zauzimaju manje i ograničeno područje na slici, te su otporne na varijacije skale i orijentacije, te djelomično zaklanjanje objekata (okluziju)
 - Precizne – moguće ih je precizno odrediti i za male objekte

Izdvajanje značajki

- Značajke (ključne točke) – zanimljiva područja u slici koja su po nečemu jedinstvena za tu specifičnu sliku
- One su u biti sažetak slike
- Ako imamo neku sliku sa sličnim objektom, vjerojatno će biti pronađene slične ključne točke koje će se moći upariti s ključnim točkama s referentne slike



Izdvajanje značajki



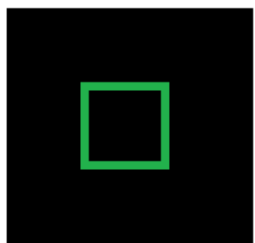
Izdvajanje značajki

- koje biste komadiće slike u ovom slučaju odabrali kao nekakve ključne točke?
- možete li locirati svaki komadić na slici i koliko precizno?
- A i B teško, C i D otprilike, E i F točno znamo gdje su

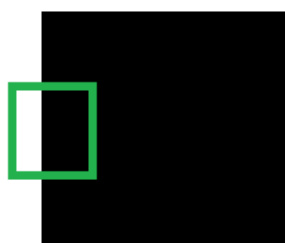


Izdvajanje značajki

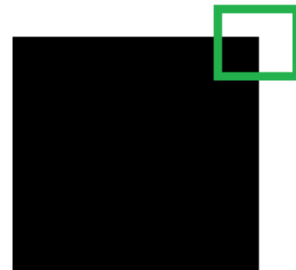
- Što definira tu „zanimljivost“ da bi nešto bila ključna točka?
 - Mnoštvo različitih i jedinstvenih informacija u danoj točki
 - Jaka promjena intenziteta
 - Rubovi
 - Uglovi
 - ...



Ravno područje - nema promjene ni u jednom smjeru

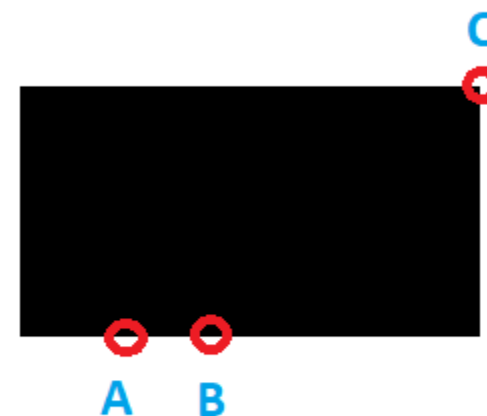


Rub - promjena u jednom smjeru



Ugao - promjena u svim smjerovima

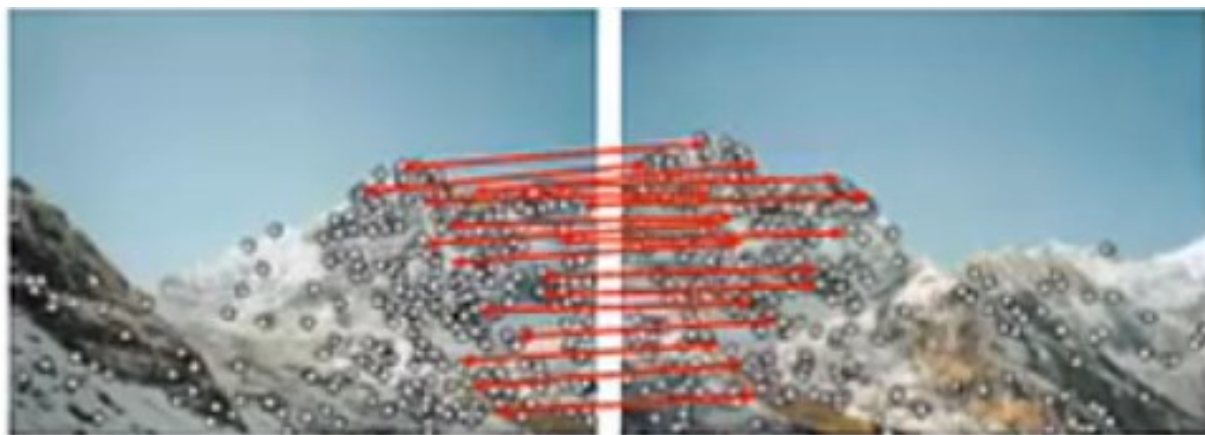
Interesantna i jedinstvena
(relativno u odnosu na svoju okolinu)



Interesantne, ali nisu jedinstvene
(relativno u odnosu na svoju okolinu)

Izdvajanje značajki

- Zašto su značajke važne?
 - Mogu biti korištene za analizu, opis i uparivanje slika
- Koriste se za
 - Poravnanje slika - npr. za dobivanje jedinstvenog panoramskog prikaza povezivanje slika s više kamera u jednu sliku



Izdvajanje značajki

- Gdje nam to može biti korisno u vozilu?
- Spajanje slike s više kamera i pokrivanje „mrtvog kuta“

Ostaje problem ujednačavanja osvjetljenja na mjestima spoja



Okvir sa srednje kamere



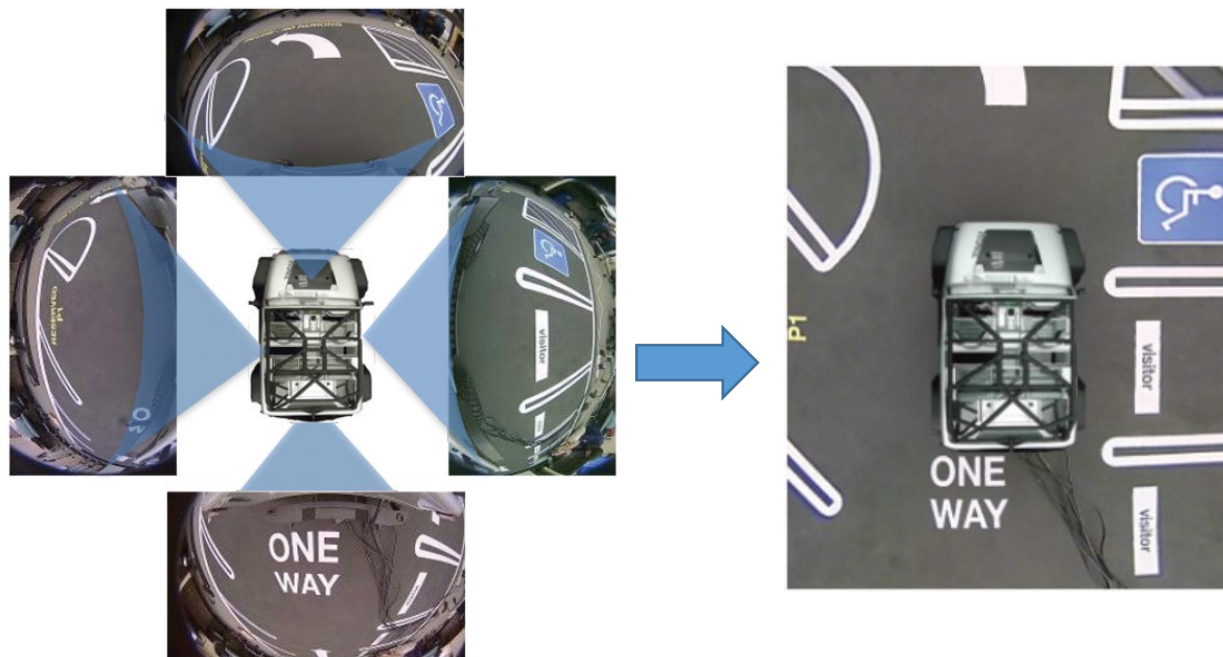
Transformirani okvir s desne kamere



Spojeni okviri sa srednje i desne kamere

Izdvajanje značajki

- Koriste se za
 - Poravnanje slika
 - pogled na vozilo odozgor na osnovu slika s 4 kamere oko vozila (top view, surround view)
 - 3D rekonstrukciju scene
 - Navigaciju robota
 - Prepoznavanje objekata
 - Praćenje objekata



Izdvajanje značajki – Harris detekcija uglova

- **cv2.cornerHarris(input image, block size, kernel size, k)**
 - input image – mora biti grayscale i float32
 - block size – veličina okoline koja se razmatra za detekciju ugla
 - kernel size – veličina Sobelovog kernela
 - k - Slobodni parametar Harris detektora

Izdvajanje značajki – Harris detekcija uglova

```
import cv2
import numpy as np

# Load image then grayscale
image = cv2.imread('images/chess.jpg')
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

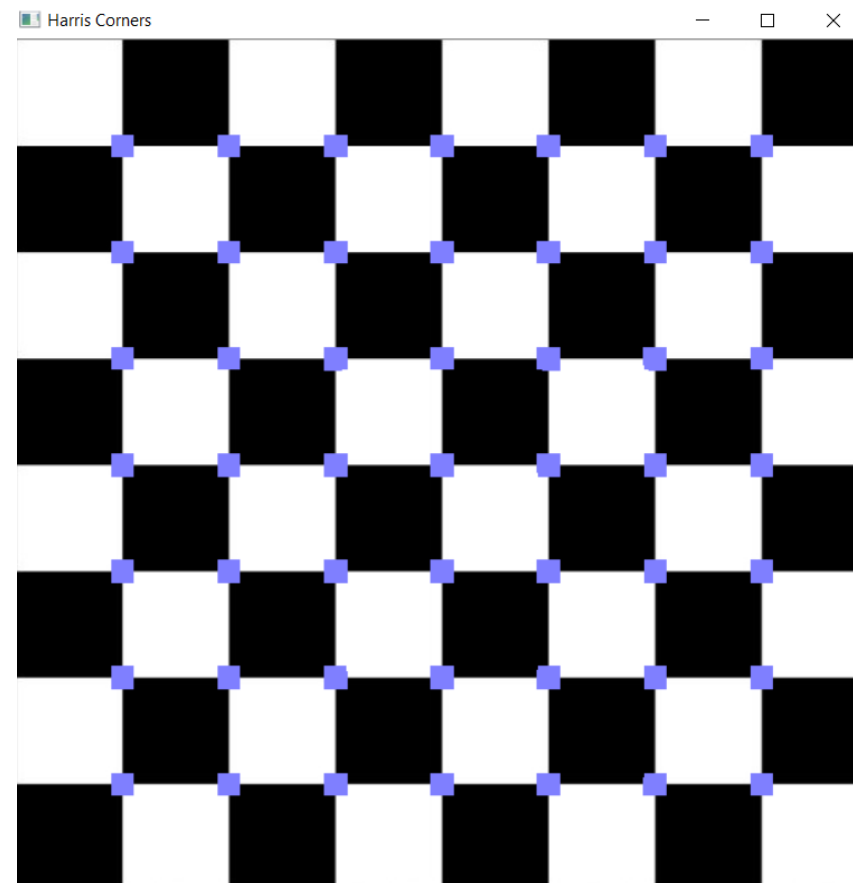
# The cornerHarris function requires the array datatype to be float32
gray = np.float32(gray)

harris_corners = cv2.cornerHarris(gray, 3, 3, 0.05)

#We use dilation of the corner points to enlarge them\
kernel = np.ones((7,7),np.uint8)
harris_corners = cv2.dilate(harris_corners, kernel, iterations = 2)

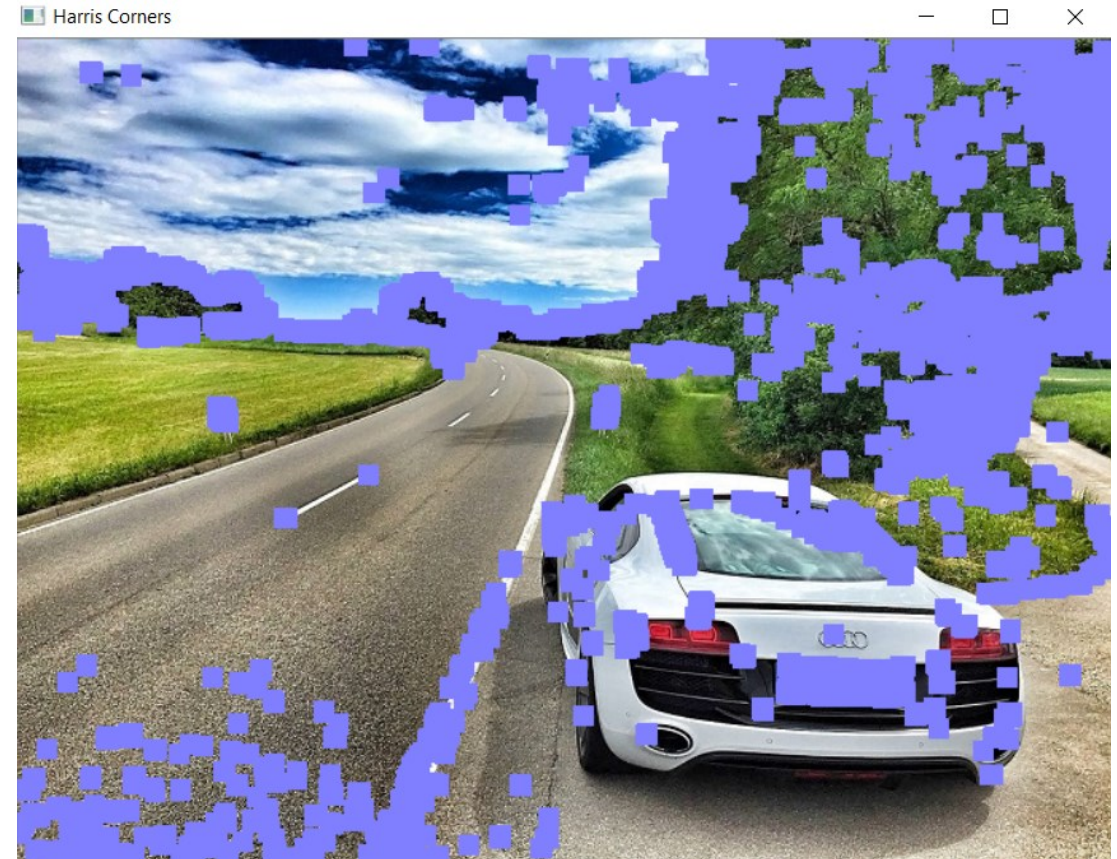
# Threshold for an optimal value, it may vary depending on the image.
image[harris_corners > 0.025 * harris_corners.max() ] = [255, 127, 127]

cv2.imshow('Harris Corners', image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



Izdvajanje značajki – Harris detekcija uglova

- Može li se uvijek koristiti?
 - Ne baš uvijek
- Potrebna predobrada
 - Blurring
 - Uklanjanje šuma
- Podešavanje parametara funkcije



Izdvajanje značajki – Good Features to Track

- Good Features to Track – metoda za detekciju uglova
 - Poboljšanje Harris metode za detekciju uglova
 - Vraća (x,y) koordinate uglova
- **cv2.goodFeaturesToTrack (input image, maxCorners, qualityLevel, minDistance)**
 - input image – 8-bitna ili floating-point 32-bitna slika, jedan kanal
 - maxCorners – maksimalan broj uglova koji treba vratiti – ako je veći broj uglova pronađen, vraćaju se najjači od njih
 - qualityLevel – parametar koji specificira minimalno prihvatljivu kvalitetu ugla. Množi se s najboljom kvalitetom ugla. Uglovi koji imaju mjeru kvalitete manju od tog produkta, odbacuju se.
 - minDistance – minimalna dozvoljena Euklidska udaljenost između bilo koja dva ugla

Izdvajanje značajki – Good Features to Track

- Good Features to Track – metoda za detekciju uglova u OpenCV

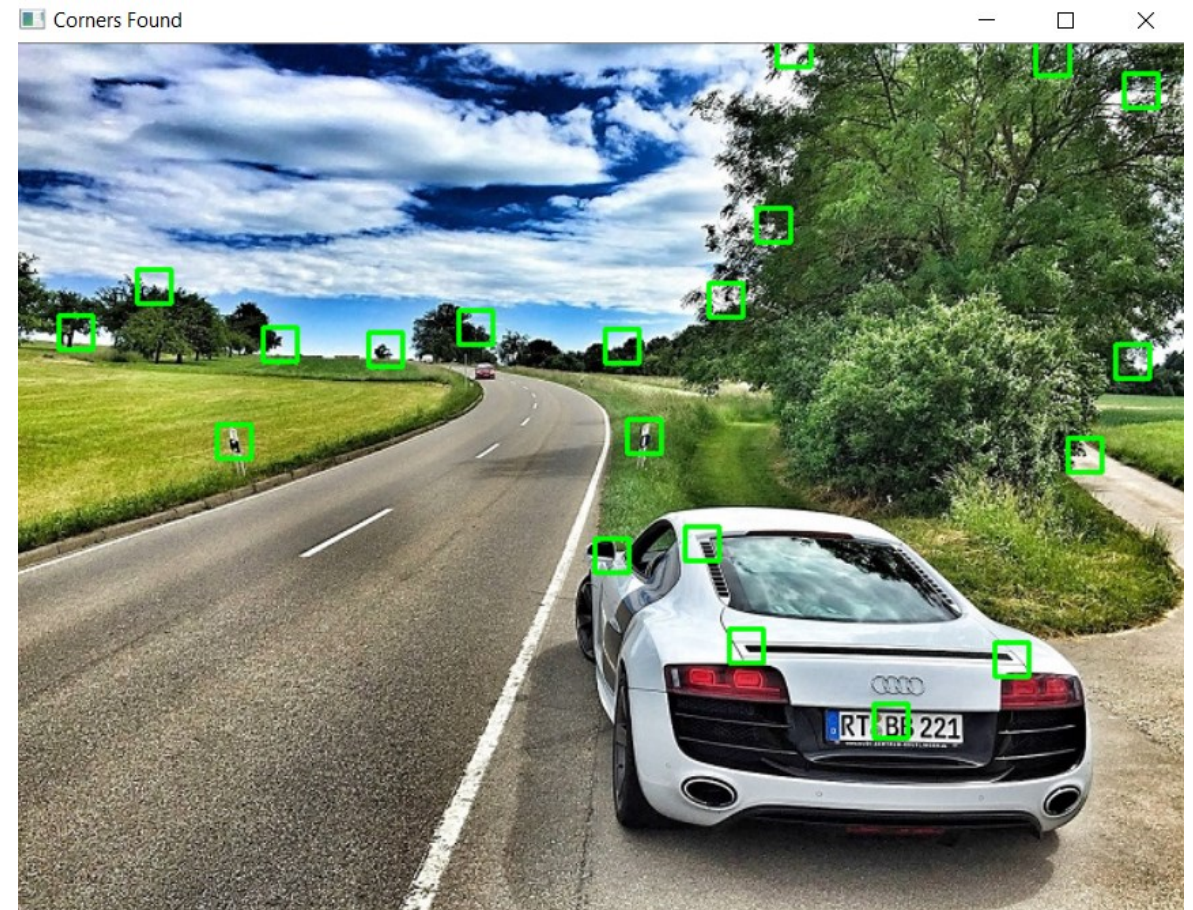
```
import cv2
import numpy as np

img = cv2.imread('images/car_and_road.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# We specify the top 50 corners
corners = cv2.goodFeaturesToTrack(gray, 20, 0.01, 50)

for corner in corners:
    x, y = corner[0]
    x = int(x)
    y = int(y)
    cv2.rectangle(img, (x-10, y-10), (x+10, y+10), (0, 255, 0), 2)

cv2.imshow("Corners Found", img)
cv2.waitKey()
cv2.destroyAllWindows()
```



Izdvajanje značajki – sparivanje uglova

- Izjednačavanje (sparivanje) uglova u slikama tolerantno je na
 - Rotaciju
 - Translaciju (pomake u slici)
 - Lagane fotometrijske promjene npr. osvjetljenja
- Izjednačavanje (sparivanje) uglova u slikama nije tolerantno na
 - Značajne fotometrijske promjene, npr. osvjetljenja
 - Skaliranje
 - Npr. povećanje rezultira u tome da više uglova bude detektirano (detektiraju se često rubovi kao uglovi)



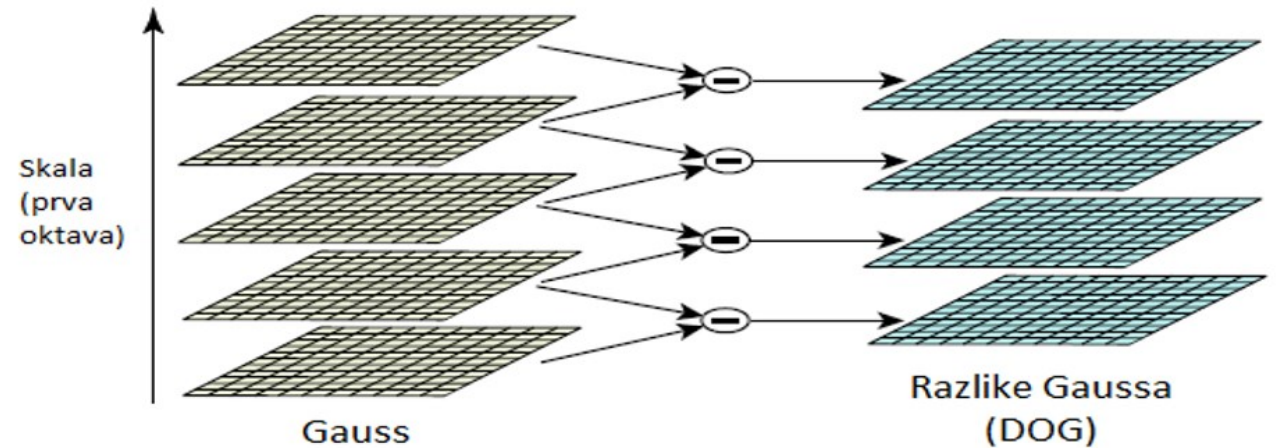
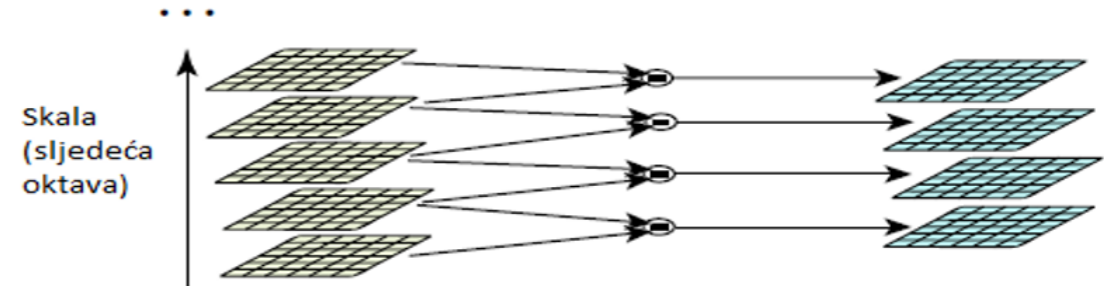
Izdvajanje značajki – naprednije metode

- Razvijen je veliki broj složenijih metoda za izdvajanje značajki
- Neke od metoda:
 - SIFT - Scale Invariant Feature Transform
 - SURF – Speeded Up Robust Features
 - FAST – Features from Accelerated Segmentation Test
 - BRIEF – Binary Robust Independant Elementary Features
 - ORB – Oriented FAST and Rotated BRIEF
 - HoGs – Histogram of Oriented Gradients

Izdvajanje značajki – razlika Gaussiana (DoG)

- Razlika Gaussiana – DoG algoritam → jedan od načina kako postići invarijantnost na skalu
 - Prvo se radi konvolucija ulazne slike s Gausovim filtrom na različitim skalama i različitim veličinama Gaussovog filtra (različita standardna devijacija)
 - Različite skale se postižu poduzorkovanjem slike u svakom koraku mijenjanja skale
 - Sljedeći korak je proračun razlika između dva rezultata ovih konvolucija na istoj skali
 - Na rezultat se primjenjuje prag te se elementi slike za koju je DoG veći od praga određuju kao područja od interesa
 - Ova metoda osigurava invarijantnost značajki na promjenu skale i orijentaciju
 - Bolja je za segmentaciju od detekcije rubova

Izdvajanje značajki – razlika Gaussiana (DoG)



Izdvajanje značajki – razlika Gaussiana (DoG)

Gaussov filter 5x5



Gaussov filter 3x3



Razlike Gaussiana (DoG)



Gaussov filter 7x7



Gaussov filter 5x5



Izdvajanje značajki – SIFT

- **Scale Invariant Feature Transform**
 - Algoritam za detekciju ključnih točaka i određivanje pripadajućih deskriptora
 - Određuje ključne točke pomoću gradijenata
 - Dobivene ključne točke su otporne na
 - Skaliranje (promjene u veličini)
 - Rotaciju
 - Osvjetljenje
 - Perspektivu slike,
 - ...što ih čini povoljnim za detekciju objekata kod dinamičnih slika

Izdvajanje značajki – SIFT

1. Detekcija ključnih točaka

- Primjenom razlike Gaussiana – DoG (različite skale se postižu poduzorkovanjem slike u svakom koraku mijenjanja skale) određuju se potencijalne ključne točke
- Pretraživanje na više skala dovodi do invarijantnosti na skaliranje
- To su područja slike gdje varijacije premašuju određeni prag i bolja su od deskriptora ruba

2. Lokalizacija ključnih točaka

- Proračun maksimuma i minimuma DoG tako da se svaki element uspoređuje s 8 susjednih elementa na istoj skali te sa po 9 elemenata na višoj i nižoj skali – bira se za ključnu točku ako je manji/veći od svih promatranih susjeda
- Izbor stabilnih/sigurnih ekstrema na određenoj skali (primjena Taylorovog reda i Hessian matrice za bolju lokalizaciju ekstrema) → za njih se kasnije radi vektor deskriptor → invarijantnost na skaliranje

Izdvajanje značajki – SIFT

3. Određivanje orijentacije

- Primjenom magnitude gradijenta određuje se jedna ili više orijentacija ključnih točaka
- Formira se histogram orijentacija od 36 intervala, te se za orijentaciju uzima onaj smjer koji u histogramu ima maksimum te svi iznad 80% maksimuma
- Informacija o orijentaciji osigurava invarijantnost na rotaciju

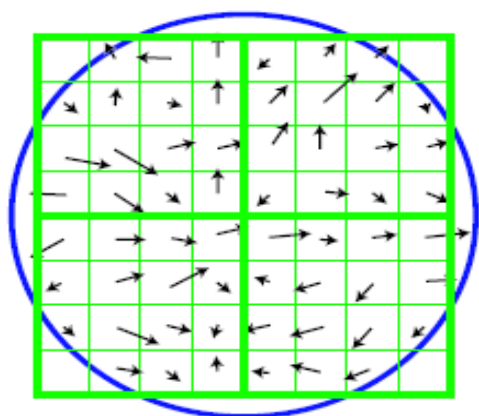
4. Oblikovanje deskriptora

- Mjere se lokalni gradijenti slike, te se formira vektor deskriptor za svaku točku s informacijom o skali, lokaciji i orijentaciji

Izdvajanje značajki – SIFT

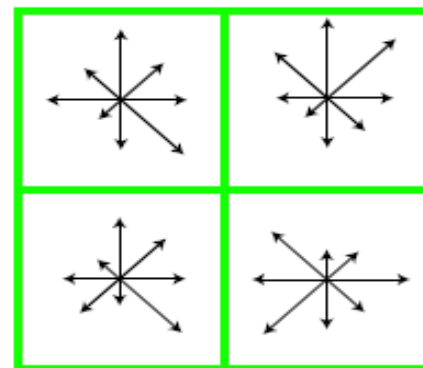
4. Oblikovanje deskriptora

- 16x16 susjedstvo ključne točke se dijeli u 16 pod-blokova veličine 4x4
- Za svaki pod-blok se formira histogram s 8 intervala (bin-ova)
- Za svaku ključnu točku to čini vektor od 16 x 8 binova koji predstavlja deskriptor te ključne točke



Gradijenti slike

Orijentacije gradijenata za 8x8
susjedstvo ključne točke



Deskriptor ključne točke

Histogrami pod-blokova s 8 binova

Ilustracija
oblikovanja
deskriptora na 8x8
susjedstvu ključne
točke

Izdvajanje značajki – SIFT

5. Usporedba značajki referentne i analizirane slike

- Deskriptori ključnih točaka dviju slika se uspoređuju na temelju njihove Euklidske udaljenosti
- Za detekciju podudaranja ključnih točaka koriste se najbliži deskriptori, ali i drugi-najbliži deskriptor što povećava robusnost metode
- Odluka da se ključne točke referentne i analizirane slike podudaraju donosi se ako je omjer udaljenosti najbližeg deskriptora i drugog najbližeg deskriptora manji od 0.8

Izdvajanje značajki – SIFT

```
import cv2
import numpy as np

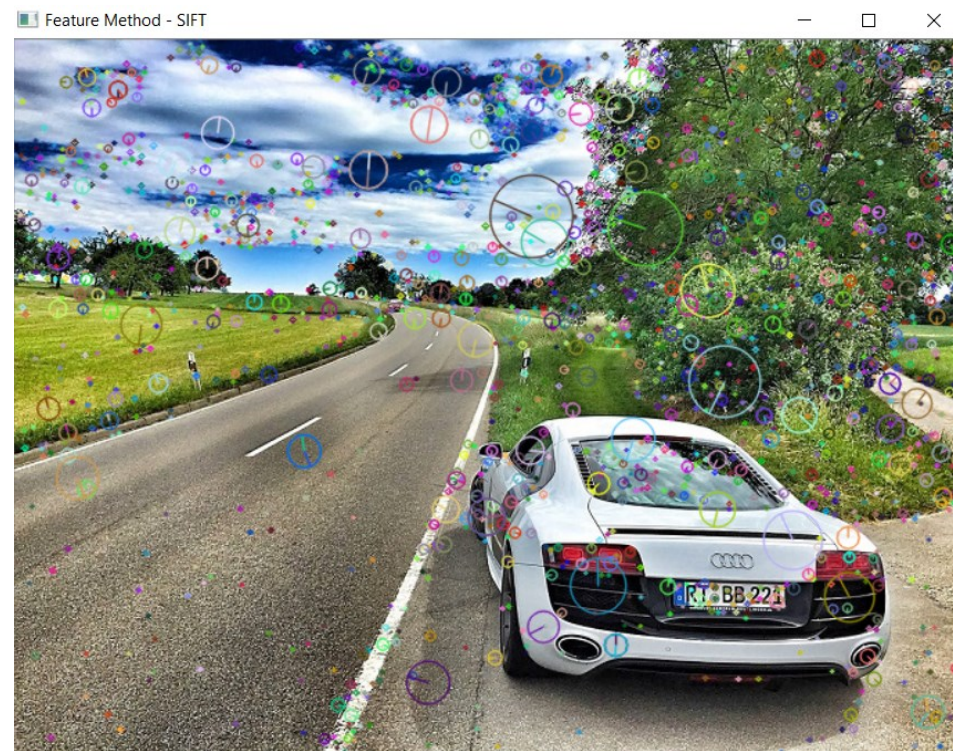
image = cv2.imread('images/car_and_road.jpg')
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

#Create SIFT Feature Detector object
sift = cv2.SIFT()

#Detect key points
keypoints = sift.detect(gray, None)
print("Number of keypoints Detected: ", len(keypoints))

# Draw rich key points on input image
image = cv2.drawKeypoints(image, keypoints, flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)

cv2.imshow('Feature Method - SIFT', image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



Izdvajanje značajki – ostale metode

- SIFT metoda efikasno detektira ključne točke (i objekte) ali je računalno zahtjevna
- SURF – Speeded Up Robust Features
 - Usporediv sa SIFT-om, ali oko 3 puta brži
 - Dobri rezultati na zamućenim slikama i kod rotacije objekata
 - Slabosti SURF-a - osjetljiv na promjene osvjetljenja i na promjenu kuta gledanja
- FAST – Features from Accelerated Segment Test
 - Radi samo detekciju ključnih točaka (nema deskriptora, mogu se koristiti SIFT i SURF da se izračunaju)
 - Koristi se u real-time aplikacijama
- BRIEF – Binary Robust Independent Elementary Features
 - Vrlo brzo računa deskriptore za detektirane ključne točke
 - Brži od SIFT i SURF
- ORB – Oriented FAST and Rotated BRIEF
 - Razvijen od strane OpenCV laboratorija
 - E. Rublee, V. Rabaud, K. Konolige, and G. Bradski, “ORB: An efficient alternative to SIFT or SURF,” in Computer Vision (ICCV), 2011 IEEE International Conference on, 2011, pp. 2564–2571.
 - Invarijantan na rotaciju i ima dobru otpornost na šum
 - Koristi modificiranu FAST metodu (Oriented FAST - oFAST) za detekciju ključnih točaka uglova i BRIEF metodu za oblikovanje deskriptora

Izdvajanje značajki – generalno u OpenCV-u

Kreiranje detektora



Primjena detektora na sliku
– dobivanje ključnih točaka



Oblikovanje deskriptora
ključnih točaka



Prikaz ključnih točaka

```
import cv2
import numpy as np

image = cv2.imread('images/car_and_road.jpg')
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Create ORB object, we can specify the number of key points we desire
orb = cv2.ORB(100)

# Determine key points
keypoints = orb.detect(gray, None)

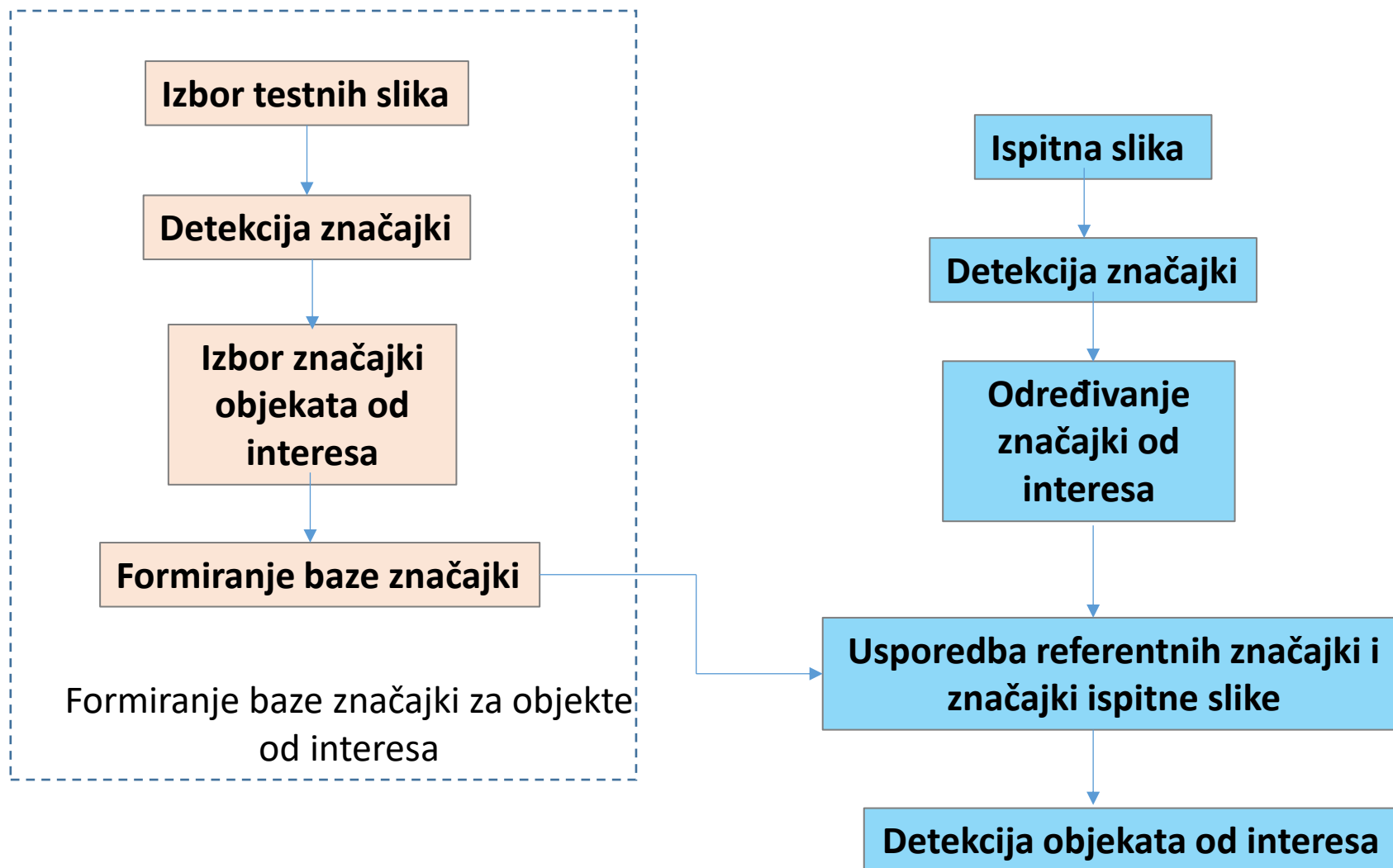
# Obtain the descriptors
keypoints, descriptors = orb.compute(gray, keypoints)
print("Number of keypoints Detected: ", len(keypoints))

# Draw rich keypoints on input image
image = cv2.drawKeypoints(image, keypoints, flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)

cv2.imshow('Feature Method - ORB', image)
cv2.waitKey()
cv2.destroyAllWindows()

('Number of keypoints Detected: ', 100)
```


Detekcija objekata primjenom značajki



Detekcija objekata - primjer ORB



Slika objekta od interesa



Ispitna slika

Detekcija objekata - primjer ORB

Detektirane ključne
točke na slici objekta od
interesa



Detektirane ključne točke na ispitnoj
slici



Detekcija objekata primjenom značajki



Prvih n uparenih ključnih točaka i detektirani objekt